

F2-REST Documentation

Created on 08/12/2025 04:54 for F2 version 13

Introduction

The F2 REST Service is an open API for third party vendors to interact with F2. Through this service it is possible to read and modify case files, matters, documents, parties and more.

REST [[Fielding](#)] means **R**epresentational **S**tate **T**ransfer and it is a network architecture designed for the web. There is a lot to say about REST (and quite a lot of misconceptions too), but for now, we will just highlight a few of the main points:

- All resources in the system are given their own uniquely identified URL. This means that any given piece of data in F2 is given its own unique URL where it can be retrieved from.
- Resources have one (or more) externally visible representations which are decoupled from the actual implementation in F2.
- Resources can be read and modified using the standard HTTP verbs (GET, POST, PUT, DELETE, PATCH and so on).
- Resources are linked together using hypermedia. A case file contains links to its child matters, a document contains a link to its creating party and so on.
- There is no up-front documentation of the URL structures. Instead we document link relations – the names of the links. So for instance we name the link from a case file to its child matters as "down" and it is then up to the client to extract the relevant link and its target URL at runtime.
- The service has a central service index which contains a list of links to all the top-level resources. The URL of the service index is the only concrete URL documented. The client must then extract the required links from the service index using the link relation names published in this documentation.

Getting started

The easiest way to get acquainted with the service is to point a browser to the service index and start exploring the capabilities from there. The server returns XML with an embedded reference to an XML style sheet which will transform the XML to nicely formatted HTML if the browser supports it.

Authentication

The service supports OAuth2 authentication.

OAuth2 avoids storing F2 user passwords on the client machine.

The supported OAuth2 flows are "Resource Owner Password Credentials Grant" (mostly used with background processing services), "Authorization Code Grant" (mostly used with end user facing applications) and "Client credentials grant" (which is rarely used).

It is furthermore possible to get information about the authenticated user with OpenID Connect [[OIDC](#)].

Searching

Searching is supported by the use of OpenSearch [[OpenSearch](#)] documents and F2 custom queries. See [F2-REST Documentation](#) and [F2-REST Documentation](#).

Structured searching by case numbers, personal identification numbers and such is also possible as described in [F2-REST Documentation](#) and [F2-REST Documentation](#).

Updating data

Updating of existing resources is supported via the HTTP verb PATCH using JSON patch documents [[JSONPatch](#)].

Software developers kit (SDK)

cBrain has a small SDK for developing C# solutions that interacts with the F2-REST service. The SDK consists of this manual, a set of classes representing the various F2 resources and some example code.

Please contact cBrain for further information about this product.

Conceptual model

At the very center of F2 we find the "Digital Archive" — a place to store case files, matters and related documents. In this chapter we give a short introduction to those elements that are exposed through F2-REST — for a more detailed explanation we recommend reading other parts of the F2 manuals.

Case files, matters and documents

The logical structure of a case file consists of digital documents organized in logical documents called "matters"^[1] which again are bundled together in case files:

```
Case file =contains many=> Matters =contains many=> Documents
```

Digital documents can be of any kind of file format such as Office files, PDF files, zip files, images and so on.

A matter often corresponds to some kind of communication — typically an e-mail — with attached files stored as documents on the matter.

Parties

An important part of case management is the parties related to a case or other object. A "party" is a broad concept that represents a person, an organization, a company or some other legal entity.

Parties are related to other F2 objects in various ways; a party may be the "owner" of a case file, the "responsible" for a matter, the "creator" of a document or the "receiver" of an e-mail and so on.

In F2 we distinguish between two different versions of a party — the registry version and the original version. The registry version of a party represents its up-to-date current state whereas the original version represents the party at the time it was associated with a specific entity in F2 — for instance when assigned as the receiver of an e-mail.

Notes

A note is an informal way to communicate in F2. A note can be attached to either a case or a matter. The note is visible to all parties who have read access to the given entity to which the note is attached. A note can have an arbitrary number of text entries, each created by any person who has access to the note.

In the case of notes on matters, a number of participants can be added. This results in the matter being marked as unread for the participants and thus shown in their inbox whenever a new entry is made.

1. "Matter" as in "legal matters" ("Akt" in Danish).

Chats

A chat is an informal way to communicate in F2. A chat can be attached to a matter. At any time, a chat has a number of assigned participants. A chat can have an arbitrary number of text entries, each created by any participant of the chat. The chat is only visible to parties who are assigned as participants of the chat.

A number of participants can be added dynamically to the chat. This results in the matter becoming visible (with read access) and marked as unread for the new participants and thus shown in their inbox whenever a new entry is made.

Participants can also be removed dynamically. This results in the matter being inaccessible for the removed participants unless they are granted access by other means.

Approvals

Approvals represent the status of approval processes associated with matters.

Task guides

Task guides in F2 represents task lists grouped in processes. Tasks can be completed (marked as complete) as well as "executed" which corresponds to invoking the main operation of a task. For an in-depth explanation of task guides, please see other appropriate F2 manuals.

In F2-REST it is possible to read the whole process/task tree with the current visibility and "completeness" of each task and process — as well as it is possible to complete and execute tasks and read/update the values of fields in the task guides.

Extension records and lists

One of the extension points of F2 allows developers to store named records of data together with case files — and such records may further more contain lists of other records (which we will call "rows" to avoid confusion).

Records can be found on case files by looking for the link relations <http://cbrain.com/casefile/rel/extension-records> and <http://cbrain.com/casefile/rel/extension-record-by-name> (see [F2-REST Documentation](#)).

Rows are all identified by a GUID (globally unique identifier) which can be used to reference the rows when doing update/delete/append operations on lists. F2-REST ensures rows are stored and retrieved in the sequence they are appended.

Record names can be chosen freely, but the special name "CaseInfo" is reserved for data stored as F2 process information ("Sagens oplysninger" in Danish).

It is not possible to embed lists in lists - so in the above case we won't be able to store the whole family tree, only one single family.

Custom XML data

Another of the extension points of F2 allows developers to associate raw XML data with existing entities such as case files and matters. The XML document and the entity are associated by a simple string key and there can be only one XML document associated for each key.

A case file may for instance have geographical information in GML XML associated to it by the key "GIS".

F2 places no restriction on the actual XML stored as custom XML data. Neither does F2 imply any specific interpretation of the XML.

Additional properties

Data in F2 can also be enriched by associating collections of simple key/value pairs with various objects (usually parties).

A person (external party) may for instance be related to a job interview and in this context, we need to store the applicant's date of birth. This can be stored as a key/value combination where the key is "DateOfBirth" and the value is the actual date of birth.

Meetings

A meeting is defined by start-time, end-time and similar meeting related meta data. It also holds a list of agenda items with associated documents.

Hypermedia elements

Navigation

Most of the F2 representations contain links to other resources that represent other parts of F2. Links are borrowed from the Atom specification [[ATOM](#)] and have the same set of attributes.

Here is an example of a link leading to a resource for creating matters:

```
<Link href="..."
      rel="http://cbrain.com/casefile/rel/create-matter"
      type="text/html"
      title="Add matter"/ >
```

URL templates

The service includes URL templates in the style of a URL with replacement parameters embedded in curly braces — like for instance <http://some-host/some-path/{x}/{y}> or <http://some-host/some-path?some-parameter={x}>.

URL templates require the client to substitute values for the template parameters and use the result as a URL for further interaction with the service.

Example: the URL template <http://some-host/some-path/{x}/{y}> with `x=1` and `y=abc` becomes: <http://some-host/some-path/1/abc>.

URL templates can be found in the service index (see [Service index](#)) and MUST be discovered at runtime — URLs should never be hard coded in any client code.

Service index

The service index contains links to all the top level resources in the service. It can be represented in JSON (`application/json`), ATOM (`application/atom+xml`) or HTML (`text/html`).

The JSON representation is a single object that maps from link relation names to actual links:

Example snippet from service index as JSON

```
{
  "http://cbrain.com/casefile/rel/create-meeting":
  {
    "href": "http://...",
    "title": "Create meeting"
  },
  "http://cbrain.com/casefile/rel/oauth2-token":
  {
    "href": "http://...",
    "title": "Token endpoint for OAuth2"
  }
}
```

The HTML version uses anchor elements `<a>` to represent links and the "rel" attribute for identifying link relations:

Example snippet from service index as HTML

```
<html xmlns="http://www.w3.org/1999/xhtml">
<body>
  <dl>

    <dt>
      <a href="http://localhost/f2-restservice-4.3/meeting-create"
        rel="http://cbrain.com/casefile/rel/create-meeting">Create meeting
      </a>
    </dt>
    <dd>URL for creating new meetings.</dd>

    <dt>
      <a href="http://localhost/f2-restservice-4.3/oauth2/token"
        rel="http://cbrain.com/casefile/rel/oauth2-token">OAuth2 Token endpoint
      </a>
    </dt>
    <dd>Token endpoint for OAuth2 according to RFC 6749</dd>

  </dl>
</body>
```

The client **MUST** read the service index at runtime and use it to locate the links it needs.

Some elements in the service index are URL templates (see [Hypermedia and URL templates](#)) and require further handling by the client before use.

Date and time

Date and time fields in F2 are generally stored without any kind of time zone indication — and it is the responsibility of the client application to apply the right time zone transformations when reading and writing data in F2. This is normally taken care of automatically by the desktop client and other standard cBrain applications — but with F2-REST the client application must be aware of this issue.

This means the client must know the time difference between the server's UTC offset and its own UTC offset — and apply this offset when reading and writing dates and times.

The recommendation for dates like a deadline is to leave out any time-part and set it to 00:00:00.

Daylight savings

Clients using F2-REST should be aware of daylight savings and take this into account when reading and writing dates and times. If the client and server operate in the same time zone then there is nothing special to do as both will change their UTC time zone offset simultaneously — otherwise the client must take daylight savings into account.

Most application programming languages, like for instance C#, have built-in functions for applying time zone differences when server and client time zones are known.

Usually it is not necessary to handle daylight savings in any specific way, as F2 works with dates only and no time-part in all input fields (such as a deadline).

Be aware that some date and time combinations are invalid — like when skipping from 02:00 to 03:00 when changing to summertime — and some combinations occur twice when skipping from 03:00 to 02:00. Similarly, some days last 23 hours only while others last 25 hours. Never try to add days by simply adding 24 hours!

String format

The standard string format is `YYYY-MM-DD'T'HH:MM:SS` with no time zone designator. This corresponds to the ISO 8601 date time format with unqualified local time zone. Leaving out the time zone designator ensure that the F2-REST server accepts the value without any server-side transformations.

It is although possible to include time zone designators but the actual result is undocumented and unsupported as it goes through various server side transformations that are difficult to control.

XML format for extension records

When working with extension records for task guide data, date and time values are represented using standard XML formatting rules. This means the value must have the attribute `xsi:type` set to `"xsd:dateTime"` and use the format `YYYY-MM-DD'T'HH:MM:SS+/-hh:mm`.

Extension records are, unlike standard F2 fields such as a matter's deadline, stored as-is in an XML blob in the database. The only exception here is that date and time values without a time zone designator will be assigned the time zone offset of the server.

Beware that the desktop client's date and time handling for extension records works differently from other date and time fields in F2 with respect to time zones. At the time of writing it will store time zone differences, including hours, in the database, whereas the standard F2 fields strip the hours from the time stamps.

Example

Consider a Danish company with clients operating from both Denmark and New York. The company decides that the server time zone is Central European with a standard UTC offset of +1 and +2 when daylight savings apply.

A client in Denmark, reading and writing dates and times from F2-REST, need not apply any time zone transformations as it operates in the same time zone as the server.

A client in New York, reading and writing dates and times from F2-REST, needs to apply time zone transformations though: New York belongs to the Eastern Time Zone which is UTC-5, so the time zone difference is "Server minus client time zone offsets" which is +1 minus -5 = 6. This means the client has to add 6 hours when reading dates and times from F2-REST — and it has to subtract 6 hours when writing dates and times.

If the time-part is irrelevant, such as for a deadline, the client should always send the desired date with 00:00:00 as the time part — regardless of daylight settings as well as time zone differences.

Data manipulation

The F2-REST service enables clients to add new content to F2 as well as modify existing data. New content is usually added using a Post Once Exactly technique (see below). Existing data is modified using the HTTP verb PATCH together with JSON Patch documents [JSONPatch].

Post Once Exactly semantics

Some data manipulation in F2 has "Post Once Exactly" (POE) semantics meaning the operation can safely be repeated in case of network failures or timeouts.

POE works by first requesting a POE resource (identified by a URL created by the server) and then POSTing the actual payload data to that URL. Should the second POST fail due to a timeout or other problems then it is safe to repeat the exact same POST again and again until it succeeds. It goes like this:

1. The client GETs a representation of some entity — for instance a case file or the service index.
2. The client finds the POE link in the returned representation by looking for a link relation marked as <http://cbrain.com/casefile/rel/create-matter> (or similar).
3. The client issues an empty POST to the POE link. The content-type should be `application/x-www-form-urlencoded`.
4. The server responds with a status code 303 (See other) and includes a URL for the new POE resource in the `Location` header.
5. The client prepares the actual payload and POSTs it to the new POE resource.

A POE operation that succeeds returns status code 201 "Created". A repeated POE operation that has succeeded already returns the status code 303 "See other". In both cases the "Location" header is set to the URL of the created resource.

Once a POE URL has been obtained by the client it can POST data for a new entity to that URL. The data can be formatted using the media type `application/x-www-form-urlencoded`, `application/json` [JSON] or `multipart/form-data` [MultipartFormdata].

POE resources are automatically reclaimed by the server after some time and deleted.

JSON patch documents

A JSON patch document [JSONPatch] consists of a JSON array of patch operations like `add`, `replace` and `remove`. The server interprets each of these operations and applies them to the target resource. Currently, only the `replace` operation is in use in F2 REST.

The use of PATCH in this way allows the client to easily perform partial updates of resources (as opposed to issuing a PUT of the complete resource).

Example

```
PATCH /target HTTP/1.1
Content-Type: application/json-patch

[
  { "op": "replace", "path": "/Type", "value": "Outbound" },
  { "op": "replace", "path": "/Title", "value": "Changed title" },
  { "op": "replace", "path": "/Text", "value": "Changed text" }
]
```

Updating parties

Single party

A single party, such as the responsible of a case or matter, must be updated by replacing the party with a new party, using a party reference to indicate which party to use (see [F2-REST Documentation](#)).

As an example, consider the responsible party of a matter. An HTTP GET on a matter resource returns something like the following structure:

Matter example

```
<Matter xmlns="http://cbrain.com/casefile/schema/">
  <Id>378052</Id>
  <Responsible>
    <Name>Lise Nielsen</Name>
    <PartyNumber>118</PartyNumber>
  </Responsible>
</Matter>
```

Updating the responsible is simply a matter of patching /Responsible with a suitable party reference:

Patch example

```
PATCH /some-path HTTP/1.1
Accept: application/vnd.cbrain.casefile+xml
Content-Type: application/json-patch

[
  { "value": {"PartyNo": 26}, "op": "replace", "path": "/Responsible" }
]
```

Lists of parties

Lists of parties, such as receivers and involved parties on cases and matters, must be updated by replacing the complete party list with a whole new list of parties — it is not possible to update single elements in the lists.

As an example, consider the involved parties of a matter. An HTTP GET on a matter resource returns something like the following structure:

Matter example

```
<Matter xmlns="http://cbrain.com/casefile/schema/">
  <Id>378052</Id>
  <InvolvedParties>
    <InvolvedParty>
      <Name>Party A</Name>
      <PartyNumber>118</PartyNumber>
    </InvolvedParty>
    <InvolvedParty>
      <Name>Party B</Name>
      <PartyNumber>75</PartyNumber>
    </InvolvedParty>
  </InvolvedParties>
</Matter>
```

Updating the list of involved parties, is simply a matter of patching /InvolvedParties with suitable party references (in this case party numbers 26 and 28):

Patch example

```
PATCH /some-path HTTP/1.1
Accept: application/vnd.cbrain.casefile+xml
Content-Type: application/json-patch

[
  {"value":[{"PartyNo":26},{"PartyNo":28}], "op":"replace", "path":"/InvolvedParties"}
]
```

Encoding rules for input data

Parameters for the creation of new resources or invoking complex business operations, are encoded using standard media types — either form-url-encoded, multipart-form-data or JSON.

Application/x-www-form-urlencoded

Basic values are encoded as key/value properties following the rules of application/x-www-form-urlencoded.

List values are encoded by adding an array index to the keys (prefixed with a colon). So for instance, a list of three different names and e-mails would be encoded as the following keys:

Example list encoding using colon for index indicator

```
Name:0  
EMail:0  
Name:1  
EMail:1  
Name:2  
EMail:2
```

Example (simple properties)

```
Name=John%20Doe&EMail=jd%40example.com
```

Example (list properties — %3a is a URL encoded colon)

```
Name%3a0=John&Name%3a1=Pete
```

Multipart/form-data

Basic values are encoded as key/value properties following the rules of multipart/form-data [[MultipartFormdata](#)].

List values are encoded by adding an array index to the keys (prefixed with a colon) in the same way as it is done for application/x-www-form-urlencoded.

Example

```
Content-type: multipart/form-data, boundary=AaB03x

--AaB03x
content-disposition: form-data; name="Name"

John Doe
--AaB03x
content-disposition: form-data; name="EMail"

jd@example.com
--AaB03x
content-disposition: form-data; name="Photo"; filename="some-photo.jpg"
Content-Type: image/jpeg
Content-Transfer-Encoding: binary

... binary data ...
--AaB03x--
```

Application/json

Basic values are encoded using JSON objects.

Example

```
{
  "Name" : "John Doe",
  "EMail" : "jd@example.com"
}
```

Lists are encoded using JSON arrays.

Example

```
[
  {
    "Name" : "John Doe",
    "EMail" : "jd@example.com"
  },
  {
    "Name" : "Pete",
    "EMail" : "pete@example.com"
  }
]
```

Error handling

F2-REST makes extensive use of standard HTTP codes to flag error conditions — but also adds an additional payload with more detailed information about the error.

The payload for error conditions is a document having a single `Message` element, an optional `Details` element, an optional `ReasonCode` and optionally an array of links related to the error condition.

Example XML response

```
HTTP/1.1 404 Not Found
Content-Length: 356
Content-Type: application/vnd.cbain.casefile+xml

<Error xmlns="http://cbain.com/casefile/schema/">
  <Message>Party snapshot with ID 200391 not found.</Message>
  <ReasonCode>DossierCaseClosed</ReasonCode>
</Error>
```

Example JSON response

```
HTTP/1.1 404 Not Found
Content-Length: 65
Content-Type: application/json

{
  "Message": "Party with ID 200391 not found.",
  "ReasonCode": "DossierCaseClosed",
  "Links": []
}
```

While the `Message` element will contain a description of the problem, an optional `Details` element may contain a more technical description primarily targeted at developers.

Table 1. Error data properties

Name	Description
Message	string General description of the problem. This may be presented to the end-user.
Details	string Technical details about the problem. This should be logged by the client.
ReasonCode	string See ReasonCode below.
Link	List of Link One or more links to other related resources.

The following is a list of the HTTP status codes used (but clients MUST be prepared to receive other status codes as infrastructure components such as web proxies may return other codes):

Table 2. HTTP status codes

Code	Text	Description
200	OK	Success.
201	Created	The request has been fulfilled and resulted in a new resource being created.
303	See other	The response to the request can be found under another URI using a GET method. When received in response to a POST (or PUT/DELETE), it should be assumed that the server has received the data and the redirect should be issued with a separate GET message.
400	Bad request	The server cannot or will not process the request due to something that is perceived to be a client error. Usually this is due to a validation error of some sort — for instance required values that are missing in a request. See the XML payload for further details about why the request failed.
401	Unauthorized	Similar to 403 Forbidden, but specifically for use when authentication is required and has failed or has not yet been provided.
403	Forbidden	The request was valid but did not have the right authorization to access the resource.
404	Not found	The requested resource could not be found. See the XML payload for further details about why the request failed.
409	Conflict	Indicates that the request could not be processed because of conflict in the request. Most often this is caused by updating a resource which is in a state that does not allow the update.
423	The resource being accessed is locked.	Indicates that the request could not be processed because another user has locked access to the resource – most likely because it is opened for edit in the desktop client. The client should retry the request again at a later time. The ReasonCode may indicate why the resource was locked.
500	Internal server error	Something is broken. Please contact your local F2 service provider.
503	Service unavailable	Indicates that the server might be overloaded or simply out-of-service for a while. Common transient reasons are database deadlocks and timeouts. The client should retry the request again at a later time.

ReasonCode

Often the HTTP status code is not enough to identify the reason for an error. The `Message` element may tell exactly what is wrong but it is not easy to react on programmatically. The `ReasonCode`

may then provide some useful information. `ReasonCode` value is a string. Note that `ReasonCode` is optional.

These are the registered reason codes:

Table 3. ReasonCode values

Code	Description
DossierBadCaseNumber	Operation on a Matter failed because of bad format of a case number.
DossierBadCPR	... because of bad format of the CPR.
DossierBadResponsible	... because of bad specification of the Responsible.
DossierBadTitle	... because of bad format of the Title.
DossierCaseClosed	... because the case is closed.
DossierCaseNotFound	... because the case is not found.
DossierDocumentNotFound	... because a document is not found.
DossierDocumentNoWriteAccess	... because a document is not accessible for writing.
DossierNoReadAccess	... because it is not accessible for reading.
DossierNoResponsible	... because of missing Responsible.
DossierNotFound	... because it is not found.
DossierNoWriteAccess	... because it is not accessible for writing.
DossierOnMeeting	... because it is located on a meeting.
Locked	Operation failed because the resource was locked (primary reason being cases transferred to external archive and then locked).
LockedByAnotherUser	Operation failed because a resource was locked by another user (typically because someone is editing it).
CaseClosed	Operation on a Case failed it is closed.

Retries

If a request to F2-REST fails for some reason, the client may retry that request again at a later time — but care must be taken to avoid duplicate inserts. To avoid duplicates, F2-REST uses a Post-Once-Exactly technique as described in [\[Data manipulation\]](#).

Typical error scenarios that should be solved by retrying requests are:

- Network failures: these happen once in a while and will often result in HTTP timeouts or broken connections as seen from the client application.

- Database deadlocks and timeouts: these can be expected when the database server is under heavy load. These situations are indicated by status code 503 Service Unavailable.
- Locking: some resources can be locked for editing by other users. When trying to modify such a resource the server returns 423 Locked.

Authentication

F2-REST supports OAuth2 authentication [[OAuth](#)].

OAuth2 allows the client to authenticate without knowing the F2 user password, and thus the password will never be stored in any program or configuration file related to the client.

Resource Owner Password Credentials Grant

The "Resource Owner Password Credentials Grant" flow is designed for background services authorizing as specific F2 users.

In this flow the client sends its own client credentials together with the F2 user name and, optionally, the F2 user password (see section 4.3 in [OAuth](#)).

To use OAuth2 in this way the client must do as follows:

1. Register a client ID and client secret. These can be obtained from your local F2 service provider.
2. Request an access token:
 - a. Create a HTTP Basic authentication header using the client ID and client secret.
 - b. Get the access token endpoint URL from the service index. It is named as `http://cbrain.com/casefile/rel/oauth2-token`.
 - c. Make a POST request to the access token endpoint with the following parameters added to the payload and encoded using `application/x-www-form-urlencoded`:
 - i. `grant_type` = "password"
 - ii. `username` = "F2 user name"
 - iii. `password` = "F2 user password" (optional)
 - iv. `organization_partyno` = "F2 organization party number" (optional).

The `organization_partyno` parameter is relevant for users with roles in more than one organization and it allows the client to specify which of the user's organizations to log in to using the party number of that organization.
 - d. The response will be a JSON encoded representation of a "bearer" token to be used in the following requests.
3. Make a request using the bearer token.
 - a. Create a HTTP Authorization header with the value "Bearer" plus the actual bearer token returned from the access token request.
 - b. Make a request to the required resource.

Example access token request from the OAuth2 specification

```
POST /token HTTP/1.1

Host: server.example.com
Authorization: Basic czZCaGRSa3F0MzpnWDFmQmF0M2JW
Content-Type: application/x-www-form-urlencoded

grant_type=password&username=johndoe&password=A3ddj3w
```

Example access token response from the OAuth2 specification:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Cache-Control: no-store
Pragma: no-cache

{
  "access_token": "2YotnFZFEjr1zCsicMWpAA",
  "token_type": "bearer",
  "expires_in": 3600,
  "refresh_token": "vKUuy1LYYptqmOYmp806"
}
```

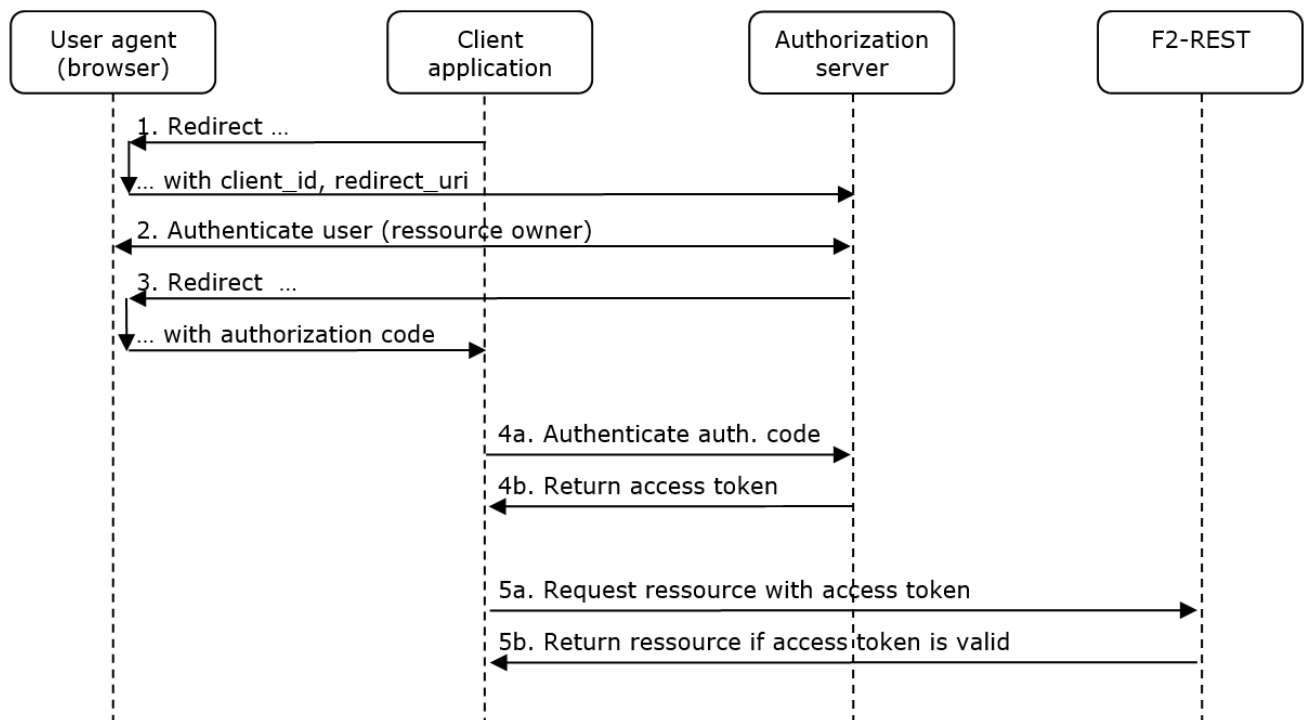
Example resource request

```
GET /resource HTTP/1.1
Host: server.example.com
Authorization: Bearer 2YotnFZFEjr1zCsicMWpAA
```

Authorization Code Grant

The "Authorization Code Grant" flow is used when a client needs to make requests on behalf of a user without knowing any of the user's credentials. Typically, this is used for scenarios where a third party application interacts with F2-REST in the context of the end user.

This flow is a multi-step flow that works as described below (see also section 4.1 in [OAuth](#)).



1. The client starts by redirecting the user agent (typically a web browser) to a predefined authorization URL.

OBS: The client is only a "client" in terms of OAuth2 — typically it is implemented as a server side component capable of sending redirects to a web browser.

2. The end user interacts with the authorization server in order to prove his/her identity (for instance by using Kerberos or Windows AD Single Sign On).
3. The authorization server redirects the user agent back to the original client with an authorization code in the URL. The client extracts the authorization code.
4. The client exchanges the authorization code for an access token by calling a predefined token handler URL.
5. The access token is used in sub-sequent calls to F2-REST as a bearer token that enables the client to work on behalf of the end user.

To use this authorization flow with Windows AD Single Sign On (supported by F2) the client must do as follows:

1. Register client and callback URL with your local F2 service provider.
2. Request an authorization code

- a. Get the authorization code endpoint URL from the F2-REST service index. It is named as `http://cbrain.com/casefile/rel/oauth2-adsso-authorization`.
 - b. Add the following parameters to the authorization code endpoint URL:
 - i. `response_type=code`
Hard coded value.
 - ii. `client_id=<client ID>`
Insert OAuth2 client identifier here.
 - iii. `state=<client state>` (optional)
Insert some client state here to avoid Cross-Site Request Forgery (see section 10.12 in [OAuth](#)).
 - iv. `redirect_uri=<redirect URL>` (optional)
Insert redirection URL here if more than one is possible — otherwise leave out this parameter as the authorization service will be configured with it anyway.
 - v. `scope = <list of requested scopes>` (optional)
Insert space delimited list of requested scopes. Currently only used for OpenID Connect (with scopes "openid" and "profile").
 - c. Redirect the browser to the resulting URL.
 - d. After user authorization the browser will be redirected back to the redirect URL.
 - e. The redirection handler extracts the authorization "code" value from the URL.
3. Request an access token
- a. Get the access token endpoint URL from the service index. It is named as `http://cbrain.com/casefile/rel/oauth2-token`.
 - b. Make a `POST` request to the access token endpoint using HTTP Basic Authentication to authenticate the client (using the client ID and client secret). Add the following parameters to the payload and encode them using `application/x-www-form-urlencoded`:
 - i. `grant_type=authorization_code`
Hard coded value.
 - ii. `code=<authorization code>`
Insert authorization code here.
 - iii. `redirect_uri=<redirect URL>`
If a redirect URL was used in the previous steps then it must be included here again.
 - c. If the access token request is valid and authorized, the authorization server issues an access token and returns it in a JSON response.
4. Make a request using the bearer token.
- a. Create an HTTP Authorization header with the value "Bearer" plus the actual bearer token returned from the access token request.
 - b. Make a request to the required resource using the acquired bearer token.

Example authorization request

```
GET /authorize?
response_type=code&client_id=s6BhdRkqt3&state=xyz&redirect_uri=https%3A%2F%2Fclient%2Eexample%
2Ecom%2Fcb HTTP/1.1
Host: server.example.com
```

Example authorization response

```
HTTP/1.1 302 Found
Location: https://client.example.com/cb?code=Sp1x10BeZQQYbYS6WxSbIA&state=xyz
```

Example access token request

```
POST /token HTTP/1.1
Host: server.example.com
Authorization: Basic czZCaGRSa3F0MzpnWDFmQmF0M2JW
Content-Type: application/x-www-form-urlencoded

grant_type=authorization_code&code=Sp1x10BeZQQYbYS6WxSbIA&redirect_uri=https%3A%2F%2Fclient%2E
example%2Ecom%2Fcb
```

Example access token response

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=UTF-8

{
  "access_token": "2YotnFZFEjr1zCsicMWpAA",
  "token_type": "bearer",
  "expires_in": 3600,
  "refresh_token": "vKUuy1LYYptqmOYmp806"
}
```

Example resource request

```
GET /resource HTTP/1.1
Host: server.example.com
Authorization: Bearer 2YotnFZFEjr1zCsicMWpAA
```

F2 Session Grant

The "F2 Session Grant" flow is used when a client needs to make requests on behalf of a user and is in possession of a F2-client session ID. Typically, this is used for scenarios where a third party application interacts with F2-REST in the context of the end user through a "WebPush" task in the task guide framework.

To use this flow the client must obtain an F2 session ID and use it as follows:

1. Register a client ID and client secret. These can be obtained from your local F2 provider.
2. Request an access token:
 - a. Create a HTTP Basic authentication header using the client ID and client secret.
 - b. Get the access token endpoint URL from the service index. It is named as `http://cbrain.com/casefile/re1/oauth2-token`.
 - c. Make a POST request to the access token endpoint with the following parameters added to the payload and encoded using `application/x-www-form-urlencoded`:
 - i. `grant_type` = `"http://cbrain.com/casefile/f2session-grant/"`
 - ii. `f2session_id` = `<F2 session ID>`
 - iii. The response will be a JSON encoded representation of a "bearer" token to be used in the following requests.
 - iv. The bearer token can now be used for resource requests as described in the previous sections.

Example access token request

```
POST /token HTTP/1.1

Host: server.example.com
Authorization: Basic czZCaGRSa3F0MzpnWDFmQmF0M2JW
Content-Type: application/x-www-form-urlencoded

grant_type=http%3A%2F%2Fcbrain.com%2Fcasefile%2Ff2session-grant%2F&f2session_id=xxxxxxx
```

Example access token response

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=UTF-8
Cache-Control: no-store
Pragma: no-cache

{
  "access_token": "2YotnFZFEjr1zCsicMWpAA",
  "token_type": "bearer",
  "expires_in": 3600,
  "refresh_token": "vKUuy1LYYptqmOYmp806"
}
```

JWT Identity token Grant

The "Identity token Grant" flow is used when a client needs to make requests on behalf of a user and is in possession of a JWT identity token from a third party identity provider.

Typically, this is used for scenarios where a third party application authenticates with an external identity provider, receives and validates a JWT identity token and then forwards the same identity token to F2-REST for authentication.

To use this flow the client must obtain an identity token and use it as follows:

1. Register a client ID and client secret. These can be obtained from your local F2 service provider.
2. Request an access token:
 - a. Create an HTTP Basic authentication header using the client ID and client secret.
 - b. Get the access token endpoint URL from the service index. It is named as `http://cbrain.com/casefile/rel/oauth2-token`.
 - c. Make a POST request to the access token endpoint with the following parameters added to the payload and encoded using `application/x-www-form-urlencoded`:
 - i. `grant_type` = "http://cbrain.com/casefile/identitytoken-grant/"
 - ii. `id_token` = <identity token>
 - d. The response will be a JSON encoded representation of a "bearer" token to be used in the following requests.
3. The bearer token can now be used as described in the previous sections.

Example access token request

```
POST /token HTTP/1.1
```

```
Host: server.example.com
```

```
Authorization: Basic czZCaGRSa3F0MzpnWDFmQmF0M2JW
```

```
Content-Type: application/x-www-form-urlencoded
```

```
grant_type= http%3A%2F%2Fcbrain.com%2Fcasefile%2Fidentitytoken-grant%2F&id_token=xxxxxxx
```

Example access token response

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Cache-Control: no-store
Pragma: no-cache

{
  "access_token": "2YotnFZFEjr1zCsicMWpAA",
  "token_type": "bearer",
  "expires_in": 3600,
  "refresh_token": "vKUuy1LYYptqmOYmp806"
}
```

OpenID Connect

F2-REST support authorization using OpenID Connect as described in [OIDC].

OpenID Connect is an identity layer on top of the OAuth 2.0 protocol. It allows Clients to verify the identity of the End-User based on the authentication performed by an Authorization Server, as well as to obtain basic profile information about the End-User in an interoperable and REST-like manner.

The OpenID Connect flow is identical to the "Authorization Code Grant" flow — with the addition of a few OpenID Connect parameters being included. The differences are:

1. Include the scopes "openid" and "profile" in the "Authorization code request" to get profile information about the authenticated user.
2. The profile information is returned in the access token response as an ID Token (per the OpenID Connect specification).
 - a. The access token request URL is not the same as for the Authorization Code Grant.
 - b. Get the OpenID Connect access token endpoint URL from the service index. It is named as `http://cbrain.com/casefile/rel/oauth2-idtoken`.
 - c. Make an access token request to the access token endpoint.
 - d. If the access token request is valid and authorized, the authorization server issues an access token and returns a response with an ID Token.

It is not possible to request the ID Token at a later time — it is only available during the authorization process.

Example authorization code request

```
GET /authorize?
response_type=code&client_id=s6BhdRkqt3&state=xyz&redirect_uri=https%3A%2F%2Fclient%2Eexample%
2Ecom%2Fcb&scope=openid%20profile HTTP/1.1
```

Example access token request

```
GET /authorize?
response_type=code&client_id=s6BhdRkqt3&state=xyz&redirect_uri=https%3A%2F%2Fclient%2Eexample%
2Ecom%2Fcb&scope=openid%20profile HTTP/1.1
Host: server.example.com
```

Example access token response

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8

{
  "access_token": "2YotnFZFEjr1zCsicMWpAA",
  "token_type": "bearer",
  "expires_in": 3600,
  "id_token": "... JWT data ..."
}
```

Refresh tokens

F2-REST supports refresh tokens as described in section 6 in [OAuth]. For this, the OAuth2 authorization flows mentioned in the previous sections returns "refresh_token" in addition to "access_token".

Please notice that refresh tokens have a limited lifetime (default is 24 hours more than the access token). This means clients should be prepared to handle a situation where both the access token and the refresh tokens have expired.

The refresh token can be used later to obtain a new access token with these steps:

1. Create a HTTP Basic authentication header using the client ID and client secret.
2. Get the access token endpoint URL from the service index. It is named as `http://cbrain.com/casefile/rel/oauth2-token`.
3. Make a POST request to the access token endpoint with the following parameters added to the payload and encoded using `application/x-www-form-urlencoded`:
 - a. `grant_type` = "refresh_token"
 - b. `refresh_token` = <refresh token>
4. The response will be a JSON encoded representation of a "bearer" token to be used in the following requests.
5. The bearer token can now be used as described in the previous sections — as well as the new refresh token can be used later on.

Example access token request

```
POST /token HTTP/1.1

Host: server.example.com
Authorization: Basic czZCaGRSa3F0MzpnWDFmQmF0M2JW
Content-Type: application/x-www-form-urlencoded

grant_type=refresh_token&refresh_token=vKUuy1LYYptqmOYmp806
```

Example access token response

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Cache-Control: no-store
Pragma: no-cache

{
  "access_token": "SC5b5LyGyqCN6UhaTeuC",
  "token_type": "bearer",
  "expires_in": 3600,
  "refresh_token": "VR0UC6H2tNEpcxSfTxUf"
}
```

OAuth 2.0 Token Introspection

F2-REST supports RFC 7662 "OAuth 2.0 Token Introspection" (see [OAuthTokenInspection](#)). This standard allows a client to send a `POST` request to F2-REST, containing an access token, and retrieve some meta information about the token.

The returned values are:

Table 4. OAuth2 token introspection response values

Name	Description
active	bool True if the access token is active.
client_id	string Client identifier for the OAuth 2.0 client that requested this token.
exp	int Integer timestamp, measured in the number of seconds since January 1 1970 UTC, indicating when this token will expire.
username	string The F2 username associated with the token.
organization_partyno	int Party number of the organization selected during login. This is relevant in cases where a user is member of more than one organization.

Get the access token endpoint URL from the service index. It is named as `http://cbrain.com/casefile/rel/oauth2-token-introspection`.

Example token inspection request

```
POST /introspect HTTP/1.1
Host: server.example.com
Accept: application/json
Content-Type: application/x-www-form-urlencoded
Authorization: Basic czZCaGRSa3F0MzpnWDFmQmF0M2JW

token=mF_9.B5f-4.1JqM
```


Example token inspection response

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "active": true,
  "client_id": "l238j323ds-23ij4",
  "username": "jdoe",
  "organization_partyno": 99,
  "exp": 1419356238,
}
```

Common data types
= Links

Link data type

The Link data type represents a relation between two resources. Links can be found in most, if not all, resources represented in F2-REST and is used to indicate related resources such as a parent/child relationship or a related command processing resource.

Table 5. Link properties

Name	Description
rel	string Link relation identifier.
href	string Actual URL to target resource.
type	string Expected media type of target resource (optional).
title	string Human readable title of link (optional).

Link example

```
<Link href="..."
      rel="http://cbrain.com/casefile/rel/enum-type"
      title="Enumeration type"/>
```

Enumerations

F2 uses a lot of different enumeration types for keywords and various other domain specific codes in the representations of F2 data such as matters and cases.

EnumerationItem data type

The value of a single enumeration property on a F2 object is represented as an object itself with the following properties:

Table 6. EnumerationItem properties

Name	Description
Title	string Displayed title of selected enumeration value.
Path	string Path in the enumeration value hierarchy.
Code	string Identifier for the selected enumeration value.
Abbreviation	string Abbreviation of selected enumeration's title. For historical reasons, the abbreviation can be used as an alternative to the <code>Code</code> property when looking up enumeration values.
Link	List of Link One or more links to other related resources.

The `Link` property may contain a link to the enumeration type definition as indicated by the link relation `http://cbrain.com/casefile/rel/enum-type`. The target will be an enumeration type resource as described in [F2-REST Documentation](#).

Enumeration example

```
<Case>
...
<Keyword>
  <Title>00.03.04 - Regionaludvikling EU</Title>
  <Path>Parent1Title/Parent2Title/ItemTitle</Path>
  <Code>00.03.04</Code>
  <Abbreviation>00.03.04</Abbreviation>
  <Link href="..."
        rel="http://cbrain.com/casefile/rel/enum-type"
        title="Enumeration type"/>
</Keyword>
</Case>
```

Party related data types

PartyItem data type

A party item represents the primary keys of a single party in a collection of parties.

Table 7. PartyItem properties

Name	Description
Name	string Displayed name of party.
EMail	string Contact e-mail.
Type	string Party type (see below).
Id	int Internal identifier of party - not visible in F2.
PartyNumber	int Internal identifier of party - displayed in F2 party details window.
SynchronizationKey	string Identifier for synchronizing parties with external repositories such as Windows Active Directory.
CPRCVR	string Personal security number or VAT number.
CVR_P	string Danish P-number extension to CVR (VAT) number.

The name `PartyItem` is used to avoid confusion with the resource type `Party`. For example, the `Responsible` element (found as a sub-element of the `Matter` resource) is a `PartyItem` as shown in the following example. The link to "self" in the `PartyItem` is a link to a `Party` resource (see [F2-REST Documentation](#)).

Possible values for `Type` are:

Table 8. Party type values

Value	Party type
User	F2 user.
Unit	F2 organization.
Team	F2 team.
External	Shared external contact (such as e-mail recipients).
DistributionList	F2 distribution list.
Private	Private external contact.

PartyItem Example

```
<Responsible>
  <Name>Emma Terkelsen</Name>
  <EMail>et@example.com</EMail>
  <Link href="..." rel="self"/>
  <Type>User</Type>
  <Id>2156</Id>
  <PartyNumber>20</PartyNumber>
  <CPRCVR>12344321</CPRCVR>
  <CVR_P>1234554321</CVR_P>
</Responsible>
```

AccessPartyItem data type

The `AccessPartyItem` represents a single party with a specific access right to a matter or case. It contains the same properties as `PartyItem` along with an `Access` property, which describes the user's access to the containing object.

Access may either be "Read", "WriteDocument" or "Write".

AccessPartyItem example

```
<Responsible>
  <Name>Emma Terkelsen</Name>
  <EMail>et@example.com</EMail>
  <Link href="..." rel="self"/>
  <Type>User</Type>
  <Id>2156</Id>
  <PartyNumber>20</PartyNumber>
  <CPRCVR>12344321</CPRCVR>
  <CVR_P>1234554321</CVR_P>
  <Access>Read</Access>
</Responsible>
```

PartyGroupItem data type

A `PartyGroupItem` represents a named group of parties (in a collection of `PartyGroupItem`).

PartyGroupItem example

```
<Stakeholders>
  <PartyGroup>
    <Name>My party group</Name>
    <SynchronizationKey>C8163116-7031-47FA-9D54-0E6880827B4D</SynchronizationKey>
    <PartyNumber>117</PartyNumber>
  </PartyGroup>
</Stakeholders>
```

PartyReference data type

Some updates require one or more party references to specify Party resources in requests. A party reference can be one of many things — an e-mail, a party number, login name and more.

Table 9. PartyReference properties

Name	Description
EMail	string Exact e-mail. For instance "joe@example.com".
Name	string Optional name of party specified together with EMail .
Contact	string Name and e-mail combined as "Name <E-Mail>". For instance "Joe Hudson <joe@example.com>".
Login	string The login name of an F2 user.
PartyNo	int Party number as found in the F2 party registry.
OrganizationNo	int Organization number of a party as found in the F2 party registry, optionally specified together with PartyNo . Organization numbers are useful specify a combination of party number and organization number when assigning for instance a responsible user on a case.
PickFirstMatch	bool PickFirstMatch may optionally be used with PartyNo to uniquely specify a Party resource. As party numbers can be used in multiple organizations, and it is optional to specify an organization number, PickFirstMatch can be used to disambiguate. If PickFirstMatch is specified as false (default) it will cause an error when the party specification is ambiguous, otherwise any (i.e. the "first") applicable organization number is picked for the party.
PartyType	string A PartyType may optionally be used together with an external party's PartyNo to attach a party type to the external party when creating a new case or matter, or patching an existing case or matter. When PartyType is specified, the F2-REST server will try to find a matching party type enumeration item by Code (external id) or alternatively by Title in the F2 party type registry. If a matching party type enumeration item is found it will be attached to the specified external party on the case/matter. Note that an external party will be bound to a party type exclusively in relation to the case/matter and not in the external party registry.

PartyGroupReference data type

A party group reference can be used to refer to a named group of parties — for instance a security group, or just a single party (user, organization or other).

Table 10. PartyGroupReference properties

Name	Description
Name	string Name of party group (as shown in the F2 desktop client).
Id	int The group identifier (found in the database, not available in the F2 desktop client).
SynckronizationKey	string The synchronization key for the party.
PartyNumber	int The party number of the party group. Can be used to referrer to any party group, including security groups.

Core resources
= Cases

Case resource

This resource represents a single F2 case.

Location

Cases can be located directly by ID or case number using the URL templates found in the service index. In addition to this there are lots of links to cases from other resources which makes it possible to locate related cases simply by following the links.

Lookup by case ID

The link relation `http://cbrain.com/casefile/rel/case-by-id` identifies a URL template for looking up cases by ID. The template uses the template parameter `id` for the case ID.

A `GET` on the generated URL will return a redirect to the corresponding case resource.

Example link from service document

```
{
  "http://cbrain.com/casefile/rel/case-by-id":
  {
    href: "https://.../{id}",
    title: "Get case by ID"
  }
}
```

Lookup by case number

The link relation `http://cbrain.com/casefile/rel/case-by-case-number` identifies a URL template for looking up cases by case number. The template uses the template parameter `caseNumber` for

the case number.

A `GET` on the generated URL will return a redirect to the corresponding case resource.

Example link from service document

```
{
  "http://cbrain.com/casefile/rel/case-by-case-number":
  {
    href: "http://.../{caseNumber}",
    title: "Get case by case number"
  }
}
```


Properties

Table 11. Case properties

Name	Description
Id	int (read-only) Internal identifier.
CaseNumber	string (read-only) Official case number (for instance "2012 — 342").
Title	string (255 characters) Case title.
CreatedDate	DateTime (read-only) Date and time the case was created.
ModifiedDate	DateTime (read-only) Date and time the case was last modified.
Keyword	Enumeration First element of all keywords. This can either be in the format of <code>##.##.##</code> when used with keywords from "Kommunernes Landsforening" — or the complete path to a specific keyword name (as seen from the desktop client).
Keywords	List of Enumeration named Keyword The complete list of all keywords.
JournalPlan	Enumeration Represents the Danish "Journalplan" used for "Kommunernes Landsforening". When PATCHing the journal plan use the path <code>/JournalPlan</code> with a string value representing the journal plan code. The actual journal plan code is setup by the F2 administrator as the value of "Ekstern ID" for journal plan entries.
Responsible	PartyItem The party who is responsible for the case.
InvolvedParties	List of PartyItem named InvolvedParty The involved parties of the case ("Sagens parter" in Danish — "case parties").
SupplementaryCaseWorkers	List of PartyItem named SupplementaryCaseWorker The supplementary case workers for the case.
ProcessInstruction	Enumeration Represents the Danish "Handlingsfacет" used for "Kommunernes Landsforening". When PATCHing the process instruction use the path <code>/ProcessInstruction</code> with a string value representing the process instruction code. The actual process instruction code is setup by the F2 administrator as the value of "Ekstern ID" for process instruction entries.

Table 11. Case properties

Name	Description
	Landsforening". When PATCHing the deletion action use the path /DeletionAction with a string value representing the deletion action code. The actual deletion action code is setup by the F2 administrator as the value of "Ekstern ID" for deletion action entries.
TaskGuideKey	string The task guide key identifying a task guide associated with the case. PATCHing this value to null removes the task guide from the case.
Link	List of Link (read-only) One or more links to other related resources.
CprNumber	string An external identification number for a party related to the case. For example in a Danish installation it could be the nationwide CVR number of a company or the nationwide CPR number of a citizen.
ExtensionData	any XML General purpose XML container for data related directly to the case. Can be set via the create-extension-data link on the case resource.
ExternalId	string (255 characters) ID representing some external ID. The actual use of this field is customer specific. Can be set by PATCH only.
PreviousCaseNumber	string (256 characters) Previous case number, usually used to indicate what the case was called before it was imported to F2.
SecurityGroups	List of PartyGroupItem named PartyGroup The security groups, users or organizations that are part of the access restrictions for the meeting. The parties are accessed by their name when patching and creating cases.
ExternalAccess	Enumeration Identifying how the case is accessible outside F2. Default codes are "Open", "PartlyOpen" and "Closed" (configurable for the installation).
Closed	bool (read-only) True if the case is closed. Otherwise the case is open. Depending on the F2 configuration certain changes on the case are not allowed when closed.

Table 11. Case properties

Name	Description
	Case deadline.

The XML root element is `<Case>` .

Case example

```
<Case xmlns="http://cbrain.com/casefile/schema/">
  <Id>1046444</Id>
  <CaseNumber>2022 - 60</CaseNumber>
  <Title>Case 1</Title>
  <CreatedDate>2022-02-03T15:31:26.293</CreatedDate>
  <ModifiedDate>2022-02-03T15:34:40</ModifiedDate>
  <Deadline>2022-02-10T15:34:40</Deadline>
  <ModifiedBy>
    <Name>James Wolt</Name>
    <EMail>jw@cbrain.dk</EMail>
    <Link href="..." rel="self" title="James Wolt" />
    <Type>User</Type>
    <Id>671771</Id>
    <PartyNumber>37301</PartyNumber>
    <CPRCVR>120390-2222</CPRCVR>
    <CVR_P />
  </ModifiedBy>
  <CprNumber>62419361</CprNumber>
  <ExternalId>AX-2468</ExternalId>
  <Keyword>
    <Title>Change request</Title>
    <Path>Issuetyper/Change request</Path>
    <Code>Emneord_Issuetyper_Change request</Code>
    <Link href="http://localhost/f2/main/restservices/enum-types/KeyWord"
rel="http://cbrain.com/casefile/rel/enum-type" type="application/vnd.cbrain.casefile+xml"
title="Enumeration type" />
  </Keyword>
  <Keywords>
    <Keyword>
      <Title>Change request</Title>
      <Path>Issuetyper/Change request</Path>
      <Code>Emneord_Issuetyper_Change request</Code>
      <Link href="http://localhost/f2/main/restservices/enum-types/KeyWord"
rel="http://cbrain.com/casefile/rel/enum-type" type="application/vnd.cbrain.casefile+xml"
title="Enumeration type" />
    </Keyword>
  </Keywords>
  <Responsible>
    <Name>James Wolt</Name>
    <EMail>jw@cbrain.dk</EMail>
    <Link href="..." rel="self" title="James Wolt" />
    <Type>User</Type>
    <Id>1006118</Id>
    <PartyNumber>37301</PartyNumber>
    <CPRCVR>120390-2222</CPRCVR>
    <CVR_P />
  </Responsible>
  <Closed>>false</Closed>
  <Link href="..." rel="self" title="Case 1" />
  <Link href="..." rel="down" title="Matters" />
  <Link href="..." rel="http://cbrain.com/casefile/rel/create-matter" title="Add matter" />
  <ExtensionData />
  <SecurityGroups>
    <PartyGroup>
      <Name>James Wolt</Name>
      <PartyNumber>37301</PartyNumber>
    </PartyGroup>
    <PartyGroup>
```

```

<PartyNumber>108</PartyNumber>
  </PartyGroup>
</SecurityGroups>
<ExternalAccess>
  <Title>Åben</Title>
  <Path>Open</Path>
  <Code>Open</Code>
  <Abbreviation>Open</Abbreviation>
  <Link href="..." rel="http://cbrain.com/casefile/rel/enum-type" title="Enumeration type"
/>
</ExternalAccess>
<ProgressCode>
  <Title>ProgressCode 1</Title>
  <Path>ProgressCode 1</Path>
  <Code>ProgressCode 1</Code>
</ProgressCode>
<InvolvedParties>
  <InvolvedParty>
    <Name>Kirsten Svanholm-Nielsen</Name>
    <EMail />
    <Link href="..." rel="self" title="Kirsten Svanholm-Nielsen" />
    <Type>External</Type>
    <Id>1006125</Id>
    <PartyNumber>39938</PartyNumber>
    <SynchronizationKey />
    <CPRCVR>070761-4218</CPRCVR>
  </InvolvedParty>
</InvolvedParties>
<SupplementaryCaseWorkers>
  <SupplementaryCaseWorker>
    <Name>Grith Malm</Name>
    <EMail>gm@cbrain.dk</EMail>
    <Link href="..." rel="self" title="Grith Malm" />
    <Type>User</Type>
    <Id>1006124</Id>
    <PartyNumber>874</PartyNumber>
    <CPRCVR />
    <CVR_P />
  </SupplementaryCaseWorker>
</SupplementaryCaseWorkers>
</Case>

```

Create new case

A case may be created using the Post Once Exactly pattern (see [Data manipulation](#)). Follow the link named `http://cbrain.com/casefile/rel/create-case` from the service index to locate the POE resource for creating a case.

Table 12. Input parameters for creating a case

Name	Description
Title	string (required, max 255 characters) The title of the new case.
KeywordCode	string The code that represents a keyword to be associated with the new case. This can either be in the format of <code>##.##.##</code> when used with keywords from "Kommunernes Landsforening" — or the complete path to a specific keyword name (as seen from the desktop client) or the external ID of the keyword. As the complete paths and external ID's may overlap, the external ID takes precedence.
KeywordCodes	List of string A list of keyword codes. Each keyword code is handled in the same way as the above singular <code>KeywordCode</code> .
JournalPlanCode	string The code which represents the journal plan to be associated with the new meeting. In Danish this is called "Journalplanskode" and is used only with keywords from "Kommunernes Landsforening".
DeletionActionCode	string The code that represents the deletion action to be associated with the new meeting. In Danish this is called "Kassationskode" and is used only with keywords from "Kommunernes Landsforening".
ProcessInstructionCode	string The code that represents the process instruction to be associated with the new case. In Danish this is called "Handlingsfacet" and is used only with keywords from "Kommunernes Landsforening".
TaskGuideKey	string The key that identifies a task guide to be associated with the new case.
SecurityGroups	List of PartyGroupReference The list of security groups, users or organizations to include in the access restrictions of the new case. The security groups are referenced by the name shown in the F2 desktop client.
ExternalAccessCode	string The enumeration code that represents the external access to be associated with the new case.
Responsible	PartyReference Specifies who is responsible for the new case.
InvolvedParties	List of PartyReference Specifies the involved parties of the new case ("Sagens parter" in Danish - -).

Table 12. Input parameters for creating a case

Name	Description
	Case deadline. .

Example request for creating a case (include first POE step)

```
POST /case-create-url HTTP/1.1
Content-Length: 0

> HTTP/1.1 201 Created
> Location: /case-create-poc-url
> Content-Length: 0

POST /case-create-poc-url HTTP/1.1
Content-Type: application/x-www-form-urlencoded
Content-Length: 19

Title=A+new+case
```

Example request for creating a case with security groups "SG4" and "SG5" (excluding first POE step)

```
POST /case-create-poc-url HTTP/1.1
Content-Type: application/x-www-form-urlencoded
Content-Length: 93

*** LINE BREAKS ADDED FOR READABILITY ***
Title=A+new+case+with+security+groups
&SecurityGroups%3a0.Name=SG4
&SecurityGroups%3a1.Name=SG5
```

Modify case

A case may be modified by issuing a `PATCH` request targeted at the case resource URL.

The simple properties are straight forward to replace with similar typed values, but some of the complex case properties need special encoding when patching. These properties are:

Table 13. Properties needing special encoding when patching

Name	Description
Keyword	Use a single string value referring to the Code value of the enumeration item.
Keywords	Use an array of string values referring to the Code value of the enumeration items.
JournalPlan	Use a single string value referring to the Code value of the enumeration item.
Responsible	Use a PartyReference object to reference the requested party.
InvolvedParties	Use an array of PartyReference objects to reference the requested parties.
SupplementaryCaseWorkers	Use an array of PartyReference objects to reference the requested parties.
ProcessInstruction	Use a single string value referring to the Code value of the enumeration item.
DeletionAction	Use a single string value referring to the Code value of the enumeration item.
ExtensionData	Use a string to contain the XML.
SecurityGroups	Use an array of PartyGroupReference objects to reference the required parties.
ExternalAccess	Use a single string value referring to the Code value of the enumeration item.
ProgressCode	Enumeration Use a single string value referring to the Code value of the enumeration item. Can be either the external Id or the title of the progress code.
Deadline	DateTime Case deadline.
Navigation	string The item key of a task-guide task.

Example PATCH request with headers

```
PATCH /case-url HTTP/1.1
Accept: application/vnd.cbrain.casefile+xml
Content-Type: application/json-patch

[
  {"value":"New case title", "op":"replace", "path":"/Title"},
  {"value":"00.07.00", "op":"replace", "path":"/Keyword"},
  {"value":"G10", "op":"replace", "path":"/ProcessInstruction"},
  {"value":"B", "op":"replace", "path":"/DeletionAction"},
  {"value":{"PartyNo":26}, "op":"replace", "path":"/Responsible"},
  {"value":[{"PartyNo":26}, {"PartyNo":28}], "op":"replace", "path":"/InvolvedParties"},
  {"value":
    [
      {"Name":"SG4"},
      {"Name":"SG5"}
    ], "op":"replace", "path":"/SecurityGroups"}
]
```

Delete case

Deleting a case in F2 is not as easy as simply issuing a `DELETE` targeted at the case resource since the delete operation requires a few parameters for input.

A case is deleted by looking up the "delete case handler" linked to a case by the link relation `http://cbrain.com/casefile/rel/delete-case`. Follow this link and issue a `POST` to it with the following parameters:

Table 14. Inputs for deleting a case

Name	Description
DeleteReason	string A string which specifies why the case is deleted. This property is required and is used for accountability purposes.
GenerateReportForDeletedItems	bool A Boolean value specifying if a report-matter should be generated, to document which documents what was deleted as a result of deleting the case.

Example request for deleting a case.

```
POST /delete-case-url HTTP/1.1
Content-Type: application/x-www-form-urlencoded

DeleteReason=Deleted+because+of+GDPR&GenerateReportForDeletedItems=true
```

The response from deleting a case indicates if the deletion was successful for the case and detailed information about matters that caused issues - like for instance in this example, where a matter was journalized and in that way blocked deleting of it together with the case:

Example response from deleting a case

```
<DeleteCaseResult>
  <CaseDeleted>false</CaseDeleted>
  <DeleteErrorMessage>A matter was not deleted</DeleteErrorMessage>
  <ReportFailed>true</ReportFailed>
  <ReportErrorMessage>Deadlock</ReportErrorMessage>
  <MatterResults>
    <MatterResult>
      <Success>false</Success>
      <Id>42</Id>
      <MatterNumber>22</MatterNumber>
      <Title>Some matter which was not deleted</Title>
      <Creator>
        <Name>Ulla Torkil</Name>
        <Email>ut@example.com</Email>
        <Link href="..." rel="self"/>
        <Type>User</Type>
        <Id>2156</Id>
        <PartyNumber>20</PartyNumber>
        <CPRCVR>12344321</CPRCVR>
        <CVR_P>1234554321</CVR_P>
      </Creator>
      <Responsible>
        <Name>Ulla Torkil</Name>
        <Email>ut@example.com</Email>
        <Link href="..." rel="self"/>
        <Type>User</Type>
        <Id>2156</Id>
        <PartyNumber>20</PartyNumber>
        <CPRCVR>12344321</CPRCVR>
        <CVR_P>1234554321</CVR_P>
      </Responsible>
      <DeleteResult>Matter is journalized</DeleteResult>
      <Link href="..." rel="self"/>
    </MatterResult>
  </MatterResults>
</DeleteCaseResult>
```

Table 15. Delete case response properties

Name	Description
CaseDeleted	bool Indicates if the case was deleted or not.
DeleteErrorMessage	string Optional error message.
ReportFailed	bool Indicates if the delete operation failed to generate a new matter with a delete-case report.
ReportErrorMessage	string Optional error message for the delete-case report.
MatterResults	List of DeleteCaseMatterResult named MatterResult See table below.
Link	List of Link Links to related items. See below.

Table 16. DeleteCaseMatterResult properties

Name	Description
Success	bool Indicates if the matter was successfully deleted or not.
Id	int Internal matter identifier.
MatterNumber	int (optional) Displayed matter number.
Title	string Matter title.
Creator	PartyItem Matter creator.
Responsible	PartyItem Matter responsible.
DeleteResult	string Message describing the result of deleting the matter.

The XML root element is `<DeleteCaseResult>`.

The link list may contain a link to the generated case-delete report. The link is named `http://cbrain.com/casefile/rel/case-delete-report`.

Dry-run deleting a case

If you only want to test what would happen when deleting a case, then there is also a "dry-run case delete handler" — it is linked to a case by the link relation <http://cbrain.com/casefile/rel/delete-case-dry-run>. Follow this link and issue a `POST` to it with no parameters.

If something fails in the dry-run it can be assumed that the same error will occur in the real deletion, if performed by the same user. If, however, the user is allowed to bypass access restrictions, then the dry-run result is purely advisory. In this case, the deletion may very well succeed, depending on what the error was for a specific matter.

Table 17. Dry-run delete case response properties

Name	Description
PreviousReason	string If the case was attempted to be deleted previously but failed, then the <code>DeleteReason</code> provided to that call, is included here.
MatterResults	List of DeleteCaseMatterResult named <code>MatterResult</code> Same content as when deleting a case.

Close and open a case

A case can be closed by issuing an empty `POST` request to the "Close case handler" linked to by the link relation <http://cbrain.com/casefile/rel/open-case> from the case itself.

In the same way, a case can be opened again by issuing an empty `POST` request to the "Open case handler" linked to by the link relation <http://cbrain.com/casefile/rel/open-case> from the case itself.

Complete case structure including matters and documents

Follow the link named <http://cbrain.com/casefile/rel/structure> to get a complete representation of the case and its matters and their documents. See the section [Complete case structure](#).

Matters

All accessible matters related to a case can be found by following the link relation `down` from the case. The response is a [collection resource](#) with links to all matters.

Table 18. Expected link relations

Rel	Description
self	Link to self.
down	Link to list of contained matters.
alternate	Link containing the URL for opening the case in the desktop client.
http://cbrain.com/casefile/rel/create-matter	Link to POE factory for creating a matter associated with the case. See Create matter .
http://cbrain.com/casefile/rel/extension-data	Link to related custom XML data.
http://cbrain.com/casefile/rel/create-extension-data	Link for creating new related custom XML data.
http://cbrain.com/casefile/rel/notes	Link to related notes. See Notes .
http://cbrain.com/casefile/rel/create-note	Link to POE factory for creating notes associated with the case. See Add note .
http://cbrain.com/casefile/rel/structure	Link to complete case structure. See pages/case-structure.html .
http://cbrain.com/casefile/rel/related-cases	Link to list of related cases.
http://cbrain.com/casefile/rel/add-related-case	Link to resource for adding related cases.
http://cbrain.com/casefile/rel/field-data	Link to related field data ("Sagens oplysninger" in Danish).
http://cbrain.com/casefile/rel/parties	Link to all the parties related to the case. See pages/party-collection.html .
http://cbrain.com/casefile/rel/taskguidescripts-execute	Link for executing task guide scripts.
http://cbrain.com/casefile/rel/taskguidetasks-complete	Link for completing a single task guide task.
http://cbrain.com/casefile/rel/taskguidetasks-execute	Link for executing a single task guide task.
http://cbrain.com/casefile/rel/taskguide-state	Link for getting the complete state of an associated task guide (visibility of tasks as well as field and variable values).
http://cbrain.com/casefile/rel/open-case	Link for opening the case. See Close and open case .
http://cbrain.com/casefile/rel/close-case	Link for closing the case. See Close and open case .

Table 18. Expected link relations

Rel	Description
http://cbrain.com/casefile/rel/create-submission-event-log	Link for creating a submission eventlog.
http://cbrain.com/casefile/rel/delete-case-dry-run	Link for dry-run deleting a case. See Dry run delete case .
http://cbrain.com/casefile/rel/delete-case	Link for deleting a case. See Delete case .
http://cbrain.com/casefile/rel/update-case-extension-data	Link which can be used to update one or more extension fields for the case. See Updating matter & case extension fields .
http://cbrain.com/casefile/rel/extension-records	Link to case extension record. See Case guide data .

Complete case structure

Complete case structure resource

This resource represents a subset of the meta data of a case and all its matters and documents as one single resource.

Location

The case structure is reachable from a case via the link relation `http://cbrain.com/casefile/rel/structure`.

Properties

Table 19. Case structure properties

Name	Description
Id	int Internal identifier.
CaseNumber	string Official case number (for instance "2012 - 342").
Title	string Case title.
CreatedDate	DateTime Date and time the case was created.
ModifiedDate	DateTime Date and time the case was last modified.
ModifiedBy	PartyItem Last user who modified the case.
Responsible	PartyItem The party who is responsible for the case.
Link	List of Link (read-only) One or more links to other related resources.
Matters	List of Matter List of all accessible matters on the case.

The XML root element is `<Case>`.

Example case structure

```
<Case xmlns="http://cbrain.com/casefile/schema/">
  <Id>517645</Id>
  <CaseNumber>2016 - 22</CaseNumber>
  <Title>Example case</Title>
  <CreatedDate>2016-01-07T12:47:50.173</CreatedDate>
  <ModifiedDate>2016-01-12T15:52:55</ModifiedDate>
  <ModifiedBy>
    <Name>Joe Noel</Name>
    <EMail>jn@cbrain.dk</EMail>
    <Link href="..."
      rel="self"
      title="Joe Noel" />
  </ModifiedBy>
  <Responsible>
    <Name>Joe Noel</Name>
    <EMail>jn@cbrain.dk</EMail>
    <Link href="..."
      rel="self"
      title="Joe Noel" />
  </Responsible>
  <Matters>
    <Matter>
      <Id>517886</Id>
      <MatterNumber>0</MatterNumber>
      <Title>Example matter</Title>
      <CreatedDate>2016-01-12T15:52:55</CreatedDate>
      <ModifiedDate>2016-01-12T15:52:56</ModifiedDate>
      <ModifiedBy>...</ModifiedBy>
      <Responsible>...</Responsible>
      <CaseNumber>2016 - 22</CaseNumber>
      <Archived>false</Archived>
      <Deadline xsi:nil="true" />
      <Status>InProgress</Status>
      <LetterDate xsi:nil="true" />
      <ReceivedDate xsi:nil="true" />
      <SentDate xsi:nil="true" />
      <ReminderDate xsi:nil="true" />
      <Documents />
      <Link href="..."
        rel="self"
        title="Example matter" />
      <Link href="..."
        rel="http://cbrain.com/casefile/rel/structure"
        title="Complete matter structure" />
    </Matter>
    <Matter>
      <Id>517777</Id>
      <MatterNumber>0</MatterNumber>
      <Title>Example matter with documents</Title>
      ...
      <Documents>
        <Document>
          <Id>517783</Id>
          <Title>Example document</Title>
          <ModifiedDate>2016-01-08T16:09:37.15</ModifiedDate>
          <ModifiedBy>...</ModifiedBy>
          <FileType>doc</FileType>
          <ContentType>application/msword</ContentType>
```

```

<Link href="..."
      rel="self"
      title="Example document" />
</Document>
</Documents>
<Link ... />
</Matter>
<Matter>
  <Id>517647</Id>
  <MatterNumber>0</MatterNumber>
  <Title>Last matter example</Title>
  ...
</Matter>
</Matters>
<Link href="..."
      rel="self"
      title="Example case" />
<Link href="..."
      rel="http://cbrain.com/casefile/rel/structure"
      title="Complete case structure" />
</Case>

```

Related cases

Related cases resource

This resource represents a list of relations to other cases. Each relation contains a link to the related case, a relation type and various links for manipulating the list.

Cases can be related to each other as "Parent" or "Child" which are both each other's inverse relationship; if case A is parent to case B then case B is automatically child of A and vice versa. In addition to this cases can be related to each other as "Duplicated" or simply "Related".

Location

The list of related cases is reachable from a case via the link relation <http://cbrain.com/casefile/rel/related-cases>.

Properties

Table 20. Case relation properties

Name	Description
CaseId	int Source case ID.
CaseNumber	string Source case number.
CaseTitle	string Source case title.
Relation	Relation Zero or more relations to other cases.
Relation.CaseId	int Related case ID.
Relation.CaseNumber	string Related case number.
Relation.CaseTitle	string Related case title.
Relation.RelationType	string Type of relation between the two cases. Possible values are: • Parent : Source case is parent of the related case. • Child : Source case is child of the related case. • Duplicated : Source case is a duplicate of the related case. • Related : Source case has no specific relationship to the related case.
Relation.Link	List of Link One or more links to other related resources.
``	x .
Link	List of Link One or more links to other related resources.

The root XML element is `Relations`.

Related cases example

```
<Relations xmlns="http://cbrain.com/casefile/schema/">
  <CaseId>5899664</CaseId>
  <CaseNumber>2016 - 74362</CaseNumber>
  <CaseTitle>Test case</CaseTitle>
  <Relation>
    <CaseId>5899676</CaseId>
    <CaseNumber>2016 - 74365</CaseNumber>
    <CaseTitle>Related case 1</CaseTitle>
    <RelationType>Parent</RelationType>
    <Link href="..." rel="http://cbrain.com/casefile/rel/related-case" />
    <Link href="..." rel="http://cbrain.com/casefile/rel/delete-related-case" />
  </Relation>
  <Relation>
    <CaseId>5899678</CaseId>
    <CaseNumber>2016 - 74366</CaseNumber>
    <CaseTitle>Related case 2</CaseTitle>
    <RelationType>Child</RelationType>
    <Link href="..." rel="http://cbrain.com/casefile/rel/related-case" />
    <Link href="..." rel="http://cbrain.com/casefile/rel/delete-related-case" />
  </Relation>
  <Link href="..." rel="self" />
  <Link href="..." rel="up" />
</Relations>
```

Table 21. Expected link relations

Rel	Description
self	Link to the list of related cases.
up	Link to parent case.

Table 22. Expected link relations for related cases

Rel	Description
http://cbrain.com/casefile/rel/related-case	Link to related case.
http://cbrain.com/casefile/rel/delete-related-case	Link to resource for deleting the containing case relation

Add new relation

Follow the link named `http://cbrain.com/casefile/rel/add-related-case` from a case to find a resource for adding new case relations. Then issue an `POST` to that resource with the input parameters listed below.

Table 23. Input parameters for adding a related case relation

Name	Description
CaseId	int Identifier of related case.
RelationType	string Case relation type. Possible values are: • Parent • Child • Duplicated • Related

Special HTTP status codes from adding a new case relation:

201 Created

Relation did not exist already and was created.

200 OK

Relation existed already and no new relation was created.

Add related case example request

```
POST add-related-case-url HTTP/1.1
Accept: application/vnd.cbrain.casefile+xml
Content-Type: application/x-www-form-urlencoded

CaseId=279641&RelationType=Parent
```

Delete case relation

Follow the link named `http://cbrain.com/casefile/rel/delete-related-case` in the link collection of a single relation, to find a resource for deleting the relation. Then issue an empty `POST` request to that resource in order to delete the relation.

A `DELETE` operation would seem more appropriate for this purpose, but it does not allow later versions of F2-REST to include parameters in the request if needed.

Delete related case example

```
POST related-case-delete-url HTTP/1.1
Accept: application/vnd.cbrain.casefile+xml
Content-Length: 0
```

Matters

Matter resource

This resource represents a single F2 matter.

Location

Matters can be located directly by ID or matter number using the URL templates found in the service index. In addition to this there are links from a case to all the matters on the case.

Lookup by matter ID

The link relation `http://cbrain.com/casefile/rel/matter-by-id` identifies a URL template for looking up matters by ID. The template uses the template parameter `id` for the matter ID.

A `GET` on the generated URL will return a redirect to the corresponding matter resource.

Example link from service document

```
{
  "http://cbrain.com/casefile/rel/matter-by-id":
  {
    href: "http://.../{id}",
    title: "Get matter by ID"
  }
}
```

Lookup by matter number

The link relation `http://cbrain.com/casefile/rel/matter-by-matter-number` identifies a URL template for looking up matters by matter number. The template uses the template parameter `matterNumber` for the matter number.

A `GET` on the generated URL will return a redirect to the corresponding matter resource.

Example link from service document

```
{
  "http://cbrain.com/casefile/rel/matter-by-matter-number":
  {
    href: "http://.../{matterNumber}",
    title: "Get matter by matter number"
  }
}
```

Properties

Table 24. Matter properties

Name	Description
Id	int (read-only) Internal matter identifier.
MatterNumber	int (read-only) Unique public matter number.
CaseMatterNumber	int (optional, read-only) Matter number relative to the matter's associated case.
VersionNumber	int (read-only) Matter version number.
IsActiveVersion	bool (read-only) Flag which indicates if this is the active version of the matter.
Title	string (255 characters) Matter title.
CreatedDate	DateTime (read-only) Date and time the matter was created.
ModifiedDate	DateTime (read-only) Date and time the matter was last modified.
BinnedDate	DateTime (read-only) Date and time the matter was binned (deleted).
CaseNumber	string (read-only) Number of the case related to the matter. The actual case representation can be found by following the link relation "up".
Text	string Text body of the matter — also called "dossier document" in the desktop client. If the matter does not have a dossier document then <code>Text</code> will be null. A patch request, patching the <code>Text</code> property, will return status <code>400</code> , if the matter does not have a dossier document (which may happen for approval matters).
Type	string Type of matter. The possible values are: <code>Internal</code> — represents a matter that exists only in F2. <code>Inbound</code> — represents a matter that has been received via an external channel. <code>Outbound</code> — represents a matter that has been sent out of F2.
AccessLevel	string Access level. The possible values are <code>Involved</code> , <code>Unit</code> and <code>All</code> .

Table 24. Matter properties

Name	Description
Deadline	DateTime Optional deadline for the matter.
Archived	bool (read-only) Archival status of the matter.
CprNumber	x (12 characters) An external identification number for a party in case a specific party is related to the matter. For example, in a Danish installation it could be the national CVR number of a company or the national CPR number of a citizen.
Status	string (read-only) Matter status. Possible values are <code>InProgress</code> and <code>Closed</code> .
PreviousCaseNumber	string (255 characters) Previous case number.
LetterDate	DateTime Letter date — a general purpose date field. If <code>LetterDate</code> is set explicitly then the assigned value will be returned as <code>LetterDate</code> too. If <code>LetterDate</code> is not set, then either <code>ReceivedDate</code> or <code>SentDate</code> will be returned as <code>LetterDate</code> (only one of <code>ReceivedDate</code> or <code>SentDate</code> should ever be set).
ReceivedDate	DateTime Received date. This property is ignored when creating outgoing matters.
SentDate	DateTime Sent date.
Responsible	PartyItem The party who is responsible for the matter.
Sender	PartyItem The sender of the matter.
Receivers	List of PartyItem named <code>Receiver</code> The receivers of the matter.
CcReceivers	List of PartyItem named <code>CcReceiver</code> The CC receivers of the matter.
BccReceivers	List of PartyItem named <code>BccReceiver</code> The BCC receivers of the matter.
InvolvedParties	List of PartyItem named <code>InvolvedParty</code> The involved parties of the matter ("Aktparter" in Danish).

Table 24. Matter properties

Name	Description
	This is a temporary value used only when modifying a matter. See Modify matter .
Alerts	List of Alert named Alert List of alerts related to this dossier (see below).
SecurityGroups	List of PartyGroupItem named PartyGroup The security groups, users or organizations that are part of the access restrictions for the meeting. The parties are accessed by their name when patching and creating matters.
Link	List of Link (read-only) One or more links to other related resources.
ExtensionData	Any XML General purpose XML container for data related directly to the matter. Can be set via the create-extension-data link on the matter resource.
ExternalAccess	Enumeration Identifying how the matter is accessible outside F2. Default values are <code>Open</code> , <code>PartlyOpen</code> and <code>Closed</code> (the complete list if configurable for the installation).
DeliveryType	string Identifies the method for sending the matter out of F2 once it is created. Possible values are: <code>None</code> — do not send. <code>Email</code> — send via e-mail. <code>OutputManager</code> — send via the Danish "Fjernprint". <code>DigitalMail</code> — send via the Danish "E-Boks". <code>BOM</code> — send via the Danish "Byg og Miljø". <code>ExternalRequisition</code> — send as external requisition. <code>DistributedRequisitionAnswer</code> — send as answer to distributed requisition. <code>DistributedRequisition</code> — send as distributed requisition. <code>M4Mail</code> — send via M4. <code>DigitalMailV3</code> — send via the Danish "Digitalpost", version 3. Beware that it is only possible to create new matters or patch existing matters with the delivery types <code>None</code> , <code>Email</code> , <code>M4Mail</code> , <code>DigitalMail</code> and <code>OutputManager</code> . The rest of the delivery types are managed internally by F2 and are only readable in GET requests

Table 24. Matter properties

Name	Description
	The item key of a task-guide task.

Table 25. Alert properties

Name	Description
Type	string The type of the alert. The list of types is not fixed. Use the value "Other" for unexpected errors.
Description	string Detailed description of the alert. It may be shown to the user in the F2 client, possibly together with a general description of the alert type.
User	PartyItem The user for which the alert should be visible.

The XML root element is `<Matter>`.

Matter example

```
<Matter xmlns="http://cbrain.com/casefile/schema/">
  <Id>1046445</Id>
  <MatterNumber>106844</MatterNumber>
  <CaseMatterNumber>1</CaseMatterNumber>
  <VersionNumber>1</VersionNumber>
  <Title>Matter 1</Title>
  <CreatedDate>2022-02-03T15:31:28</CreatedDate>
  <ModifiedDate>2022-02-07T11:04:23</ModifiedDate>
  <ModifiedBy>
    <Name>James Wolt</Name>
    <EMail>jw@cbrain.dk</EMail>
    <Link href="..." rel="self" title="James Wolt" />
    <Type>User</Type>
    <Id>671771</Id>
    <PartyNumber>37301</PartyNumber>
    <CPRCVR>120390-2222</CPRCVR>
    <CVR_P />
  </ModifiedBy>
  <CaseNumber>2022 - 60</CaseNumber>
  <Text>...</Text>
  <CprNumber>12345678</CprNumber>
  <Type>Internal</Type>
  <AccessLevel>All</AccessLevel>
  <Deadline>2022-03-04T00:00:00</Deadline>
  <Archived>true</Archived>
  <Status>InProgress</Status>
  <PreviousCaseNumber>Example-2910-YE23</PreviousCaseNumber>
  <LetterDate>2022-02-07T00:00:00</LetterDate>
  <ReceivedDate>2022-02-07T00:00:00</ReceivedDate>
  <SentDate>2022-02-07T00:00:00</SentDate>
  <ReminderDate>2022-02-17T00:00:00</ReminderDate>
  <Sender>
    <Name>James Wolt</Name>
    <EMail>jw@cbrain.dk</EMail>
    <Link href="..." rel="self" title="James Wolt" />
    <Type>User</Type>
    <Id>1006121</Id>
    <PartyNumber>37301</PartyNumber>
    <CPRCVR>120390-2222</CPRCVR>
    <CVR_P />
  </Sender>
  <Receivers>
    <Receiver>
      <Name>Gunnar Holgersen</Name>
      <EMail />
      <Link href="..." rel="self" title="Gunnar Holgersen" />
      <Type>External</Type>
      <Id>1006145</Id>
      <PartyNumber>39942</PartyNumber>
      <SynchronizationKey />
      <CPRCVR>090441-1234</CPRCVR>
    </Receiver>
  </Receivers>
  <CcReceivers />
  <BccReceivers />
  <InvolvedParties>
    <InvolvedParty>
      <Name>Allerslev I/S</Name>
```

```

<Link href="..." rel="self" title="Allerslev I/S" />
  <Type>External</Type>
  <Id>1006147</Id>
  <PartyNumber>81716</PartyNumber>
  <SynchronizationKey>CVR_14147683</SynchronizationKey>
  <CPRCVR>14147683</CPRCVR>
</InvolvedParty>
</InvolvedParties>
<Responsible>
  <Name>James Wolt</Name>
  <EMail>jw@cbrain.dk</EMail>
  <Link href="..." rel="self" title="James Wolt" />
  <Type>User</Type>
  <Id>1006122</Id>
  <PartyNumber>37301</PartyNumber>
  <CPRCVR>120390-2222</CPRCVR>
  <CVR_P />
</Responsible>
<SupplementaryCaseWorkers>
  <SupplementaryCaseWorker>
    <Name>Sue Cheriff</Name>
    <EMail />
    <Link href="..." rel="self" title="Sue Cheriff" />
    <Type>User</Type>
    <Id>1006146</Id>
    <PartyNumber>108</PartyNumber>
    <CPRCVR />
    <CVR_P />
    <Access>Read</Access>
  </SupplementaryCaseWorker>
</SupplementaryCaseWorkers>
<SkipInboxForCaseWorkers>false</SkipInboxForCaseWorkers>
<Link href="..." rel="self" title="Matter 1" />
<Link href="..." rel="http://cbrain.com/casefile/rel/pdf-content" title="PDF content" />
<Link href="..." rel="http://cbrain.com/casefile/rel/create-document" title="Add document"
/>
<ExtensionData />
<SecurityGroups>
  <PartyGroup>
    <Name>James Wolt</Name>
    <PartyNumber>37301</PartyNumber>
  </PartyGroup>
  <PartyGroup>
    <Name>Sue Cheriff</Name>
    <PartyNumber>108</PartyNumber>
  </PartyGroup>
</SecurityGroups>
<Alerts>
  <Alert>
    <Type>SendMail</Type>
    <Description>The dossier cannot be sent because it belongs to a closed case.
</Description>
  <User>
    <Name>James Wolt</Name>
    <EMail>jw@cbrain.dk</EMail>
    <Link href="..." rel="self" title="James Wolt" />
    <Type>User</Type>
    <Id>671771</Id>
    <PartyNumber>37301</PartyNumber>
    <CPRCVR>120390-2222</CPRCVR>

```

```

</User>
  </Alert>
</Alerts>
<ExternalAccess>
  <Title>Åben</Title>
  <Path>Open</Path>
  <Code>Open</Code>
  <Abbreviation>Open</Abbreviation>
  <Link href="..." rel="http://cbrain.com/casefile/rel/enum-type" title="Enumeration type"
/>
</ExternalAccess>
<DeliveryType>Email</DeliveryType>
<Keywords>
  <Keyword>
    <Title>Emneord A</Title>
    <Path>Område A/Emneord A</Path>
    <Code>Emneord_Område A_Emneord A</Code>
    <Abbreviation />
    <Link href="..." rel="http://cbrain.com/casefile/rel/enum-type" title="Enumeration type"
  />
</Keyword>
</Keywords>
<Versions>
  <MatterVersion>
    <VersionNumber>2</VersionNumber>
    <IsActiveVersion>true</IsActiveVersion>
    <Link href="..." rel="self" />
  </MatterVersion>
  <MatterVersion>
    <VersionNumber>1</VersionNumber>
    <IsActiveVersion>false</IsActiveVersion>
    <Link href="..." rel="self" />
  </MatterVersion>
</Versions>
</Matter>

```

Table 26. Expected link relations

Rel	Description
self	Link to self
down	Link to list of contained documents.
up	Link to parent case file.
alternate	Link containing the URL for opening the matter in the desktop client.
http://cbrain.com/casefile/rel/pdf-content	Link to PDF representation of the matter text.
http://cbrain.com/casefile/rel/create-document	Link to POE factory for creating documents associated with the matter. See Create document .
http://cbrain.com/casefile/rel/upload-validation	Link to upload validation for validation before creating a document. See Upload validation .
http://cbrain.com/casefile/rel/extension-data	
http://cbrain.com/casefile/rel/create-extension-data	
http://cbrain.com/casefile/rel/notes	Link to list of related notes. See Notes .
http://cbrain.com/casefile/rel/create-note	Link to POE factory for creating notes associated with the matter. See Add note .
http://cbrain.com/casefile/rel/chats	Link to list of related chats. See Chats .
http://cbrain.com/casefile/rel/create-chat	Link to POE factory for creating chats associated with the matter. See Add chat .
http://cbrain.com/casefile/rel/structure	Link to complete matter structure. See pages/matter-structure.html .

Table 26. Expected link relations

Rel	Description
	• exclude_meta: Exclude matter meta data • exclude_matter_doc Exclude matter document • exclude_approval_meta: Exclude approval meta data • include_approval_doc Include approval document • exclude_attachments: Exclude attachments • exclude_meta: Exclude matter meta data • exclude_matter_doc Exclude matter document • exclude_approval_meta: Exclude approval meta data • include_approval_doc Include approval document • exclude_attachments: Exclude attachments
http://cbrain.com/casefile/rel/approval	Link to approval process associated with matter. The link will only show up if an approval process is associated with the matter.
http://cbrain.com/casefile/rel/create-approval	Link to POE factory for creating an approval process associated with the matter. The link will only show up if no approval processes are associated with the matter.
http://cbrain.com/casefile/rel/set-case	Link which can be used to attach the matter to a case. See Attach matter to case .
http://cbrain.com/casefile/rel/update-matter-extension-data	Link which can be used to update one or more extension fields for the matter. See Updating matter & case extension fields .
http://cbrain.com/casefile/rel/set-responsible	Link which can be used to set or change the responsible party. See Set responsible party for matter .
http://cbrain.com/casefile/rel/active-version	Link which can be used to get the active version of the current matter.

Table 26. Expected link relations

Rel	Description
http://cbrain.com/casefile/rel/mark-as-physical-mail	Link which can be used to mark a matter as having been sent via physical mail. This is done by issuing a POST to the link. Note that the operation is not POE protected, as it is idempotent. It should however be noted, that performing the operation twice on the same matter, will result in a 400. Check that the matter is marked as being sent, in order to verify that the operation has succeeded.

Create new matter

A matter may be created using the Post Once Exactly pattern (see [Data manipulation](#)). Follow the link named <http://cbrain.com/casefile/rel/create-matter> from the service index to locate the POE resource for creating a matter—or follow the same link from a specific case in order to associate the created matter with that case.

Table 27. Input parameters for creating a matter

Name	Description
Title	string (required, 255 characters) The title of the new matter.
Type	string (required) The type of matter to create. The list of possible values is the same as those for the property <code>Type</code> on a Matter resource.
Text	string The textual body (also called "dossier document"). The text should be represented as HTML.
CprNumber	string (12 characters) An external identification number for a party related to the matter. For example, in a Danish installation it could be the national CVR number of a company or the national CPR number of a citizen.
AccessLevel	string Specifies access level. The list of possible values is the same as that for the property <code>AccessLevel</code> on a Matter resource.
Deadline	DateTime Specifies deadline for the new matter.
Archived	bool Specifies whether or not the new matter is archived.
Status	string Specifies status. The list of possible values is the same as that for the property <code>Status</code> on a Matter resource.
Responsible	PartyReference Specifies who is responsible for the new matter.
Sender	PartyReference Specifies the sender of the new matter.
Receivers	List of PartyReference Specifies the receivers of the new matter.
CcReceivers	List of PartyReference Specifies the CC receivers of the new matter.
BccReceivers	List of PartyReference Specifies the BCC receivers of the new matter.
InvolvedParties	List of PartyReference Specifies the involved parties of the new matter ("Aktparter" in Danish).
ExternalAccessCode	string The enumeration code that represents the external access to be associated

Table 27. Input parameters for creating a matter

Name	Description
	with the new matter.
LetterDate	DateTime Specifies letter date.
ReceivedDate	DateTime Specifies when the matter was received. The value is ignored when creating output matters.
SentDate	DateTime Specifies when the matter was sent.
ReminderDate	DateTime Specifies reminder date ("Erindringsdato" in Danish).
PreviousCaseNumber	string (255 characters) Specifies previous case number (can be any string — not necessarily a F2 case number).
SecurityGroups	List of PartyGroupReference The list of security groups, users or organizations to include in the access restrictions of the new case. The security groups are referenced by the name shown in the F2 desktop client.
DeliveryType	string Possible values are: • None — do not send. • EMail — send via e-mail. • OutputManager — send via the Danish "Fjernprint". • DigitalMail — send via the Danish "E-Boks". • M4Mail — send via M4. • DigitalMailV3 — send via the Danish "Digitalpost", version 3.
KeywordCodes	List of string A list of keyword codes to be associated with the new matter. This can either be in the format of ##.##.## when used with keywords from "Kommunernes Landsforening" — or the complete path to a specific keyword name (as seen from the desktop client) or the external id of the keyword. As the complete path and external id may overlap, the external id takes precedence.

There is no need to supply any reference to the parent case since that is embedded in the URL of the factory resource.

Input values are encoded according to the rules in [Input encoding](#).

Example 1 (simple URL encoded parameters including initial POE step)

```
POST /matter-create-url HTTP/1.1
Content-Length: 0

> HTTP/1.1 201 Created
> Location: /matter-create-poc-url
> Content-Length: 0

POST /matter-create-poc-url HTTP/1.1
Content-Type: application/x-www-form-urlencoded
Content-Length: 50

Type=Internal&Title=A+new+matter&Archived=false
```

Example 2 (complex URL encoded parameters)

```
POST /matter-create-poc-url HTTP/1.1
Accept: application/vnd.cbrain.casefile+xml
Content-Type: application/x-www-form-urlencoded
Authorization: Bearer tgkVLxr3J7x2BZzUoKPB
Content-Length: 288

*** LINE BREAKS ADDED FOR READABILITY ***
Type=Outbound
&Title=En+ny+akt&Text=Denne+akt+er+oprettet+som+REST+demo.
&Archived=false
&Sender.Name=Peter+Nielsen
&Sender.Email=pn%40example.com
&Receivers%3a0.Name=cBrain+DK
&Receivers%3a0.Email=dk%40cbrain.dk
&Receivers%3a1.Name=cBrain+US
&Receivers%3a1.Email=info%40cbrain.com
```

Example 3 (complex JSON encoded parameters, same values as above)

```
POST /matter-create-poc-url HTTP/1.1
Accept: application/vnd.cbrain.casefile+xml
Content-Type: application/json
Authorization: Bearer D6pV2jNchtZ0TwFYtyAU
Content-Length: 588

{
  "Type": "Internal",
  "Title": "En ny akt",
  "Text": "Denne akt er oprettet som REST demo.",
  "Archived": false,
  "Sender": { "Name": "Peter Nielsen", "Email": "pn@example.com" },
  "Receivers": [
    { "Name": "cBrain DK", "Email": "dk@cbrain.dk" },
    { "Name": "cBrain US", "Email": "info@cbrain.com" }
  ]
}
```

Modify matter

The simple properties are straight forward to replace with similar typed values, but some of the complex matter properties need special encoding when patching. These properties are:

Table 28. Properties needing special encoding when patching

Name	Description
Sender	Use a PartyReference object to reference the requested party.
Receivers	Use an array of PartyReference objects to reference the requested parties.
CcReceivers	Use an array of PartyReference objects to reference the requested parties.
BccReceivers	Use an array of PartyReference objects to reference the requested parties.
InvolvedParties	Use an array of PartyReference objects to reference the requested parties.
SupplementaryCaseWorkers	Use an array of PartyReference objects to reference the requested parties.
SkipInboxForCaseWorkers	Set SkipInboxForCaseWorkers to true to avoid the matter being moved to the inbox for the supplementary case workers when patching such in SupplementaryCaseWorkers .
Keywords	Use an array of string values refering to the Code value of the enumeration items.
Responsible	Use a PartyReference object to reference the requested party.
SecurityGroups	Use an array of PartyGroupReference objects to reference the required parties.
ExternalAccess	Use a single string value refering to the Code value of the enumeration item.
Alerts	Use an array of complete Alert objects (see Alert above) — but use a PartyReference object for the User part.

Example PATCH request with headers

```
PATCH /matter-url HTTP/1.1
Accept: application/vnd.cbrain.casefile+xml
Content-Type: application/json-patch

[
  {"value":"Outbound","op":"replace","path":"/Type"},
  {"value":"Changed title","op":"replace","path":"/Title"},
  {"value":"Changed text","op":"replace","path":"/Text"},
  {"value":"Closed","op":"replace","path":"/Status"},
  {"value":"2013-05-14T08:15:29.436Z","op":"replace","path":"/Deadline"},
  {"value":"010100-1234","op":"replace","path":"/CprNumber"},
  {"value":"All","op":"replace","path":"/AccessLevel"},
  {"value":true,"op":"replace","path":"/Archived"},
  {"value":{"PartyNo":26},"op":"replace","path":"/Responsible"},
  {"value":[{"PartyNo":26},{"PartyNo":28]},"op":"replace","path":"/InvolvedParties"},
  {"value":{"Contact":"Knud Nielsen <kn@cbrain.dk>"},"op":"replace","path":"/Sender"},
  {"value":
    [
      {"Contact":"John <john@cbrain.dk>"},
      {"Contact":"J\u00FCrgen \u00C6sel\u00F8re <jaeoe@cbrain.com>"}
    ],"op":"replace","path":"/Receivers"},
  {"value":
    [
      {"Contact":"Carsten <carsten@cbrain.dk>"}
    ],"op":"replace","path":"/CcReceivers"},
  {"value":
    [
      {"Contact":"Pia <pia@cbrain.dk>"}
    ],"op":"replace","path":"/BccReceivers"},
  {"value":
    [
      {"Name":"SG4"},
      {"Name":"SG5"}
    ],"op":"replace","path":"/SecurityGroups"}
]
```

Attach matter to case

Attaching a matter to a case is done by sending a `POST` to the link with rel type `http://cbrain.com/casefile/rel/set-case`, found on the matter resource, rather than through a matter `PATCH`. Attaching a matter to a case can change the access restrictions of the matter.

Table 29. Input parameters for attaching a matter to a case

Name	Description
CaseNumber	string (required) The case number of the case, which the matter should be attached to.
InheritNewCaseSecurityGroups	bool Specifies whether or not the matter should inherit the access restriction from the new case. If the matter does not have any security groups itself the value of this parameter is ignored and the value is set to true. If this parameter is not specified the value will default to true.

Delete matter for current user only

A matter may be deleted for the current user only, simply by issuing an `DELETE` request at the matter resource.

Delete matter for everybody

A matter can be deleted completely for everybody, by looking up the "delete matter handler" linked to a matter by the link relation `http://cbrain.com/casefile/rel/matter-delete-everyone`. Follow this link and issue an empty `POST` to it in order to delete the matter.

Multiple identical `POST` requests are allowed (with no additional effect) as deleting a matter multiple times will not have any additional effect.

Delete matter via cleaning list

A matter can be deleted for everybody as if done via a cleaning list. This is somewhat similar to [Delete matter for everybody](#). The main difference being that this resource requires privileges related cleaning lists, in order to be used. If a user has the correct privilege, this resource allows for more aggressive deletion, such as overruling case semantics.

In order to use this resource find the link on the Matter resource, with rel type `http://cbrain.com/casefile/rel/matter-delete-cleaning-list`. Follow the link and issue a `POST`.

Table 30. Inputs for deleting a matter

Name	Description
DeleteReason	string A string which specifies why the matter is deleted. This property is required and is used for accountability purposes.
GenerateReportForDeletedItems	bool A Boolean value specifying if a report-matter should be generated, to document which documents what was deleted as a result of deleting the matter.

Example request for deleting a matter.

```
POST /delete-case-url HTTP/1.1
Content-Type: application/x-www-form-urlencoded

DeleteReason=Deleted+because+of+GDPR&GenerateReportForDeletedItems=true
```

The response from deleting a matter indicates if the deletion successful.

Example response from deleting a matter

```
<DeleteMatterResult>
  <MatterDeleted>false</MatterDeleted>
  <DeleteErrorMessage>A matter was not deleted</DeleteErrorMessage>
  <ReportFailed>true</ReportFailed>
  <ReportErrorMessage>Deadlock</ReportErrorMessage>
</DeleteMatterResult>
```

Table 31. Delete matter response properties

Name	Description
MatterDeleted	bool Indicates if the matter was deleted or not.
DeleteErrorMessage	string Optional error message.
ReportFailed	bool Indicates if the delete operation failed to generate a new matter with a delete-case report.
ReportErrorMessage	string Optional error message for the delete-case report.
Link	List of Link Links to related items. See below.

The XML root element is `<DeleteMatterResult>`.

The link list may contain a link to the generated matter-delete report. The link is named `http://cbrain.com/casefile/rel/matter-delete-report`.

Documents

All documents related to a matter can be found by following the link relation `down` from the matter. The response is a [collection resource](#) with links to all documents.

Set responsible party for matter

Setting responsible party can be done by sending a `POST` to the link with rel type `http://cbrain.com/casefile/rel/set-responsible`, found on the matter resource. This can also be done through a matter [PATCH](#), but this link only updates the responsible party. It can therefore

be used in cases where the user performing the request does not have write access to the matter, but has been given the `ReassignAllDossiers` privilege.

Table 32. Input parameters for setting responsible party

Name	Description
Responsible	PartyItem The party who is responsible for the matter.

Matter version

Represents a pointer to a specific version of a matter. Each version points to a full [matter](#).

Table 33. MatterVersion properties

Name	Description
VersionNumber	int (read-only) Matter version number.
IsActiveVersion	bool (read-only) Flag which indicates if this is the active version of the matter.

Table 34. Expected link relations

Rel	Description
self	Link to the full matter this version represents.

Complete matter structure

Complete matter structure resource

This resource represents a subset of the meta data of a matter and all its documents as one single resource.

Location

The matter structure is reachable from a matter via the link relation <http://cbrain.com/casefile/rel/structure>.

Properties

Table 35. Matter structure properties

Name	Description
Id	int Internal identifier.
MatterNumber	int Unique public matter number.
CaseMatterNumber	int (optional) Matter number relative to it's associated case.
Title	string Matter title.
CreatedDate	DateTime Date and time the matter was created.
ModifiedDate	DateTime Date and time the matter was last modified.
ModifiedBy	PartyItem Last user who modified the matter.
BinnedDate	DateTime (read-only) Date and time the matter was binned (deleted).
CaseNumber	string Number of the case related to the matter. The actual case representation can be found by following the link relation <code>up</code> .
Deadline	DateTime Optional deadline for the matter.
Archived	bool Archival status of the matter.
CprNumber	string An external identification number for a party in case a specific party is related to the matter.
LetterDate	DateTime Letter date ("brevdato" in Danish) — a general purpose date field. If <code>LetterDate</code> is set explicitly then the assigned value will be returned as <code>LetterDate</code> too. If <code>LetterDate</code> is not set then either <code>ReceivedDate</code> or <code>SentDate</code> will be returned as <code>LetterDate</code> (only one of <code>ReceivedDate</code> or <code>SentDate</code> should ever be set).
ReceivedDate	DateTime Received date.
SentDate	DateTime Sent date.

Table 35. Matter structure properties

Name	Description
Responsible	PartyItem The party who is responsible for the matter.
Link	List of Link (read-only) One or more links to other related resources.
Documents	List of Document List of all documents on the matter. Each element in the list represents a subset of the meta data of a document.
Document.Id	int Internal identifier.
Document.Title	string Document title.
Document.ModifiedDate	DateTime Date and time the document was last modified.
Document.ModifiedBy	PartyItem Last user who modified the document.
Document.FileType	string Document file type (file name extension as for instance "pdf" or "doc").
Document.ContentType	string Document mime type (for instance "application/pdf").
Document.FileSize	int Size of document (number of bytes).
Document.Link	Link One or more links to other related resources.

The XML root element is `<Matter>`.

Matter structure example

```
<Matter xmlns="http://cbrain.com/casefile/schema/">
  <Id>517777</Id>
  <MatterNumber>0</MatterNumber>
  <Title>Example matter</Title>
  <CreatedDate>2016-01-08T15:56:48</CreatedDate>
  <ModifiedDate>2016-01-08T16:09:42</ModifiedDate>
  <ModifiedBy>
    <Name>Joe Noel</Name>
    <Email>jn@cbrain.dk</Email>
    <Link href="..."
      rel="self"
      title="Joe Noel" />
  </ModifiedBy>
  <Responsible>...</Responsible>
  <CaseNumber>2016 - 22</CaseNumber>
  <Archived>false</Archived>
  <Deadline xsi:nil="true" />
  <Status>InProgress</Status>
  <LetterDate xsi:nil="true" />
  <ReceivedDate xsi:nil="true" />
  <SentDate xsi:nil="true" />
  <ReminderDate xsi:nil="true" />
  <Documents>
    <Document>
      <Id>517783</Id>
      <Title>Example document</Title>
      <ModifiedDate>2016-01-08T16:09:37.15</ModifiedDate>
      <ModifiedBy>...</ModifiedBy>
      <FileType>doc</FileType>
      <ContentType>application/msword</ContentType>
      <Size>71680</Size>
      <Link href="..."
        rel="self"
        title="Example document" />
    </Document>
  </Documents>
  <Link href="..."
    rel="self"
    title="Example matter" />
  <Link href="..."
    rel="http://cbrain.com/casefile/rel/structure"
    title="Complete matter structure" />
</Matter>
```

Documents

Document resource

This resource represents the meta data for a single document in F2. The actual binary content is contained in a different resource linked by the link relation `http://cbrain.com/casefile/rel/content`.

Location

Documents can be located directly by ID using the URL template found in the service index. In addition to this there are links from a matter to all the documents on the matter.

Lookup by document ID

The link relation `http://cbrain.com/casefile/rel/document-by-id` identifies a URL template for looking up documents by ID. The template uses the template parameter `id` for the document ID.

A `GET` on the generated URL will return a redirect to the corresponding document resource.

Example link from service document

```
{
  "http://cbrain.com/casefile/rel/document-by-id":
  {
    href: "http://.../{id}",
    title: "Get document by ID"
  }
}
```


Properties

Table 36. Document properties

Name	Description
Id	int (read-only) Internal identifier.
Title	string (255 characters) Document title.
ModifiedDate	DateTime (read-only) Date and time the document was last modified.
Description	string Document description.
FileType	string (read-only) Document file type (file name extension as for instance "pdf" or "doc" without preceding dot). The file type is set automatically on document creation.
ContentType	string (read-only) Document mime type (for instance application/pdf). The document type is set automatically on document creation.
FileSize	int (read-only) Size of document (number of bytes).
PreferredAttachmentFormat	string Preferred format for attaching the document to its matter when sending. Possible values are "PDF" and "Original" .

Table 36. Document properties

Name	Description
Link	List of Link (read-only) One or more links to other related resources.

The XML root element is `<Document>`.

Document example

```
<Document xmlns="http://cbrain.com/casefile/schema/">
  <Id>280016</Id>
  <Title>Test document</Title>
  <ModifiedDate>2022-03-30T08:49:16.27</ModifiedDate>
  <ModifiedBy>
    <Name>Case Worker 1</Name>
    <EMail>test@cbrain.com</EMail>
    <Link href="..."
      rel="self"
      type="application/vnd.cbrain.casefile+xml" />
    <Type>User</Type>
    <Id>634396</Id>
    <PartyNumber>2000001001</PartyNumber>
    <CPRCVR />
    <CVR_P />
  </ModifiedBy>
  <Description>Blah blah ...</Description>
  <FileType>html</FileType>
  <ContentType>text/html</ContentType>
  <Size>31</Size>
  <PreferredAttachmentFormat />
  <OutputManagerTemplateType>None</OutputManagerTemplateType>
  <Link href="..."
    rel="self"
    type="application/vnd.cbrain.casefile+xml" />
  <Link href="..."
    rel="up"
    type="application/vnd.cbrain.casefile+xml" />
  <Link href="..."
    rel="alternate"
    type="application/vnd.cbrain.f2.client" />
  <Link href="..."
    rel="http://cbrain.com/casefile/rel/content" />
  <Link href="..."
    rel="http://cbrain.com/casefile/rel/upload-validation" />
  <Link href="..."
    rel="http://cbrain.com/casefile/rel/text-content" />
</Document>
```

Create new document

A document including binary content may be created using the Post Once Exactly pattern (see [Data manipulation](#)). Follow the link named `http://cbrain.com/casefile/rel/create-document` from a specific matter in order to add the document to that matter.

Table 37. Input parameters for creating a document

Name	Description
Title	string (required, 255 characters) The title of the new document. This will override the <code>Content-Disposition</code> attribute <code>filename</code> and should be used to supply a human readable document title. We recommend that <code>Title</code> is used to supply a document title without the file extension such that a file named "Document.pdf" will be shown as "Document" in the F2 client. <code>Title</code> may also include UTF-8 characters which is not possible with the <code>filename</code> attribute of the <code>Content-Disposition</code> element in multipart encodings.
Description	string Document description.
PreferredAttachmentFormat	string Preferred format for attaching the document to its matter when sending. Possible values are "PDF" and "Original".
File	binary data The actual binary file data to be uploaded. The filename-extension of <code>Content-Disposition</code> attribute <code>filename</code> is used to define the file type, so be careful to supply an informative filename extension, as F2 otherwise will have trouble to display the content correctly.
TemplateKey	string As an alternative to providing file content with the <code>File</code> property, the document may be generated from an F2 template. <code>TemplateKey</code> specifies the path of the template in the F2 template tree (as shown in the template properties in the F2 client).
OutputManagerTemplateType	string Specifies how the document can be used with the output manager ("Fjernprint" in Danish). Possible values are: none this document cannot be sent via the output manager. MainDocument this document can be sent as the main document. Attachment this document can be sent as an attachment.

There is no need to supply any reference to the parent matter—that is embedded in the URL of the factory resource.

Example request for creating a new document

```
POST /document-create-url HTTP/1.1
Content-Length: 0

> HTTP/1.1 201 Created
> Location: /document-create-poc-url
> Content-Length: 0

POST /document-create-poc-url HTTP/1.1
Accept: application/vnd.cbbrain.casefile+xml
Content-Type: multipart/form-data; boundary=b00d75a3-bccc-4f5b-978b-225e55c5b521

--b00d75a3-bccc-4f5b-978b-225e55c5b521
Content-Disposition: form-data; name="Title"
Content-Type: text/plain; charset=utf-8

File and Template example with UTF-8 emoji 🤖 in title
--b00d75a3-bccc-4f5b-978b-225e55c5b521
Content-Disposition: form-data; name="File"; filename="my-document.txt"
Content-Type: application/octet-stream

... binary data ...
--b00d75a3-bccc-4f5b-978b-225e55c5b521
Content-Disposition: form-data; name="TemplateKey"
Content-Type: text/plain; charset=utf-8

Standard/Template1
--b00d75a3-bccc-4f5b-978b-225e55c5b521--
```

Get binary content

Follow the link relation `http://cbbrain.com/casefile/rel/content` from the document meta data and issue an `GET` request to get the binary content of the document.

Get PDF content

Follow the link relation `http://cbbrain.com/casefile/rel/pdf-content` from the document meta data and issue an `GET` request to get the PDF content of the document. The link is only included in the meta data if a PDF representation exists.

Get flattened PDF content

Follow the link relation `http://cbbrain.com/casefile/rel/pdf-content-flattened` from the document meta data and issue an `GET` request to get the flattened PDF content of the document. The link is only included in the meta data if a PDF representation exists.

Get text content

Follow the link relation `http://cbbrain.com/casefile/rel/text-content` from the document meta data and issue an `GET` request to get the text content of the document. The link is only included in the meta data if text content has been extracted from the document. The text content will contain all text elements in the original document extracted as separate words in a text document.

Modify document meta data

Document meta data may be modified by issuing an `PATCH` request targeted at the document resource URL.

Example PATCH request with headers

```
PATCH /document-url HTTP/1.1
Accept: application/vnd.cbrain.casefile+xml
Content-Type: application/json-patch+json

[
  {"value":"New document title","op":"replace","path":"/Title"},
  {"value":"New description","op":"replace","path":"/Description"}
]
```

Modifying binary content

The binary content of a document may be replaced by following the link relation `http://cbrain.com/casefile/rel/content` from the document meta data and issue an `PUT` request at the content URL, including the binary document content in the HTTP body.

Upload validation

The internal upload validation mechanism of F2 may be invoked by looking up the validation URL template from the link relation `http://cbrain.com/casefile/rel/upload-validation` in the document meta data or in the links of a matter. The document template contains the following two parameters which need to be replaced in the URL before issuing an `GET` to the resulting URL:

Table 38. Upload validation URL parameters

Name	Description
filename	string Filename of document to upload.
size	int Size, in bytes, of document to upload.

Table 39. Upload validation result properties

Name	Description
Result	string The validation result. Possible values are "Valid", "ValidWithWarnings" and "Invalid". If the result is "ValidWithWarnings" the user should be asked for confirmation if possible.
Messages	List of Message (string) One or more warnings or error messages if the result is "ValidWithWarnings" or "Invalid". Each Message contains a string. If possible, these messages should be shown to the user.
Link	List of Link (read-only) One or more links to other related resources.

Upload validation request example

```
GET .../upload-validation-url?filename=My+document.docx&size=10000 HTTP/1.1
Accept: application/vnd.cbrain.casefile+xml
```

Upload validation response example

```
HTTP/1.1 200 OK
Content-Type: application/vnd.cbrain.casefile+xml

<UploadValidation xmlns="http://cbrain.com/casefile/schema/">
  <Result>ValidWithWarnings</Result>
  <Messages>
    <Message>The document you are uploading is very large</Message>
  </Messages>
  <Link href="..." rel="up" title="Matter" />
</UploadValidation>
```

Delete document

A document may be deleted simply by issuing an HTTP DELETE request to the document meta data URL.

Table 40. Expected link relations

Rel	Description
self	
up	Link to parent matter.
http://cbrain.com/casefile/rel/content	Link to binary document content.
http://cbrain.com/casefile/rel/upload-validation	Link to upload validation, used for validation before updating the document. See Upload validation .
http://cbrain.com/casefile/rel/text-content	Link to textual document content (it should be noted that this operation runs asynchronously, see section 12.60 for clarification).
http://cbrain.com/casefile/rel/pdf-content	Link to PDF rendering of content (if a PDF version is available).

Chats

Chat resource

This resource represents a single F2 chat with all chat entries included.

Location

Chats are located by following the link relation `http://cbrain.com/casefile/rel/chats` from a matter. This leads to a resource (see [Chats](#)) containing links to all chats on the matter—follow each link to get the chat resource.

Properties

Table 41. Chat properties

Name	Description
Id	int (read-only) Internal identifier.
Participants	List of Participant One or more participants of the chat communication.
Participants.Participant.Party	PartyItem (read-only) A party who is a participant of the chat.
Participants.Participant.HasSeen	bool (read-only) Indication of whether or not the participant has seen/read the chat.
Entries	List of Entry One or more chat entries (i.e. messages in the chat).
Entries.Entry.Text	string (read-only) The message text of the chat entry.
Entries.Entry.CreatedBy	PartyItem (read-only) The creator of the message.
Entries.Entry.CreatedDate	DateTime (read-only) The creation date of the message.
Link	List of Link (read-only) One or more links to other related resources.

The XML root element is `<Chat>`.

Chat example

```
<Chat xmlns="http://cbrain.com/casefile/schema/">
  <Id>10514</Id>
  <Participants>
    <Participant>
      <Party>
        <Name>Brian Ipsum Hansen</Name>
        <Link ... rel="self" title="Brian Ipsum Hansen" />
      </Party>
      <HasSeen>true</HasSeen>
    </Participant>
    <Participant> ... </Participant>
    ...
  </Participants>
  <Entries>
    <Entry>
      <Id>6798</Id>
      <Text>Are we ready to go?</Text>
      <CreatedBy>
        <Name>Brian Ipsum Hansen</Name>
        <Email>bih@thecompany.net</Email>
        <Link ... />
      </CreatedBy>
      <CreateDate>2016-01-12T11:27:13.16</CreateDate>
    </Entry>
    <Entry>
      <Id>7185</Id>
      <Text>Yes, I think so.</Text>
      <CreatedBy>
        <Name>Karin Ingemann Petersen</Name>
        <Email>kip@thecompany.net</Email>
        <Link ... />
      </CreatedBy>
      <CreateDate>2016-01-12T11:28:31.007</CreateDate>
    </Entry>
  </Entries>
  <Link ... rel="self" title="Chat" />
  <Link ... rel="up" title="Parent matter" />
  <Link ... rel="http://cbrain.com/casefile/rel/add-chat-entry"
    title="Add entry" />
  <Link ... rel="http://cbrain.com/casefile/rel/add-chat-participants"
    title="Add participants" />
  <Link ... rel="http://cbrain.com/casefile/rel/remove-chat-participants"
    title="Remove participants" />
</Chat>
```

Add chat entry

A chat entry may be created using the Post Once Exactly pattern (see [Data manipulation](#)). Follow the link relation `http://cbrain.com/casefile/rel/add-chat-entry` from an existing chat to locate the relevant POE resource.

Table 42. Input parameters for adding a chat entry

Name	Description
Text	string The text content of the new entry.

Add chat entry example

```
POST /chat-entry-add-url HTTP/1.1
Content-Length: 0

> HTTP/1.1 201 Created
> Location: /chat-entry-add-poe-url
> Content-Length: 0

POST /chat-entry-add-poe-url HTTP/1.1
Accept: application/vnd.cbrain.casefile+xml
Content-Type: application/x-www-form-urlencoded

Text=Thank+you

> HTTP/1.1 200 OK
> Content-Type: application/vnd.cbrain.casefile+xml
> Location: /chat-url
> Content-Length: 3139
> ... XML representation of resulting chat ...
```

Add chat participants

Follow the link relation `http://cbrain.com/casefile/rel/add-chat-participants` from an existing chat to locate the resource for adding chat participants. Then issue an `POST` request containing the list of participant references.

Multiple `POST` requests with the same participants will only result in the creation of one occurrence of each participant.

Table 43. Input parameters for adding chat participants

Name	Description
Participants	List of PartyReference Participants to be added to the chat/note.

There is no need to supply any reference to the parent chat — that is embedded in the URL of the resource.

Add chat participants example

```
POST /chat-participants-add-url HTTP/1.1
Content-Type: application/x-www-form-urlencoded
Content-Length: 136

Participants%3a0.PartyNo=117&Participants%3a0.PickFirstMatch=false&Participants%3a1.Login=Mr+W
hite&Participants%3a1.PickFirstMatch=false
```

Remove chat participants

Follow the link relation `http://cbrain.com/casefile/rel/remove-chat-participants` from an existing chat to locate the action resource for removing chat participants. Then issue an `POST` request containing the list of participant references.

Multiple `POST` requests with the same participants is allowed (with no additional effect) as removing a party that is not a participant has no effect.

Table 44. Input parameters for removing chat participants

Name	Description
Participants	List of PartyReference Participants to be removed from the chat.

There is no need to supply any reference to the parent chat — that is embedded in the URL of the resource.

Remove chat participants example

```
POST /chat-participants-remove-url HTTP/1.1
Content-Type: application/x-www-form-urlencoded
Content-Length: 136

Participants%3a0.PartyNo=117&Participants%3a0.PickFirstMatch=false&Participants%3a1.Login=Mr+W
hite&Participants%3a1.PickFirstMatch=false
```

Chats

This resource represents a list of condensed chats of a matter. The condensed chats contain only the last entry/message on the chat. For each condensed chat the full chat with all entries can be reached via the `self` link.

Location

The chat list is located by following the link relation `http://cbrain.com/casefile/rel/chats` from a matter.

Properties

The chat list has one `<Chat>` element for each chat in the list. Each chat contains the following properties:

Table 45. Chat list entry properties

Name	Description
Id	int Internal identifier.
LastEntry	Entry The last entry/message on the chat.
LastEntry.Text	string The last message text of the chat.
LastEntry.CreatedBy	PartyItem The creator of the last message.
LastEntry.CreatedDate	DateTime The creation date of the last message.
Chat.Link	List of Link Contains a <code>self</code> link to the full chat .
Link	List of Link One or more links to other related resources.

The XML root element is `<Chats>`.

Chat list example

```
<Chats xmlns="http://cbrain.com/casefile/schema/">
  <Chat>
    <Id>10514</Id>
    <LastEntry>
      <Text>The scope is extended to also cover human resources</Text>
      <CreatedBy>
        <Name>Karin Ingemann Petersen</Name>
        <Email> kip@thecompany.net</Email>
        <Link href="..." type="application/vnd.cbrain.casefile+xml"
          rel="self" title="Karin Ingemann Petersen" />
      </CreatedBy>
      <CreatedDate>2016-01-12T11:28:31.007</CreatedDate>
    </LastEntry>
    <Link href="..." type="application/vnd.cbrain.casefile+xml"
      rel="self" title="Full chat" />
  </Chat>
  <Chat>
    <Id>10525</Id>
    <LastEntry>
      ...
    </LastEntry>
    <Link ... />
  </Chat>
  <Link href="..." type="application/vnd.cbrain.casefile+xml"
    rel="self" title="Chats" />
  <Link href="..." type="application/vnd.cbrain.casefile+xml"
    rel="up" title="Parent matter" />
</Chats>
```

Add new chat

A new chat may be created using the Post Once Exactly pattern (see [Data manipulation](#)). Follow the link named `http://cbrain.com/casefile/rel/create-chat` from a specific matter in order to add the chat to that matter.

Table 46. Input parameters for adding a chat

Name	Description
Text	string The text content of the first entry in the new chat.
Participants	List of PartyReference named Participant A list of PartyReferences used to identify who is included in the chat. Note that the chat creator is added implicitly and does not need to appear in the Participants list.

Add chat example

```
POST /chat-create-url HTTP/1.1
Content-Length: 0

> HTTP/1.1 201 Created
> Location: /chat-create-poe-url
> Content-Length: 0

POST /chat-create-poe-url HTTP/1.1
Accept: application/vnd.cbrain.casefile+xml
Content-Type: application/x-www-form-urlencoded
Content-Length: 327

*** Line breaks included for readability ***
Text=Please+take+a+look+at+this
&Participants%3a0.Login=Sagsbehandler1
&Participants%3a0.PickFirstMatch=false
&Participants%3a1.Login=Sagsbehandler24
&Participants%3a1.PickFirstMatch=false
&Participants%3a2.PartyNo=2000000002
&Participants%3a2.PickFirstMatch=false
&Participants%3a3.PartyNo=125
&Participants%3a3.PickFirstMatch=false

> HTTP/1.1 201 Created
> Content-Type: application/vnd.cbrain.casefile+xml
> Location: /chat-url
> Content-Length: 3467
> ... Chat XML ...
```

Notes

Note resource

This resource represents a single F2 note with all note entries included.

Location

Notes are located by following the link relation `http://cbrain.com/casefile/rel/notes` from a matter or case. This leads to a resource (see [Notes](#)) containing links to all notes on the matter/case — follow each link to get the note resource.

Properties

Table 47. Note properties

Name	Description
Id	int (read-only) Internal identifier.
Participants	List of Participant One or more participants of the note communication.
Participants.Participant.Party	PartyItem (read-only) A party who is a participant of the note.
Participants.Participant.HasSeen	bool (read-only) Indication of whether or not the participant has seen/read the note.
Entries	List of Entry One or more note entries.
Entries.Entry.Text	string (read-only) The text of the note entry.
Entries.Entry.CreatedBy	PartyItem (read-only) The creator of the note entry.
Entries.Entry.CreatedDate	DateTime (read-only) The creation date of the note entry.
Link	List of Link (read-only) One or more links to other related resources.

The XML root element is `<Note>`.

Note example

```
<Note xmlns="http://cbrain.com/casefile/schema/">
  <Id>10513</Id>
  <Participants>
    <Participant>
      <Party>
        <Name>Brian Ipsum Hansen</Name>
        <Link href="..." type="application/vnd.cbrain.casefile+xml"
          rel="self" title="Brian Ipsum Hansen" />
      </Party>
      <HasSeen>true</HasSeen>
    </Participant>
    <Participant> ... </Participant>
    ...
  </Participants>
  <Entries>
    <Entry>
      <Id>6758</Id>
      <Text>This file describes the time line of the project.</Text>
      <CreatedBy>
        <Name>Brian Ipsum Hansen</Name>
        <EMail>bih@thecompany.net</EMail>
        <Link ... />
      </CreatedBy>
      <CreatedDate>2016-01-12T11:27:13.16</CreatedDate>
    </Entry>
    <Entry>
      <Id>6785</Id>
      <Text>The scope is extended to also cover human resources</Text>
      <CreatedBy>
        <Name>Karin Ingemann Petersen</Name>
        <EMail>kip@thecompany.net</EMail>
        <Link ... />
      </CreatedBy>
      <CreatedDate>2016-01-12T11:28:31.007</CreatedDate>
    </Entry>
  </Entries>
  <Link ... rel="self" title="Note" />
  <Link ... rel="up" title="Parent matter" />
  <Link ... rel=http://cbrain.com/casefile/rel/add-note-entry
    title="Add entry" />
  <Link ... rel=http://cbrain.com/casefile/rel/add-note-participants
    title="Add participants" />
  <Link ... rel=http://cbrain.com/casefile/rel/remove-note-participants
    title="Remove participants" />
</Note>
```

Add note entry

A note entry may be created using the Post Once Exactly pattern (see [Data manipulation](#)). Follow the link relation `http://cbrain.com/casefile/rel/add-note-entry` from an existing note to locate the relevant POE resource.

Table 48. Input parameters for adding a note entry

Name	Description
Text	string The text content of the new entry.

Add note entry example

```
POST /note-entry-add-url HTTP/1.1
Content-Length: 0

> HTTP/1.1 201 Created
> Location: /note-entry-add-poe-url
> Content-Length: 0

POST /note-entry-add-poe-url HTTP/1.1
Accept: application/vnd.cbrain.casefile+xml, application/vnd.cbrain.casefile+xml
Content-Type: application/x-www-form-urlencoded
Content-Length: 14

Text=Thank+you

> HTTP/1.1 200 OK
> Content-Type: application/vnd.cbrain.casefile+xml
> Location: /note-url
> Content-Length: 3168
> ... XML representation of resulting note ...
```

Add note participants

Follow the link relation <http://cbrain.com/casefile/re1/add-note-participants> from an existing note to locate the resource for adding note participants. Then issue an **POST** request containing the list of participant references.

Multiple **POST** requests with the same participants will only result in the creation of one occurrence of each participant.

Table 49. Input parameters for adding note participants

Name	Description
Participants	List of PartyReference Participants to be added to the note/note.

There is no need to supply any reference to the parent note — that is embedded in the URL of the resource.

Add note participants example

```
POST /note-participants-add-url HTTP/1.1
Content-Type: application/x-www-form-urlencoded
Content-Length: 136

Participants%3a0.PartyNo=117&Participants%3a0.PickFirstMatch=false&Participants%3a1.Login=Mr+W
hite&Participants%3a1.PickFirstMatch=false
```

Remove note participants

Follow the link relation `http://cbrain.com/casefile/rel/remove-note-participants` from an existing note to locate the action resource for removing note participants. Then issue an `POST` request containing the list of participant references.

Multiple `POST` requests with the same participants is allowed (with no additional effect) as removing a party that is not a participant has no effect.

Table 50. Input parameters for removing note participants

Name	Description
Participants	List of PartyReference Participants to be removed from the note.

There is no need to supply any reference to the parent note — that is embedded in the URL of the resource.

Remove note participants example

```
POST /note-participants-remove-url HTTP/1.1
Content-Type: application/x-www-form-urlencoded
Content-Length: 136

Participants%3a0.PartyNo=117&Participants%3a0.PickFirstMatch=false&Participants%3a1.Login=Mr+W
hite&Participants%3a1.PickFirstMatch=false
```

Notes

This resource represents a list of condensed notes of a matter or case. The condensed notes contain only the last entry on the note. For each condensed note the full note with all entries can be reached via the `self` link.

Location

The note list is located by following the link relation `http://cbrain.com/casefile/rel/notes` from a matter/case.

Properties

The note list has one `<Note>` element for each note in the list. Each note contains the following properties:

Table 51. Note list entry properties

Name	Description
Id	int Internal identifier.
LastEntry	Entry The last entry on the note.
LastEntry.Text	string The last entry text of the note.
LastEntry.CreatedBy	PartyItem The creator of the last entry.
LastEntry.CreatedDate	DateTime The creation date of the last note entry.
Note.Link	List of Link Contains a <code>self</code> link to the full note .
Link	List of Link One or more links to other related resources.

The XML root element is `<Notes>` .

Note list example

```
<Notes xmlns="http://cbrain.com/casefile/schema/">
  <Note>
    <Id>10514</Id>
    <LastEntry>
      <Text>The scope is extended to also cover human resources</Text>
      <CreatedBy>
        <Name>Karin Ingemann Petersen</Name>
        <Email> kip@thecompany.net</Email>
        <Link href="..." type="application/vnd.cbrain.casefile+xml"
          rel="self" title="Karin Ingemann Petersen" />
      </CreatedBy>
      <CreatedDate>2016-01-12T11:28:31.007</CreatedDate>
    </LastEntry>
    <Link href="..." type="application/vnd.cbrain.casefile+xml"
      rel="self" title="Full note" />
  </Note>
  <Note>
    <Id>10525</Id>
    <LastEntry>
      ...
    </LastEntry>
    <Link ... />
  </Note>
  <Link href="..." type="application/vnd.cbrain.casefile+xml"
    rel="self" title="Notes" />
  <Link href="..." type="application/vnd.cbrain.casefile+xml"
    rel="up" title="Parent matter" />
</Notes>
```

Add new note

A new note may be created using the Post Once Exactly pattern (see [Data manipulation](#)). Follow the link named `http://cbrain.com/casefile/rel/create-note` from a specific matter or case in order to add the note to it.

Table 52. Input parameters for adding a note

Name	Description
Text	string The text content of the first entry in the new note.
Participants	List of PartyReference named Participant A list of PartyReferences used to identify who is included in the note. Note that the note creator is added implicitly and does not need to appear in the Participants list.

Add note example

```
POST /note-create-url HTTP/1.1
Content-Length: 0

> HTTP/1.1 201 Created
> Location: /note-create-poe-url
> Content-Length: 0

POST /note-create-poe-url HTTP/1.1
Accept: application/vnd.cbrain.casefile+xml
Content-Type: application/x-www-form-urlencoded
Content-Length: 327

*** Line breaks included for readability ***
Text=This+is+the+conclusion
&Participants%3a0.Login=Sagsbehandler1
&Participants%3a0.PickFirstMatch=false
&Participants%3a1.Login=Sagsbehandler24
&Participants%3a1.PickFirstMatch=false
&Participants%3a2.PartyNo=2000000002
&Participants%3a2.PickFirstMatch=false
&Participants%3a3.PartyNo=125
&Participants%3a3.PickFirstMatch=false

> HTTP/1.1 201 Created
> Content-Type: application/vnd.cbrain.casefile+xml
> Location: /note-url
> Content-Length: 3467
> ... Note XML ...
```

Party resources

Common Party properties

Table 53. Common Party properties

Name	Description
Id	int Internal identifier.
Name	string Name of party.
PartyNumber	int A number that identifies a party in F2. This number is visible in the F2 client as opposed to the <code>Id</code> which is internal (not visible). It is the same as <code>PartyNo</code> used in party references.
Relation	string The party's relation to a specific matter or case. Examples of relations could be Sender, CaseWorker, ResponsibleUnit, Approver, etc. The relation values are common to all F2 installations.
PartyType	string The party type title assigned to the party in the context of a specific matter or case. The party types are F2 customer specific i.e. configured for each F2 installation.
PartyTypeKey	string The identifier of the party type.
Email	string (100 characters) The email address of the party.
CVRCPR	string (20 characters) An external identification number for the party. For example in a Danish installation it could be the nationwide CVR number of a company or the nationwide CPR number of a citizen.
CVR_P	string (20 characters) An external identification number for the party, to be combined with the CVRCPR number. For example, in a Danish installation it could be combined with the nationwide CVR number of a company, where one company with the same CVR number can have different branches that each have a unique CVR_P number.
Phone	string (50 characters) The phone number of the party.
LocalNumber	string (10 characters) Local extension to the phone number of the party.
MobilePhone	string (50 characters) The mobile phone number of a party.
Address1	string (255 characters) The (first line of the) address of the party. Usually this contains the street name

Table 53. Common Party properties

Name	Description
	and house number.
Address2	string (255 characters) The (second line of the) address of the party.
PostalCode	string (40 characters) The postal code of the party's address.
ProtectedAddress	bool Whether or not this party is marked for having a protected address.
City	string (100 characters) The city of the party's address.
CountryCode	string (10 characters) The country code of the party's address.
Telefax	string (50 characters) The fax number of the party.
IsRegistryVersion	bool Indicates whether this party representation is the current registry version or a snapshot of the original.
Link	List of Link One or more links to other related resources.
Active	bool Whether or not this party is marked as active in F2.

Organization

This resource represents an organization in F2.

Properties

An organization includes only the common properties listed above.

The XML root element is `<Organization>`.

Table 54. Expected link relations

Rel	Description
self	Link to self
up	Link to the parent of the party. A link only exists if there is a parent for the party.
down	Link to a list of the children of the party.
http://cbrain.com/casefile/rel/field-data	Link to additional field data specified for the party. See F2-REST Documentation .
http://cbrain.com/casefile/rel/registry	Link to current registry version of the party. This link is only available on the original snapshot version of the party.

External party

This resource represents an external party in F2.

Properties

In addition to the properties listed above, an external party representation includes the following properties:

Table 55. External party properties

Name	Description
ContactPerson	string (255 characters) This property can be used to store the name of a contact person related to the party.
PostageGroup	string (100 characters) Postage group.
SynchronizationKey	string (255 characters) A synchronization key related to the party. Also known as the External Party Number from the Party Registry within F2.
Web	string (100 characters) This property can be used for storing a web address related to the party.
Organization	F2-REST Documentation The organization which the external party belongs to.

The XML root element is `<External>`.

External party example

```
<External xmlns="http://cbrain.com/casefile/schema/">
  <Id>400775</Id>
  <Name>The Chocolate Company</Name>
  <Active>true</Name>
  <PartyType>PropertyOwner</PartyType>
  <Address1>Helevejen 1</Address1>
  <Address2 />
  <City>Egebjerg</City>
  <CountryCode>DK</CountryCode>
  <CVRCPR>12345678</CVRCPR>
  <CVR_P>0987654321</CVR_P>
  <EMail>tcc@example.dk</EMail>
  <LocalNumber>123</LocalNumber>
  <MobilePhone>12345678</MobilePhone>
  <Phone>12345678</Phone>
  <PostalCode>2751</PostalCode>
  <ProtectedAddress>false</ProtectedAddress >
  <SynchronizationKey>xpq123</SynchronizationKey >
  <Telefax>22334455</Telefax>
  <Web>http://cbrain.dk</Web>
  <Organization>
    <Name>Parent company</Name>
    <Type>Unit</Type>
    <Id>2156</Id>
    <PartyNumber>20</PartyNumber>
    <CPRCVR>12344321</CPRCVR>
    <CVR_P>1234554321</CVR_P>
    <Name>Parent company</Name>
    <Link href="..."
      rel="self"
      type="application/vnd.cbrain.casefile+xml"
      title="Parent company" />
  </Organization>
  <Link href="..."
    rel="self"
    type="application/vnd.cbrain.casefile+xml"
    title="The Chocolate Company" />
</External>
```

Table 56. Expected link relations

Rel	Description
self	Link to self
up	Link to the parent of the party. A link only exists if there is a parent for the party.
down	Link to a list of the children of the party.
http://cbrain.com/casefile/rel/field-data	Link to additional field data specified for the party. See F2-REST Documentation .
http://cbrain.com/casefile/rel/registry	Link to current registry version of the party. This link is only available on the original snapshot version of the party.
http://cbrain.com/casefile/rel/move-external-party	Link for moving the external party. This will update the Organization property. See Move external party .

Create external party

An external party may be created using the Post Once Exactly pattern (see [Data manipulation](#)). Follow the link named `http://cbrain.com/casefile/rel/create-external-party` from the service index and `POST` the following parameters to the referenced resource.

Table 57. Input parameters for creating an external party

Name	Description
Name	string (255 characters) The name of the external party to be created.
ParentOrganization	PartyReference This property specifies the organization that the external party is to be created under. If this property is not specified, a standard organization will be selected instead.
Address1	string
Address2	string
City	string
ContactPerson	string
CountryCode	string
CVRCPR	string
CVR_P	string
EMail	string
LocalNumber	string
MobilePhone	string
Phone	string
PostageGroup	string
PostalCode	string
ProtectedAddress	string
Relation	string
SynchronizationKey	string
Telefax	string
Web	string

External party creation example

```
POST party-create-url HTTP/1.1
Content-Length: 0

> HTTP/1.1 201 Created
> Location: party-create-poe-url
> Content-Length: 0

POST party-create-poe-url HTTP/1.1
Accept: application/vnd.cbrain.casefile+xml
Content-Type: application/x-www-form-urlencoded

ContactPerson=George Locus&
PostageGroup=Group+A&
SynchronizationKey=637840753521414881&
Web=https%3A%2F%2Fwww.dr.dk&
Address1=The+Road+1&
Address2=&
City=Copenhagen&
CountryCode=DK&
CVRCPR=130896-1234&
CVR_P=9876543210&
EMail=hello%40cbrain.net&
LocalNumber=1234&
MobilePhone=12345678&
Name=Adam+Adler&
Phone=78907890&
PostalCode=1234&
ProtectedAddress=false&
Telefax=SomeFaxNo
```

Move external party

The location of an external party can be changed from one organization to another. Look up the "party moving" resource through the link relation `http://cbrain.com/casefile/rel/move-party` found in the external party's link collection. Then issue an `POST` request to the resource with the following input parameters:

Table 58. Input parameters for moving an external party

Name	Description
Destination	PartyReference A party reference specifying the new organization of the external party.

Move external party example

```
POST move-party-url HTTP/1.1
Content-Type: application/x-www-form-urlencoded

Destination.PartyNo=11163&Destination.PickFirstMatch=false
```

User

This resource represents an F2 user belonging to an organization.

Properties

In addition to the properties listed above, a user representation includes the following properties:

Table 59. External party properties

Name	Description
Initials	string (255 characters) The initials of the user.
Organization	F2-REST Documentation The organization which the user belongs to.

The XML root element is `<User>`.

User example

```
<User xmlns="http://cbrain.com/casefile/schema/">
  <Id>400776</Id>
  <Name>The Boss</Name>
  <Organization>
    <Name>Parent company</Name>
    <Type>Unit</Type>
    <Id>2156</Id>
    <PartyNumber>20</PartyNumber>
    <CPRCVR>12344321</CPRCVR>
    <CVR_P>1234554321</CVR_P>
    <Name>Parent company</Name>
    <Link href="..."
      rel="self"
      type="application/vnd.cbrain.casefile+xml"
      title="Parent company" />
  </Organization>
  <Link href="..."
    rel="self"
    type="application/vnd.cbrain.casefile+xml"
    title="The Boss" />
</User>
```

Table 60. Expected link relations

Rel	Description
self	Link to self
up	Link to the parent of the party. A link only exists if there is a parent for the party.
http://cbrain.com/casefile/rel/field-data	Link to additional field data specified for the party. See F2-REST Documentation .
http://cbrain.com/casefile/rel/registry	Link to current registry version of the party. This link is only available on the original snapshot version of the party.

Party collection

Party collection resource

This resource represents a collection of parties such as all the parties related to a case. The collection represents the original version of the parties.

The XML representation of the collection uses different XML element names for the different types of parties.

There are different elements representing a party; a [User](#) element, an [Organization](#) element, an [External](#) party element, and possibly other kinds of party elements in future versions. These party elements differ by the properties they may have.

Properties

Table 61. Partylist properties

Name	Description
Items	mixed Contains one or more parties. The party sub elements can be one or more of User elements, Organization elements, External party elements — and possibly other kinds in future versions.
Link	List of Link (read-only) One or more links to other related resources.

The XML root element is `<Parties>`.

Party list example

```
<Parties xmlns="http://cbrain.com/casefile/schema/">
  <Items>
    <External>
      ...
    </External>
    <Organization>
      ...
    </Organization>
    ...
  </Items>
  <Link href="..."
        rel="self"
        type="application/vnd.cbrain.casefile+xml"
        title="Parties" />
  <Link href="..."
        rel="up"
        type="application/vnd.cbrain.casefile+xml"
        title="Parent" />
</Parties>
```


Table 62. Expected link relations

Rel	Description
self	
up	Link to the parent of the party list (such as the case which the parties are referenced from).

Approvals

Approval resource

This resource represents a single approval process associated with a [matter](#).

Location

An approval associated with a matter can be found by following the link `http://cbrain.com/casefile/rel/approval` from the matter's link collection. Approvals can also be found by looking up on approval ID directly.

Lookup by approval ID

The link relation `http://cbrain.com/casefile/rel/approval-by-id`, found in the service index, identifies a URL template for looking up approvals by ID. The template uses the template parameter `id` for the approval ID.

A `GET` on the generated URL will return a redirect to the corresponding approval resource.

Example link from service document

```
{
  "http://cbrain.com/casefile/rel/approval-by-id":
  {
    href: "http://.../{id}",
    title: "Get approval by ID"
  }
}
```

Properties

Table 63. Approval properties

Name	Description
Id	int Internal approval identifier.
MatterId	int Associated matter identifier.
ApprovalType	Enumeration Approval type.
Deadline	DateTime Approval deadline.
Urgent	bool Urgent or non-urgent approval.
Text	string Text content of approval process document.
Responsible	Responsible Details about responsible party.
Responsible.Party	PartyItem Party responsible for the approval process.
Responsible.Comment	string Responsible party's comment.
CreatedDate	DateTime Timestamp for when the approval process was created.
StartedDate	DateTime Timestamp for when the approval process was started.
FinishedDate	DateTime Timestamp for when the approval process was finished (either approved or cancelled).
Status	string Status of the approval process. Possible values are: - Created - InProgress - Cancelled - Approved
StatusDetails	string Human readable text describing in-depth status of the approval process.

Table 63. Approval properties

Name	Description
StatusDescription	string Human readable text describing status of the approval process.

The XML root is `Approval`.

Approval example

```
<Approval xmlns="http://cbrain.com/casefile/schema/">
  <Id>9290</Id>
  <MatterId>279243</MatterId>
  <ApprovalType>
    <Title>Type 1</Title>
    <Path>Type 1</Path>
    <Code>Type_1</Code>
    <Link href="..."
      rel="http://cbrain.com/casefile/rel/enum-type"
      type="application/vnd.cbrain.casefile+xml" />
  </ApprovalType>
  <Deadline>2020-11-30T00:00:00</Deadline>
  <Urgent>true</Urgent>
  <Text>... HTML content ...</Text>
  <Responsible>
    <Party>
      <Name>Maria Suneson</Name>
      <EMail>test@cbrain.com</EMail>
      <Link href="..."
        rel="self"
        type="application/vnd.cbrain.casefile+xml" />
      <Type>User</Type>
      <Id>407189</Id>
      <PartyNumber>2000001001</PartyNumber>
      <CPRCVR />
      <CVR_P />
    </Party>
  </Responsible>
  <CreatedDate>2022-03-25T12:27:39.183</CreatedDate>
  <StartDate xsi:nil="true" />
  <FinishedDate xsi:nil="true" />
  <Status>Created</Status>
  <StatusDetails>Approval process not yet started</StatusDetails>
  <StatusDescription>In progress</StatusDescription>
  <Link href="..."
    rel="self"
    type="application/vnd.cbrain.casefile+xml" />
  <Link href="..."
    rel="up"
    type="application/vnd.cbrain.casefile+xml" />
  <Link href="..."
    rel="http://cbrain.com/casefile/rel/pdf-content"
    type="application/pdf" />
  <Link href="..."
    rel="http://cbrain.com/casefile/rel/start-approval"
    type="application/vnd.cbrain.casefile+xml" />
</Approval>
```

Table 64. Expected link relations

Rel	Description
self	
up	Link to parent matter.
http://cbrain.com/casefile/rel/start-approval	Link to approval process starter resource.
http://cbrain.com/casefile/rel/pdf-content	Link to approval process description in PDF format.

Create new approval

An approval process may be created using the Post Once Exactly pattern (see [Data manipulation](#)). Follow the link named <http://cbrain.com/casefile/rel/create-approval> from a specific matter in order to add the approval process to that matter.

Table 65. Input parameters for creating an approval process

Name	Description
ApprovalProcessTemplate_Id	int Internal database identifier for approval process template.
ApprovalProcessTemplate_ExternalId	int External identifier for approval process template.
Deadline	DateTime Deadline for approval.
ApprovalTypeId	string External enumeration ID for approval type.
Urgent	bool Urgent indication for approval process.
Steps	Step List of steps in approval process.
Steps.Approvers	List of PartyReference List of approvers for approval step.
Steps.Deadline	DateTime Deadline for approval step.
Steps.SubscribedToActions	string Comma-separated list of subscriptions for the assigned approvers. Possible subscription values are: - NoExtraNotifications - WhenRevisitingAfterReturning - WhenStepChanges - WhenFinallyApproved - WhenCommentIsAdded
AutoStart	bool Pass true to start approval process immediately after creation.

Approval process creation example

```
POST approval-creation-url HTTP/1.1
Content-Length: 0

> HTTP/1.1 201 Created
> Location: approval-process-poe-url

POST approval-creation-poe-url HTTP/1.1
Accept: application/vnd.cbain.casefile+xml
Content-Type: application/x-www-form-urlencoded

Steps%3a0.Approvers%3a0.PartyNo=66&
Steps%3a0.Approvers%3a0.PickFirstMatch=false&
Steps%3a0.Approvers%3a1.PartyNo=200001024&
Steps%3a0.Approvers%3a1.PickFirstMatch=false&
Steps%3a0.Deadline=2019-02-02T00%3a00%3a00&
Steps%3a0.SubscribedToActions=WhenCommentIsAdded+%2cWhenRevisitingAfterReturning&
Steps%3a1.Approvers%3a0.PartyNo=200001003&
Steps%3a1.Approvers%3a0.PickFirstMatch=false&
Steps%3a1.Deadline=2018-03-03T00%3a00%3a00&
AutoStart=false&
ApprovalTypeId=Approval-process-type&
Urgent=true
```

Modify approval process

Approvals cannot, at the time of writing, be modified.

Start approval process

An approval process can be started by looking up the link relation `http://cbain.com/casefile/rel/start-approval` in the approval's link collection and then issue an empty POST to that resource.

Table 66. Start approval process response properties

Name	Description
Success	bool Success indicator.
Warnings	string Warning message in case of failure.

Example approval process start request

```
POST approval-process-start-url HTTP/1.1
Accept: application/vnd.cbain.casefile+xml
Content-Length: 0
```


Example approval process start response

```
HTTP/1.1 200 OK
Content-Type: application/vnd.cbrain.casefile+xml

<ApprovalProcessStartedResult xmlns="http://cbrain.com/casefile/schema/">
  <Success>true</Success>
</ApprovalProcessStartedResult>
```

Enumerations

Enumeration type resource

This resource represents an enumeration type definition and all of it's possible enumeration items.

Location

Properties

Table 67. Enumeration type properties

Name	Description
Title	string Enumeration type title.
Id	string Enumeration type ID.
Items	List of enumeration items named Item (see below) All enumeration items in enumeration. The enumeration type represents the root of a tree and items represent the nodes under the root.

The XML root element is `EnumerationType`.

Table 68. Enumeration item properties

Name	Description
Title	string The title of the enumeration item.
Path	string Represents the enumeration items position in the tree. If the top level item "ItemTitle" contains the child "ItemTitleChild", then the Path of "ItemTitleChild" will be "ItemTitle/ItemTitleChild".
Code	string An Id which can be used to find the item in question in other contexts.
Abbreviation	string An Id that was historically used the same way as <code>Code</code> . Often the same as <code>Code</code> or empty.
Items	List of enumeration item named Item A list of child items. This list represents the items that are the children of the current enumeration item.

Enumeration type example

```
<EnumerationType xmlns="http://cbrain.com/casefile/schema/">
  <Title>TypeTitle</Title>
  <Id>TypeId</Id>
  <Items>
    <Item>
      <Title>ItemTitle</Title>
      <Path>ItemTitle</Path>
      <Code>ItemId</Code>
      <Abbreviation>ItemId</Abbreviation>
      <Items>
        <Item>
          <Title>ItemTitleChild</Title>
          <Path>ItemTitle/ItemTitleChild</Path>
          <Code>ItemIdChild</Code>
          <Abbreviation>ItemIdChild</Abbreviation>
          <Items/>
        </Item>
      </Items>
    </Item>
  </Items>
</EnumerationType>
```

Security group

Security group resource

This resource represents a security group in F2. These groups can be given access to F2 resources such as cases and matters, granting access to these resources to each of the group members. As such, they function to manage the security clearances for groups of users.

Location

Security groups can be located using their ID, their synchronization key or their party number.

Lookup by ID

A security group can be accessed using the link relation `http://cbrain.com/casefile/rel/securitygroup-by-id` to find the URL template for lookups on ID.

A `GET` on the generated URL resource will redirect to the appropriate security group resource.

Example link from service document

```
{
  "http://cbrain.com/casefile/rel/securitygroup-by-id":
  {
    href: "http://.../{id}",
    title: "Get security group by id"
  }
}
```

Lookup by synchronization key

A security group can be accessed using the link relation `http://cbrain.com/casefile/rel/securitygroup-by-synch-key` to find the URL template for lookups on synchronization keys.

A `GET` on the generated URL resource will redirect to the appropriate security group resource.

Example link from service document

```
{
  "http://cbrain.com/casefile/rel/securitygroup-by-synch-key":
  {
    href: "http://.../{synchKey}",
    title: "Get security group by synchronization key"
  }
}
```

Lookup by party number

A **GET** on the generated URL resource will redirect to the appropriate security group resource.

Example link from service document

```
{
  "http://cbrain.com/casefile/rel/securitygroup-by-party-number":
  {
    href: "http://.../{partyNumber}",
    title: "Get security group by party number"
  }
}
```

Properties

Table 69. Security group properties

Name	Description
AuthorityTitle	string (read-only) The name of the authority the security group belongs under
GroupName	string (255 characters) The name of the security group
SynchronizationKey	string (255 characters) The synchronization key
Id	int (read-only) Internal security group identifier.
Party Number	int (read-only) The party number of the security group.
IsActive	bool (read-only) Indicates whether or not the security group is active.
PartyItems	List<PartyItem> The users currently in the security group.

Security group example

```
<SecurityGroupItem
  xmlns:xsd="http://www.w3.org/2001/XMLSchema"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns="http://cbrain.com/casefile/schema/">
  <AuthorityTitle>Departementet</AuthorityTitle>
  <GroupName>SG-SyncKey</GroupName>
  <SynchronizationKey>C8163116-7031-47FA-9D54-0E6880827B4D</SynchronizationKey>
  <IsActive>true</IsActive>
  <Id>634576</Id>
  <PartyNumber>230</PartyNumber>
  <Link href="..." rel="self" title="SG-SyncKey" />
  <Link href="..." rel="http://cbrain.com/casefile/rel/securitygroup-add-member" title="Add
member" />
  <Link href="..." rel="http://cbrain.com/casefile/rel/securitygroup-remove-member"
title="Remove member" />
  <Link href="..." rel="http://cbrain.com/casefile/rel/securitygroup-activate" title="Activate
security group member" />
  <Link href="..." rel="http://cbrain.com/casefile/rel/securitygroup-deactivate"
title="Deactivate security group" />
  <PartyItems>
    <PartyItem>
      <Name>CaseWorkerPartyWithNameInBothMinisteries</Name>
      <EMail />
      <Link href="..." rel="self" title="CaseWorkerPartyWithNameInBothMinisteries" />
      <Type>User</Type>
      <Id>634581</Id>
      <PartyNumber>235</PartyNumber>
      <CPRCVR />
      <CVR_P />
    </PartyItem>
    <PartyItem>
      <Name>DobbeltAnsatt</Name>
      <EMail>DobbeltAnsatt@cbrain.dk</EMail>
      <Link href="..." rel="self" title="DobbeltAnsatt" />
      <Type>User</Type>
      <Id>634574</Id>
      <PartyNumber>228</PartyNumber>
      <CPRCVR />
      <CVR_P />
    </PartyItem>
  </PartyItems>
</SecurityGroupItem>
```

Table 70. Expected link relations

Rel	Description
self	Link to self
http://cbrain.com/casefile/rel/securitygroup-add-member	Adds member to security group. See Add member to security group .
http://cbrain.com/casefile/rel/securitygroup-remove-member	Removes member from group See Remove member from security group .
http://cbrain.com/casefile/rel/securitygroup-activate	Sets the status of an inactive security group to active (accepts POST requests).
http://cbrain.com/casefile/rel/securitygroup-deactivate	Sets the status of an active security group to inactive (accepts POST requests).

Modify security group

Some properties of a security group may be modified by issuing a PATCH request targeted at the security group resource URL. Only the GroupName, SynchronizationKey and PartyItems list can be changed for a security group using a patch request. For the Name and Synchronizationkey, string values are accepted. For PartyItems, an array of PartyReferences is needed to change the current members of a security group to the new list.

Example PATCH request with headers

```
PATCH /securitygroup-url HTTP/1.1
Accept: application/vnd.cbrain.casefile+xml
Content-Type: application/json-patch

[
  {"value":"Changed sync key","op":"replace","path":"/SynchronizationKey"},
  {"value":"Changed title","op":"replace","path":"/GroupName"},
  {"value":[{"PartyNo":26},{"PartyNo":28}], "op":"replace", "path":"/PartyItems"},
]
```

Create new security group

A security group may be created using the Post Once Exactly pattern (see [Data manipulation](#)). Form a POST request on the link <http://cbrain.com/casefile/rel/securitygroup-create> from the service index to locate the POE resource for creating a security group.

Table 71. Input parameters for creating a security group

Name	Description
AuthorityId	int (required) The Id of the authority the security group will belong to.
Name	string (required, 255 characters) The name of the new security group.
SynchronizationKey	string The synchronization key of the security group

Add member to security group

Adding a member to a security group may be done using the Post Once Exactly pattern (see [Data manipulation](#)). Form a POST request on the link `http://cbrain.com/casefile/rel/securitygroup-add-member` from the service index to locate the POE resource for adding a member to a security group. Despite the POE pattern, adding a the same member to a security group more than once has no effect.

Table 72. Input parameters for adding members to a security group

Name	Description
Party	PartyReference (required) The user to be added to the security group

Remove member from security group

Removing a member from a security group may be done by using a POST request on the link received from the link relation `http://cbrain.com/casefile/rel/securitygroup-remove-member`. Removing a security group member from a group multiple times has no effect.

Table 73. Input parameters for removing members from a security group

Name	Description
Party	PartyReference (required) The user to be removed from the security group

Task guides
= Task guide state

Task guide state resource

This resource represents the current state of a task guide — that is, the visibility and completeness of tasks and processes as well as the values of fields and variables used by the task guide. The data is only intended for reading.

Location

The state of a task guide can be found by following the link relation <http://cbrain.com/casefile/rel/taskguide-state> from the resource representing the related case.

Properties

Table 74. Task guide state properties

Name	Description
TaskGuideKey	string Task guide key (file name of task guide file).
Processes	List List of processes.
Processes.Process.ItemKey	string Item key identifying a single process.
Processes.Process.Visible	bool Visibility of single process.
Processes/Process/Tasks	List List of tasks in process.
Processes.Process.Tasks.Task.ItemKey	string Item key identifying a single task.
Processes.Process.Tasks.Task.Visible	bool Visibility of single task.
Processes.Process.Tasks.Task.Complete	bool Completeness of single task.
Fields	List List of fields and task guide variables.
Fields.Field.Name	string Name of field.
Fields/Field/IsList	bool Indicates if the field is a list or a simple value. If true then the element <code><List></code> contains the rows of the list, otherwise the element <code><Value></code> contains the simple value.
Fields/Field/Value	mixed Simple value of field (booleans, strings, dates and numbers as well as sets).
Fields/Field/IsWritable	bool Indicates whether or not the field is writable by the current user.
Fields/Field/List	List List of rows.
Fields.Field.List.Row	List List of fields included in the row.

Table 74. Task guide state properties

Name	Description
	A single named field and its value. Row fields have the same (recursive) structure as a single Fields/Field element.

The XML root element is `<TaskGuideState>`.

Task guide state example

```
<TaskGuideState xmlns="http://cbrain.com/casefile/schema/">
  <TaskGuideKey>DemoTaskGuide</TaskGuideKey>
  <Processes>
    <Process>
      <ItemKey>P1</ItemKey>
      <Visible>true</Visible>
      <Tasks>
        <Task>
          <ItemKey>P1_T1</ItemKey>
          <Visible>false</Visible>
          <Complete>false</Complete>
        </Task>
        <Task>
          <ItemKey>P1_T2</ItemKey>
          <Visible>true</Visible>
          <Complete>false</Complete>
        </Task>
      </Tasks>
    </Process>
    ...
  </Processes>
  <Fields>
    <Field>
      <Name>DemoName</Name>
      <IsList>false</IsList>
      <Value xsi:type="xsd:string">Hari Seldon</Value>
      <List xsi:nil="true" />
    </Field>
    <Field>
      <Name>DemoAttribute</Name>
      <IsList>false</IsList>
      <Value xsi:type="Set" Enumeration="AttributeSet">
        <Elements>
          <Element>A1</Element>
          <Element>A2</Element>
        </Elements>
      </Value>
      <List xsi:nil="true" />
    </Field>
    <Field>
      <Name>DemoList</Name>
      <IsList>true</IsList>
      <Value xsi:nil="true" />
      <List>
        <Row>
          <Field>
            <Name>Name</Name>
            <IsList>false</IsList>
            <Value xsi:type="xsd:string">Gaal Dornick</Value>
            <List xsi:nil="true" />
          </Field>
        </Row>
        <Row>
          <Field>
            <Name>Name</Name>
            <IsList>false</IsList>
            <Value xsi:type="xsd:string">Salvor Hardin</Value>
            <List xsi:nil="true" />
          </Field>
        </Row>
      </List>
    </Field>
  </Fields>
</TaskGuideState>
```

```
</Row>
  </List>
</Field>
</Fields>
</TaskGuideState>
```

Execute single task

The actual semantics of executing a task depends on the type of task involved—it simply corresponds to the action that would happen when activating the primary task button or a specific action button in the desktop client UI.

Executing a task may result in various related actions or scripts on the task being executed too—precisely in the same way as it works on the desktop client.

A single task can be executed by looking up the "execute task handler" linked to a case by the link relation <http://cbrain.com/casefile/rel/taskguidetasks-execute>. Follow this link and issue an HTTP POST to it with the parameters shown below.

The "execute task handler" is a Post Once Exactly resource and as such it is necessary to POST twice to this resource as described in [Data manipulation](#). The response does although behave a little bit different as no resources are created—thus the HTTP response status code is 200 where after the POE status must be decoded from the `<Status>` element of the response.

Table 75. Inputs for executing a task

Name	Description
ItemKey	string The item key of a task to execute.
ActionName	string The name of an action to execute. For use with <code><Action></code> tasks containing multiple actions.

Execute task example

```
POST /execute-task-url HTTP/1.1
Content-Length: 0

> HTTP/1.1 201 Created
> Content-Type: text/html
> Location: /execute-task-poe-url
> ... HTML content for browser based API exploration ...

POST /execute-task-poe-url HTTP/1.1
Accept: application/vnd.cbrain.casefile+xml
Content-Type: application/x-www-form-urlencoded

ItemKey=T_SetName

> HTTP/1.1 200 OK
> Content-Type: application/vnd.cbrain.casefile+xml
> ... XML response ...
```

Table 76. Execute task response properties

Name	Description
Status	string Status of operation. See table below for possible values.
Message	string Message to show end-user. The message may be empty if there is nothing interesting to present.
Warnings	List of strings named Warning List of warnings to show end-user. The list may be empty if there is nothing interesting to present.

Table 77. Possible values for Status

Name	Description
Ok	Task was executed without issues.
RequireMoreInfo	Task cannot be completed without supplying more information (this is currently not supported by F2-REST).
PartialOk	Task execution involved data from a list, but the task was only executed for some of the elements in the list.
AlreadyCompleted	Task execution was completed by an earlier post-once-exactly operation.

Execute task response example

```
<ExecuteTaskResult xmlns="http://cbrain.com/casefile/schema/">
  <Status>Ok</Status>
  <Message>Some message</Message>
  <Warnings>
    <Warning>Some warning</Warning>
  </Warnings>
</ExecuteTaskResult>
```

Allowed methods

GET

Get a representation of the handler. This will return an HTML page describing the resource for use with browser-based exploration of the API.

POST

Execute task using the Post Once Exactly pattern.

Complete single task

Completing a task via F2-REST corresponds to checking off the "Complete" checkbox beside a task in the desktop client. Completing a task may result in various related actions or scripts on the task being executed too — precisely in the same way as it works on the desktop client.

A single task can be completed by looking up the "complete task handler" linked to a case by the link relation <http://cbrain.com/casefile/rel/taskguidetasks-complete>. Follow this link and issue an HTTP POST to it with the parameters shown below.

The "complete task handler" is a Post Once Exactly resource and as such it is necessary to POST twice to this resource as described in [Data manipulation](#). The response does although behave a little bit different as no resources are created — thus the HTTP response status code is 200 where after the POE status must be decoded from the `<Status>` element of the response.

Table 78. Inputs for completing a task

Name	Description
ItemKey	string The item key of a task to complete.
Complete	bool Specifies whether to complete the task or uncomplete it.

Complete task example

```
POST /complete-task-url HTTP/1.1
Content-Length: 0

> HTTP/1.1 201 Created
> Content-Type: text/html
> Location: /complete-task-poe-url
> ... HTML content for browser based API exploration ...

POST /complete-task-poe-url HTTP/1.1
Accept: application/vnd.cbain.casefile+xml
Content-Type: application/x-www-form-urlencoded

ItemKey=T_SetName&Complete=true

> HTTP/1.1 200 OK
> Content-Type: application/vnd.cbain.casefile+xml
> ... XML response ...
```

Table 79. Complete task response properties

Name	Description
Status	string Status of operation. See table below for possible values.
Message	string Message to show end-user. The message may be empty if there is nothing interesting to present.

Table 80. Possible values for `Status`

Name	Description
Ok	Task was completed without issues.
AlreadyCompleted	Task was completed by an earlier post-once-exactly operation.

Complete task response example

```
<CompleteTaskResult xmlns="http://cbain.com/casefile/schema/">
  <Status>Ok</Status>
  <Message>Some message</Message>
</CompleteTaskResult>
```

Allowed methods

GET

Get a representation of the handler. This will return an HTML page describing the resource for use with browser-based exploration of the API.

POST

Complete (or uncomplete) task using the Post Once Exactly pattern.

Execute OnSave actions for single task

Executing OnSave actions on a task via F2-REST corresponds to pressing "Edit" and then saving the content of that task. Executing the OnSave actions may result in various related actions or scripts on the task being executed too — precisely in the same way as it works on the desktop client.

A single task can be invoked by looking up the "save task handler" linked to a case by the link relation <http://cbrain.com/casefile/rel/taskguidetasks-save>. Follow this link and issue an HTTP POST to it with the parameters shown below.

The "save task handler" is a Post Once Exactly resource and as such it is necessary to POST twice to this resource as described in [Data manipulation](#). The response does although behave a little bit different as no resources are created — thus the HTTP response status code is 200 where after the POE status must be decoded from the `<Status>` element of the response.

Table 81. Inputs for executing OnSave actions for a task

Name	Description
ItemKey	string The item key of a task to invoke.

Save task example

```
POST /save-task-url HTTP/1.1
Content-Length: 0

> HTTP/1.1 201 Created
> Content-Type: text/html
> Location: /save-task-poe-url
> ... HTML content for browser based API exploration ...

POST /save-task-poe-url HTTP/1.1
Accept: application/vnd.cbrain.casefile+xml
Content-Type: application/x-www-form-urlencoded

ItemKey=T_EditDetails

> HTTP/1.1 200 OK
> Content-Type: application/vnd.cbrain.casefile+xml
> ... XML response ...
```

Table 82. Save task response properties

Name	Description
Status	string Status of operation. See table below for possible values.
Message	string Message to show end-user. The message may be empty if there is nothing interesting to present.

Table 83. Possible values for Status

Name	Description
Ok	Task was saved without issues.
AlreadySaved	Task was saved by an earlier post-once-exactly operation.

Complete save response example

```
<SavekResult xmlns="http://cbrain.com/casefile/schema/">
  <Status>Ok</Status>
  <Message>Some message</Message>
</SaveTaskResult>
```

Allowed methods

GET

Get a representation of the handler. This will return an HTML page describing the resource for use with browser-based exploration of the API.

POST

Execute OnSave actions for task using the Post Once Exactly pattern.

Execute task guide script

A task guide script can be executed by looking up the "execute script handler" linked to a case by the link relation `http://cbrain.com/casefile/rel/taskguidescripts-execute`. Follow this link and issue a `POST` to it with the parameters shown below.

The `execute script handler` is a Post Once Exactly resource and as such it is necessary to `POST` twice to this resource as described in [Data manipulation](#).

Table 84. Inputs for executing a script

Name	Description
ScriptName	string The name of the script.

Execute script example

```
POST /execute-script-url HTTP/1.1
Content-Length: 0

> HTTP/1.1 201 Created
> Content-Type: text/html
> Location: /execute-script-poe-url
> ... HTML content for browser based API exploration ...

POST /execute-script-poe-url HTTP/1.1
Accept: application/vnd.cbrian.casefile+xml
Content-Type: application/x-www-form-urlencoded

ScriptName=NameOfScript

> HTTP/1.1 200 OK
```

Result structure

Due to the nature of how task guide scripts works. This operation is not expected to return an error status code, in case an error occurred in one of the executed scripts. This is because the operation could be a partial success. The result structure needs to be inspected in order to determine which scripts has failed.

The result structure is shown in the table [Properties of ExecuteTaskGuideScriptResult](#).

Table 85. Properties of `ExecuteTaskGuideScriptResult`

Name	Description
Message	string Status of operation. Possible values are: - This operation has already completed. - Executed taskguide script.
Results	List of ScriptResults named ScriptResult Recursive list of results from running the script. See <code>ScriptResult</code> below. If a task in the script contains a reference to another script which is run successfully, then <code>Results.Count > 1</code> else <code>Results.Count == 1</code> .

Table 86. Properties of ScriptResult

Name	Description
CaseId	Long NodeId of the case which the script is run on.
ScriptName	String Name of the script.
TaskResults	List of TaskResult and ActionTaskResult List of results of each task run. See TaskResult below. If action type is ExecuteAction then the type will be ActionTaskResult which extends TaskResult .
IsOk	Bool Is used to indicate if the entire script has failed. Normal errors are marked in a specific TaskResult or ActionTaskResult . Some errors, such as exceeding the maximum number of derived scripts, occur before a specific task is executed. As such the error is indicated here.
Message	String If the property IsOk is false this property describes what the error was.

Table 87. Properties of TaskResult

Name	Description
TaskType	String Specifies the type of the task. Possible values are: - CompleteTask — a complete or uncomplete task. - ExecuteTask — execute a task. - ExecuteAction — execute an action on a task. - DeleteReplyLink — Delete reply link on selfservice task. - Exception — representing a failed task.
TaskKey	String Unique key of the task
IgnoreErrors	Bool true if the task was run with the IgnoreErrors flag
ActionFailed	Bool true if the task failed.
IsOk	Bool true if the task succeded without errors or if IgnoreErrors is true .
Errors	List of String List of error messages if the task or related tasks failed.

Table 88. Properties of ActionTaskResult extending TaskResult

Name	Description
ActionKey	String Unique key of the action.

Execute taskguide script response example

```
<ExecuteTaskGuideScriptResult xmlns:xsd="http://www.w3.org/2001/XMLSchema"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns="http://cbrain.com/casefile/schema/">
  <Message>Executed TaskGuideScript.</Message>
  <Results>
    <ScriptResult>
      <CaseId>12345</CaseId>
      <ScriptName>ExampleScript</ScriptName>
      <IsOk>true</IsOk>
      <TaskResults>
        <TaskResult>
          <TaskType>ExecuteTask</TaskType>
          <TaskKey>ExampleTask</TaskKey>
          <IsOk>true</IsOk>
          <IgnoreErrors>false</IgnoreErrors>
          <ActionFailed>false</ActionFailed>
          <Errors />
        </TaskResult>
        <ActionTaskResult>
          <TaskType>ExecuteAction</TaskType>
          <TaskKey>ExampleTask2</TaskKey>
          <IsOk>true</IsOk>
          <IgnoreErrors>false</IgnoreErrors>
          <ActionFailed>false</ActionFailed>
          <Errors />
          <ActionKey>ExampleAction</ActionKey>
        </ActionTaskResult>
      </TaskResults>
    </ScriptResult>
    <ScriptResult>
      <CaseId>54321</CaseId>
      <ScriptName>ExampleScript2</ScriptName>
      <IsOk>true</IsOk>
      <Message>Exceeded the maximum number of derived scripts.</Message>
      <TaskResults/>
    </ScriptResult>
  </Results>
</ExecuteTaskGuideScriptResult>
```

Allowed methods

GET

Get a representation of the handler. This will return an HTML page describing the resource for use with browser-based exploration of the API.

POST

Execute task guide script using the Post Once Exactly pattern.

```
Extensions
= Case guide data
```

A core concept of F2 case guides, is to be able to store a mixed set of fields (named values) in the case guide. This case data can be manipulated through F2-REST where it is referred to as

"extension records" — where each record is an XML blob storing all of the case guide data related to the case.

In addition to the global case guide data, it is also possible to store named lists of records where each record represents a single row in the list. These lists corresponds to the lists in case guide data.

The system allows multiple named extension records for each case. In practice though, only the records named `CaseInfo` is used — these represent the case guide data.

Record data, rows and fields

The data stored in a record, consists of a list of fields — each of which contains a field name and a field value.

A row in a list is the same as a record with an additional row identifier.

In the following sections we use the data types `Record`, `Row` and `Field`:

Table 89. Record properties

Name	Description
Fields	List of Field List of all fields in the record.

Table 90. Row properties

Name	Description
Id	string Row identifier (GUID). All rows have unique identifiers irrespective of which lists they are contained in. Use the identifier when updating, deleting or replacing rows.
Fields	List of Field List of all fields in the row.

Table 91. Field properties

Name	Description
@name	string Name of field (XML attribute).
Value	mixed Value of field.
Value@xsi:type	string Value type.

Field values

Field values are encoded using XML schema data types as documented in [XMLTypes] and [XMLSchema].

Table 92. Used namespaces

Alias	Namespace
xsi	http://www.w3.org/2001/XMLSchema-instance
xsd	http://www.w3.org/2001/XMLSchema

Table 93. Supported types and XML encoding

Name	Example
bool	<code><Value xsi:type="xsd:boolean">false</Value></code>
int	<code><Value xsi:type="xsd:int">10</Value></code>
decimal	<code><Value xsi:type="xsd:decimal">35.50</Value></code>
double	<code><Value xsi:type="xsd:double">20.12</Value></code>
string	<code><Value xsi:type="xsd:string">Hej mor</Value></code>
datetime	<code><Value xsi:type="xsd:dateTime">2013-08-27T10:20:30+02:00</Value></code>
set	<pre><Value xsi:type="Set" Enumeration="ATest"> <Elements> <Element>A</Element> <Element>B</Element> <Element>C</Element> </Elements> </Value></pre>

Extension record

This resource represents a single F2 extension record with links to the lists related to it.

Location

The link relation `http://cbrain.com/casefile/rel/extension-record-by-name`, found in the links from a case, identifies a URL template for looking up extension records on that case by name. The template uses the template parameter `recordName` for the record name. Use the record name `CaseInfo` for the standard case guide data.

A GET on the generated URL will return a redirect to the corresponding extension record resource.

Example link from case

```
<Link href="http://record-url/{recordName}"
      rel="http://cbrain.com/casefile/rel/extension-record-by-name"
      type="application/vnd.cbrain.casefile+xml" />
```

Properties

Table 94. Extension record properties

Name	Description
Id	int Record identifier.
Name	string Record name.
LastUpdated	DateTime Last updated time.
Fields	List of Field List of fields stored in the record.
Lists	List of List List of lists related to record.
Lists.List.Name	string Name of list.
Lists.List.Link	List of Link Link to related resource. Follow <code>self</code> link to get list content.
Link	List of Link One or more links to other related resources.

Extension record example

```
<Record xmlns="http://cbrain.com/casefile/schema/">
  <Id>13016</Id>
  <Name>CaseInfo</Name>
  <Fields>
    <Field name="Name">
      <Value xsi:type="xsd:string">Ofelia</Value>
    </Field>
    <Field name="EMail">
      <Value xsi:type="xsd:string">example@cbrain.dk</Value>
    </Field>
    <Field name="Address1Field">
      <Value xsi:nil="true" />
    </Field>
  </Fields>
  <Link href="..."
        rel="self"
        type="application/vnd.cbrain.casefile+xml"
        title="Record" />
  <Lists>
    <List Name="Books">
      <Link href="..." rel="self" type="application/vnd.cbrain.casefile+xml" />
    </List>
    <List Name="Tickets">
      <Link href="..." rel="self" type="application/vnd.cbrain.casefile+xml" />
    </List>
  </Lists>
</Record>
```

Table 95. Expected link relations

Rel	Description
self	
http://cbrain.com/casefile/rel/update	Link to resource for updating a single record.
http://cbrain.com/casefile/rel/update-lists	Link to resource for updating lists in a record.

Create new extension record

An extension record may be created by posting to an appropriate resource. Follow the link named `http://cbrain.com/casefile/rel/create-record` from a case to locate the resource. The Post Once Exactly pattern (see [Data manipulation](#)) is not required here as it impossible to create more than one named record for each case.

The input format must be XML to be able to encode all the different value types.

Table 96. Input parameters for creating an extension record

Name	Description
Name	string Name of record. Use <code>CaseInfo</code> for case guide data.
Record	Record Record containing the initial fields for the extension record.

Extension record creation example

```
POST create-record-url HTTP/1.1
Accept: application/vnd.cbrain.casefile+xml
Content-Type: application/xml

<NewRecordArgs
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns:xsd="http://www.w3.org/2001/XMLSchema"
  xmlns="http://cbrain.com/casefile/schema/">
  <Record>
    <Fields>
      <Field name="A">
        <Value xsi:type="xsd:int">999</Value>
      </Field>
    </Fields>
  </Record>
  <Name>CaseInfo</Name>
</NewRecordArgs>
```

Update extension record

The link relation `http://cbrain.com/casefile/rel/update`, found on a record, leads to a resource for updating the record. Either `POST` or `PUT` new field values to this resource — a `POST` will merge the new fields into the existing record whereas a `PUT` will clear the existing record first before adding the fields passed to the resource.

The input format must be XML to be able to encode all the different value types.

Table 97. Input parameters for updating an extension record

Name	Description
Record	Record Record containing the fields for the extension record.

Extension record update example

```
POST update-record-url HTTP/1.1
Accept: application/vnd.cbrain.casefile+xml
Content-Type: application/xml

<UpdateRecordArgs
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns:xsd="http://www.w3.org/2001/XMLSchema"
  xmlns="http://cbrain.com/casefile/schema/">
  <Record>
    <Fields>
      <Field name="A">
        <Value xsi:type="xsd:string">Hello world</Value>
      </Field>
      <Field name="K">
        <Value xsi:type="xsd:double">100.234</Value>
      </Field>
    </Fields>
  </Record>
</UpdateRecordArgs>
```

Delete extension record

An extension record may be deleted simply by issuing a `DELETE` request at the extension record resource.

Extension list

This resource represents all the rows of a single list.

Location

Lists are referenced from an extension record in its `<Lists>` element - from which it is necessary to follow the `self` link to get the actual list content. It looks like this:

Extension record example with links to lists

```
<Record xmlns="http://cbrain.com/casefile/schema/">
  <Id>13016</Id>
  <Name>CaseInfo</Name>
  <Fields>
    ...
  </Fields>
  <Lists>
    <List Name="Books">
      <Link href="..." rel="self" type="application/vnd.cbrain.casefile+xml" />
    </List>
    <List Name="Tickets">
      <Link href="..." rel="self" type="application/vnd.cbrain.casefile+xml" />
    </List>
  </Lists>
</Record>
```

Properties

Table 98. List properties

Name	Description
Name	string Name of list.
Rows	List of Row All rows in the list.

The XML root element is `<List>`.

List example

```
<List xmlns="http://cbrain.com/casefile/schema/">
  <Name>Books</Name>
  <Rows>
    <Row>
      <Id>cc1d9d03-f042-424d-b1cc-7adcdf0da59d</Id>
      <Fields>
        <Field name="Title">
          <Value xsi:type="xsd:string">Rice and fall of pi</Value>
        </Field>
      </Fields>
    </Row>
    <Row>
      <Id>ddd02eea-31fd-4fc5-ab69-ad1fc12202de</Id>
      <Fields>
        <Field name="Title">
          <Value xsi:type="xsd:string">A history of real numbers</Value>
        </Field>
      </Fields>
    </Row>
  </Rows>
  <Link href="..."
    rel="up"
    type="application/vnd.cbrain.casefile+xml"
    title="Parent" />
</List>
```

Update list

Follow the link relation `http://cbrain.com/casefile/rel/update-lists` from an extension record to find a resource for updating the lists on the record. Then use the Post Once Exactly pattern (see [Data manipulation](#)) to issue a set of operations on the lists to append, delete, update or replace rows.

The input format must be XML to be able to encode all the different value types.

Table 99. Update list inputs

Name	Description
Actions	List of Append, Update, Replace, Delete or DeleteList List of operations on lists.

The XML root element is `UpdateListArgs`.

Table 100. Append properties

Name	Description
ListName	string Name of list to append row to. If the list does not exist already it will be created automatically. The new row is appended to the end of the list.
Row	Row Row to append. New rows MUST be assigned a globally unique row identifier before appending it.

Table 101. Update properties

Name	Description
Row	Row Row to update. Use the row identifier to specify which row to update. Updating a row only touches the fields referenced in the row input. Since row identifiers are globally unique, it is not necessary to supply a list name.

Table 102. Replace properties

Name	Description
Row	Row Row to replace. Use the row identifier to specify which row to replace. Replacing a row removes all existing fields in the row before adding the fields from the input. Since row identifiers are globally unique, it is not necessary to supply a list name.

Table 103. Delete properties

Name	Description
Id	string Identifies which row to delete. Since row identifiers are globally unique, it is not necessary to supply a list name.

Table 104. DeleteList properties

Name	Description
ListName	string Identifies a list to delete completely.

Table 105. Update list response properties

Name	Description
Ok	bool Update status.
Message	string Optional error message.

The XML root element is `UpdateListResult`.

List update example

```
POST update-list-url HTTP/1.1
Content-Length: 0
```

```
> HTTP/1.1 201 Created
> Location: update-list-url-poe
> Content-Length: 0
```

```
POST update-list-url-poe HTTP/1.1
Accept: application/vnd.cbrain.casefile+xml
Content-Type: application/xml
```

```
<UpdateListArgs xmlns="http://cbrain.com/casefile/schema/">
  <Actions>
    <Update>
      <Row>
        <Fields>
          <Field name="Name">
            <Value xsi:type="xsd:string">Frank</Value>
          </Field>
        </Fields>
        <Id>c97445e3-2e72-4a60-9ed0-cd82f212017a</Id>
      </Row>
    </Update>
    <Delete>
      <Id>0d24d36e-6eeb-4b24-91b8-3b4d5551d4b5</Id>
    </Delete>
    <Append>
      <ListName>Books</ListName>
      <Row>
        <Fields>
          <Field name="Name">
            <Value xsi:type="xsd:string">Between numbers</Value>
          </Field>
        </Fields>
        <Id>58eff98a-cb54-421a-99e9-1ab4dd88a028</Id>
      </Row>
    </Append>
  </Actions>
</UpdateListArgs>

> HTTP/1.1 200 OK
> Content-Type: application/vnd.cbrain.casefile+xml
>
> <UpdateListResult xmlns="http://cbrain.com/casefile/schema/">
>   <Ok>true</Ok>
>   <Message>Success</Message>
> </UpdateListResult>
```

Custom XML data

One way to extend F2, is to store custom XML data associated with **cases** and **matters**. The XML schema for the actual data can be chosen freely - F2 puts no constraint on it. Further more, there is room for multiple named XML data elements for each case or matter.

Custom XML data is used by F2 to store data from extension fields embedded in the case and matter windows.

Custom XML data resource

Location

Custom XML data located by following the link relation `http://cbrain.com/casefile/rel/extension-data` from a matter or case. This leads to a resource containing the custom XML data.

Properties

The custom XML data resource contains the following properties:

Table 106. Custom XML data properties

Name	Description
Extension	Extension Zero or more named custom XML data elements.
Extension.ProcessName	string Name of extension.
Extension.Data	mixed Custom XML data.
Link	List of Link One or more links to other related resources.

The XML root element is `Extensions`.

Custom XML data example

```
<Extensions xmlns="http://cbrain.com/casefile/schema/">
  <Extension>
    <ProcessName>Abc</ProcessName>
    <Data><Size xmlns="http://123">10</Size></Data>
  </Extension>
  <Extension>
    <ProcessName>Xyz</ProcessName>
    <Data><Position xmlns="http://123">20,10</Position></Data>
  </Extension>
  <Link href="..."
        rel="self"
        type="application/vnd.cbrain.casefile+xml"
        title="Extensions" />
  <Link href="..."
        rel="up"
        type="application/vnd.cbrain.casefile+xml"
        title="Parent" />
</Extensions>
```

Add or update custom XML data

Follow the link named <http://cbrain.com/casefile/rel/create-extension-data> to locate a handler for creating custom XML data. Then issue an **POST** to the resource with the input properties described below.

Multiple POST requests with the same data will only result in the creation of one data set for the given process name - the last request overwrites earlier data.

Table 107. Input parameters for creating custom XML data

Name	Description
ProcessName	string (255 characters) Process name associated with the custom XML data.
Data	string Actual raw XML payload.

Add custom XML data example

```
POST extension-data-create-url HTTP/1.1
Content-Type: application/x-www-form-urlencoded
Content-Length: 108

ProcessName=Test&Data=%3cTest+xmlns%3d%22http%3a%2f%2f123%22%3e%3cName%3eAnders%3c%2fName%3e%3c%2fTest%3e
```

Updating matter & case extension fields

It is possible to update individual extension fields on matters and cases. Matters contain the link named <http://cbrain.com/casefile/rel/update-matter-extension-data> and cases contain the link named <http://cbrain.com/casefile/rel/update-case-extension-data>. In order to update a

field, perform an `POST` request to the link. The Post Once Exactly pattern (see [Data manipulation](#)) is not required here as the same `POST` operation can be performed more than once, to no effect.

The request body should be formatted as XML. The request body should contain a list of name-value pairs. The name-value list is merged into the existing extension fields.

Extension fields update example JSON

```
POST update-extension-url HTTP/1.1
Content-Type: application/json

{
  "Fields":[
    {
      "Name":"FieldName1",
      "Value":"FieldValue1"
    },
    {
      "Name":"FieldName2",
      "Value":"FieldValue2"
    }
  ]
}
```

Additional properties

Data in F2 may also be enriched by associating collections of simple key/value pairs with various objects (usually parties).

A person (external party) may for instance be related to a job interview and in this context, we need to store the applicant's date of birth. This can be stored as a key/value combination where the key is "DateOfBirth" and the value is the actual date of birth.

Additional properties resource

This resource contains a set of key/value properties that represents additional fields as used in F2's party management.

The key/value properties are represented as `application/x-www-form-urlencoded` values with UTF-8 encoding.

Date values should be formatted according to RFC 3339 "Date and Time on the Internet: Timestamps". Like for instance `1985-04-12T23:20:50.52Z`.

Location

The additional properties are located by following the link relation `http://cbrain.com/casefile/rel/field-data` from a matter or case.

Properties

There are no predefined properties. All stored properties are returned as `application/x-www-form-urlencoded`.

Additional properties example

```
Submitted_ByName=Frank+Monacho&
Submitted_ByPhone=12345678&
Submitted_ByMail=example%40cbrain.com
```

Update properties

Issue a `POST` to the additional properties resource to add or update fields. The body should contain an `application/x-www-form-urlencoded` encoded list of key/value pairs to represent the additional properties which should be updated. See the additional properties example above.

Replace all properties

Issue a `PUT` to the additional properties resource to clear all fields and add new ones. The body should contain an `application/x-www-form-urlencoded` encoded list of key/value pairs to represent the new properties. See the additional properties example above.

```
Searching
= Filtered case search
```

Case search resource

This resource represents a search engine for searching cases by various properties.

Location

The case search resource can be found by following the link relation <http://cbrain.com/casefile/rel/case-search-filter> from the service index.

A GET on the base URL returns an HTML form for testing the queries.

Input parameters

Table 108. Input query parameters for case search

Name	Description
caseNumber	string
id	int
title	string
cprNumber	string
externalId	string

The search result can be represented as either an ATOM feed [ATOM], CSV [CSV] or JSON [JSON] as controlled by the standard HTTP content negotiation mechanism using the "Accept" header.

Search request example

```
GET case-search-url?cprNumber=311299-1234 HTTP/1.1
Accept: text/csv
```

Output values

Table 109. Output values for case search

Name	Description
Id	int
CaseNumber	string
Title	string
CprNumber	string
ExternalId	string
Created`	DateTime
Updated`	DateTime
alternate	URL Link to case in F2-REST.

Search response example

```
HTTP/1.1 200 OK
Content-Type: text/csv
```

```
Id,CaseNumber,Title,CprNumber,ExternalId,Created,Updated,alternate
"279666","2022 - 1172","A test case for case search","311299-1234","P2103568","2022-03-29T11:00:58","2022-03-29T11:00:58","https://case-url"
```

Filtered matter search

Matter search resource

This resource represents a search engine for searching matters by various properties.

Location

The matter search resource can be found by following the link relation `http://cbrain.com/casefile/rel/matter-search-filter` from the service index.

A `GET` on the base URL returns an HTML form for testing the queries.

Input parameters

Table 110. Input query parameters for matter search

Name	Description
matterNumber	int
id	int
title	string
caseNumber	string
cprNumber	string

The search result can be represented as either an ATOM feed [[ATOM](#)], CSV [[CSV](#)] or JSON [[JSON](#)] as controlled by the standard HTTP content negotiation mechanism using the `Accept` header.

Search request example

```
GET matter-search-url?id=&matterNumber=&title=&caseNumber=2022-1170&cprNumber= HTTP/1.1
Accept: text/csv
```


Output values

Table 111. Output values for matter search

Name	Description
Id	int
MatterNumber	int
CaseNumber	string
CaseTitle	string
CprNumber	string
Type	string
AccessLevel	string
Deadline	DateTime
Archived	bool
Status	DateTime
Created`	DateTime
Updated`	DateTime
alternate	URL Link to matter in F2-REST.

Search response example

```
HTTP/1.1 200 OK
Content-Type: text/csv
```

```
Id,MatterNumber,Title,CaseNumber,CprNumber,Type,AccessLevel,Deadline,Archived,Status,Created,U
pdated,alternate
"279655","13249","Test dossier for case 279654","2022 - 1170","","","","","","","2022-03-
29T10:44:18","2022-03-29T10:44:19","https://matter-url"
```

OpenSearch document

OpenSearch [[OpenSearch](#)] is a format for describing the web interface of a search engine — in this case the full text search engine of F2. The format is rather simple — at the core of it there is a URL template where the search text can be merged into, whereafter the client should perform a GET of the resulting URL. The server will then return the search result as an ATOM feed [[ATOM](#)].

Table 112. URL template variables for OpenSearch

Name	Description
{searchTerms}	string The string to search for.
{count}	int The number of returned search results.

OpenSearch document example

```
<OpenSearchDescription xmlns="http://a9.com/-/spec/opensearch/1.1/">
  <ShortName>Matter search</ShortName>
  <Description>Searching by any text for matters in F2</Description>
  <Author>cBrain</Author>
  <Contact>cbrain@cbrain.dk</Contact>
  <Url type="application/atom+xml"
    template="http://example.com/search?q={searchTerms}&count={count}" />
</OpenSearchDescription>
```

Matter search

To get the OpenSearch description document for searching for matters, follow the link relation <http://cbrain.com/casefile/rel/matter-search>.

Case search

To get the OpenSearch description document for searching for matters, follow the link relation <http://cbrain.com/casefile/rel/case-search>.

Custom queries

Custom query resource

This resource represents a customized way to do searches. This may be useful when the search is more complex and requires detailed knowledge of F2 internal resources that are not part of the REST API abstraction.

Usage

A custom query is build up from a base URL with a query name and added query parameters. The parameter profile for a given query is defined by a Custom Query description document (as documented in the internal F2-REST installation manual). The installed custom queries are listed in the custom query index (found in the service index by looking for the link relation <http://cbrain.com/casefile/rel/custom-query-index>).

If we assume a base URL of <http://example.com/custom-queries/THEQUERY> for the custom query named THEQUERY then the search can be executed with parameters P1 = value1 and P2 = value2 using the URL <http://example.com/custom-queries/THEQUERY?P1=value1&P2=value2>.

A GET on the base URL returns an HTML form for testing the query.

The search result can be represented as either an ATOM feed [[ATOM](#)], CSV [[CSV](#)], XML, JSON or HTML as controlled by the standard HTTP content negotiation mechanism using the "Accept" header.

A custom query named THEQUERY is installed in a designated folder in the F2 REST installation in a custom query description document called THEQUERY.query.xml.

CSV example output

In CSV the column headers matches column names in SQL:

```
Bool,Int,Number1,Number2,String1,String2,Date,DateTime,Null
"1","2","3.3","4.4","Abcd","Xyz","2025-08-05T00:00:00","2025-08-05T17:22:45",""
"1","2","3.3","4.4","Abcd","Xyz","2025-08-05T00:00:00","2025-08-05T17:22:45",""
```

JSON example output

In JSON the output is an array of objects where each object has property names that matches the SQL column names:

```

{
  "value": [
    {
      "Bool": 1,
      "Int": 2,
      "Number1": 3.3,
      "Number2": 4.4,
      "String1": "Abcd",
      "String2": "Xyz",
      "Date": "2025-08-05T00:00:00+02:00",
      "DateTime": "2025-08-05T17:22:45+02:00",
      "Null": null
    },
    {
      "Bool": 1,
      "Int": 2,
      "Number1": 3.3,
      "Number2": 4.4,
      "String1": "Abcd",
      "String2": "Xyz",
      "Date": "2025-08-05T00:00:00+02:00",
      "DateTime": "2025-08-05T17:22:45+02:00",
      "Null": null
    }
  ]
}

```

XML example output

In XML the the output is a list of `<Item>` elements with sub-elements that matches the SQL column names:

Custom query as self-service Web Action target

This resource represents a custom query as described above. It does however transform the query result into an XML format which the self-service web action expects.

This version of the query can be accessed in the same manner as normal, and can also take Post request, with either Url-Form or JSON encoded data. The URL segment "custom-queries" should also be replaced with "selfservice-webaction". It is important to note that not all custom queries should be accessed through this URL, as it expects the output to only have one row, which at the very least has the columns: "Success" and "Message". "Success" should be a Boolean/bit value, which informs the Web Action whether or not the operation was a success. "Message" is an error message to be shown in case of failure. Every column besides these two are treated as self-service properties, which will be updated with the corresponding row value. A value of NULL is however ignored and the corresponding field is not updated.

When used as a web action, the custom query supports JWT authorization. Refer to the self-service manual for information related to the expected form of the authorization token.

The custom query will only accept JWT validation if the custom query authorization attribute has the value "WebActionJwt".

The JWT token is expected to have the values:

```
iss = http://cbrain.com/f2/self-service/  
sub = http://cbrain.com/f2/rest/
```

The token is only considered valid if the server time has not exceeded the time defined by exp.

Custom query as taskguide Webpush target

This resource represents a custom query as described [here](#). Similarly to self-service web action queries, the result of such a query transforms the result to JSON in a format suitable for the task guide.

This version of the query can be accessed in the same manner as normal with the URL segment "custom-queries" replaced with "taskmanagement-webpush". It is important to note that not all custom queries should be accessed through this URL, as it expects the output to only have one row, which at the very least has the columns: "Success" and "Message". "Success" should be a Boolean/bit value, which informs the Web Action whether or not the operation was a success. "Message" is an error message to be shown in case of failure. Every column besides these two are treated as taskguide fields, which will be updated with the corresponding row value. A value of NULL is however ignored and the corresponding field is not updated.

When used as a web push, the custom query supports JWT authorization. Refer to the task guide manual for information related to the expected form of the authorization token.

The custom query will only accept JWT validation if the custom query authorization attribute has the value "WebActionJwt".

Receiver = <http://cbrain.com/f2/rest/>

11.7 Custom query as self-service validator

F2-REST implements the "external validator" interface of F2 Self Service.

As an example, F2-REST may be used by F2 Self-service as a validator to check if a certain user is allowed to submit a Self Service form. F2 Self-service will then use a self-service validator resource of F2-REST. The result of the validator indicates if the user can continue or if the F2 Self-service client should show a message to the user about the failed validation.

F2 Self-service validators are used in two contexts: 1) For login validation where a message is shown on the login page, and 2) for page validation where a message is shown in a list together with other kinds of validation messages e.g. for empty mandatory fields, etc.

A self-service validator is a specialized search using custom queries. The custom query of a self-service validator must always return the following properties:

Table 113. Self-service response properties

Name	Description
IsValid	int Validation result represented by 0 (failure) or 1 (success).
Code	string Some custom code describing the validation result. It could, for instance, describe the reason for failure.
Message	string A message suitable for presentation to a self-service user after performing the validation. For login validators the message should be html formatted as it will be included on an html web page. For page validators the message should be a single line of plain text as it will be included in a standard list of validation errors.

In order to use a custom query as a self-service validator, the resource URL should be slightly different than the corresponding custom query URL. For example, if the custom query resource URL is <http://example.com/custom-queries/THEQUERY> then the Self Service validator resource URL is <http://example.com/selfservice-validator/THEQUERY>.

A POST on the validator URL returns a validator result (see above) in XML format.

The input parameters to the validator are specified in the POST content as multipart/form-data. POST headers to use:

```
Accept: application/xml
Content-Type: multipart/form-data
```

As for ordinary custom queries, an additional parameter named Authorization may be added in the POST content (thus this name cannot be used for other parameters). This is used to authorize the

client. The value must match the AuthorizationSecret specified in the descriptor document for the corresponding installed self-service validator.

A self-service validator named THEQUERY is installed in the same place as custom queries in a designated folder in the F2-REST installation in a custom query description document called THEQUERY.query.xml.

Custom query for auto completer

Both F2 Self-service and F2 task guides have an external auto completer feature (read more about them in their manual). F2-REST implements a handle to use a custom query for those auto completers. It works the same way as normal custom queries, however the URL segment "custom-queries" should however be replaced with "external-auto-completer".

The result is expected to have the columns "Display" and "Value". See for instance the custom query "auto-completer-get-enum.query.xml" for an example of how to use it for journal plans.

Other resources
= Simple collection

Simple collection resource

This resource represents a list of items.

A collection is represented by 1) a set of aggregated collection properties and 2) a list of the items in the collection.

The list items are represented by their title together with links to more detailed information.

The root element is `<Collection>`. This element contains the following aggregated collection properties:

Table 114. Collection properties

Name	Description
Title	string The title of the collection.
Link	List of Link One or more links to other related resources.

The item collection is marked with an `<Items>` element and contains zero or more `<Item>` elements.

Each `<Item>` element contains the following properties:

Table 115. Item properties

Name	Description
Id	string An identifier for the item.
Title	string The title of the item.
Link	List of Link One or more links to other related resources.

The XML root element is `<Collection>`.

Example collection

```
<Collection xmlns="http://cbrain.com/casefile/schema/">
  <Title>Documents for matter 'Borgerhenvendelse vedrørende Bjørneklo'</Title>
  <Link href="..."
        rel="self"
        type="application/vnd.cbrain.casefile.collection+xml"
        title="Documents" />
  <Link href="..."
        rel="up"
        type="application/vnd.cbrain.casefile+xml"
        title="Matter" />
  <Items>
    <Item>
      <Id>378053</Id>
      <Title>DossierDocument</Title>
      <Link href="..."
            rel="self"
            type="application/vnd.cbrain.casefile+xml"
            title="DossierDocument" />
    </Item>
    <Item>
      <Id>378054</Id>
      <Title>Photo2.gif</Title>
      <Link href="..."
            rel="self"
            type="application/vnd.cbrain.casefile+xml"
            title="Photo2.gif" />
    </Item>
  </Items>
</Collection>
```


Meetings

Meeting resource

This resource represents a single meeting in F2.

Location

Meetings can only be found by URLs supplied other places in F2 (Manager). This may be improved on in later versions.

Properties

Table 116. Meeting properties

Name	Description
Id	int (read-only) Internal identifier.
MeetingNumber	string (read-only) Public visible meeting number (for instance "2012 - 341").
Title	string Meeting title.
Category	Enumeration Meeting category.
Description	string Description of meeting.
Active	bool Active status of meeting.
MeetingStart	DateTime Start of meeting.
MeetingEnd	DateTime End of meeting.
VisibilityStart	DateTime Start of visibility period.
VisibilityEnd	DateTime End of visibility period.
Participants	List of PartyItem named Party Meeting participants.
Stakeholders	List of PartyItem named Party Meeting stakeholders.
SecurityGroups	List of PartyGroupItem named PartyGroup The security groups, users or organizations that are part of the access restrictions for the meeting. The parties are accessed by their name when patching and creating meetings.
Link	List of Link One or more links to other related resources.

The XML root element is `<Meeting>`.

Meeting example

```
<Meeting xmlns="http://cbrain.com/casefile/schema/">
  <Id>386596</Id>
  <MeetingNumber>2013 - 45</MeetingNumber>
  <Title>My meeting</Title>
  <Category>
    <Title>Kultur- og Fritidsudvalget</Title>
    <Code>KUFR</Code>
  </Category>
  <Description>Blah blah blah ...</Description>
  <Active>true</Active>
  <MeetingStart>2013-02-20T15:00:00</MeetingStart>
  <MeetingEnd>2013-02-20T16:00:00</MeetingEnd>
  <VisibilityStart>2013-02-20T00:00:00</VisibilityStart>
  <VisibilityEnd>2013-03-22T00:00:00</VisibilityEnd>
  <Participants>
    <Party>
      <Name>Karin Ingemann Petersen</Name>
      <EMail />
      <Link href="..."
        rel="self"
        type="application/vnd.cbrain.casefile+xml"
        title="Self" />
    </Party>
  </Participants>
  <Stakeholders>
    <Party>
      <Name>Brian Ipsum Hansen</Name>
      <EMail />
      <Link href="..."
        rel="self"
        type="application/vnd.cbrain.casefile+xml"
        title="Self" />
    </Party>
  </Stakeholders>
  <Link href="..."
    rel="self"
    type="application/vnd.cbrain.casefile+xml"
    title="My meetings" />
  <Link href="..."
    rel="http://cbrain.com/casefile/rel/related-case"
    type="application/vnd.cbrain.casefile+xml"
    title="Related case" />
</Meeting>
```

Create new meeting

A meeting may be created using the Post Once Exactly pattern (see [Data manipulation](#)). Follow the link named <http://cbrain.com/casefile/rel/create-meeting> from the service index to locate the POE resource for creating a meeting.

Table 117. Input parameters for creating a meeting

Name	Description
Title	string (255 characters) Meeting title.
Category	string Meeting category reference (external ID or abbreviation of enumeration item).
Description	string Description of meeting.
Active	bool Active status of meeting.
MeetingStart	DateTime Start of meeting.
MeetingEnd	DateTime End of meeting.
VisibilityStart	DateTime Start of visibility period.
VisibilityEnd	DateTime End of visibility period.
Participants	List of PartyReference Meeting participants.
Stakeholders	List of PartyReference Meeting stakeholders.
SecurityGroups	List of PartyGroupItem named PartyGroup The list of security groups, users or organizations to include in the access restrictions of the new meeting. The security groups are referenced by the name shown in the F2 desktop client.
KeywordCode	string (read-only) The code that represents a keyword to be associated with the new meeting. This can either be in the format of <code>##.##.##</code> when used with keywords from "Kommunernes Landsforening" — or the complete path to a specific keyword name (as seen from the desktop client) or the external ID of the keyword. As the complete paths and external ID's may overlap, the external ID takes precedence.
KeywordCodes	List of string (read-only) A list of keyword codes. Each keyword code is handled in the same way as the above singular <code>KeywordCode</code> .

Example request for creating a meeting (include first POE step)

```
POST /meeting-create-url HTTP/1.1
Content-Length: 0

> HTTP/1.1 201 Created
> Location: /meeting-create-poc-url
> Content-Length: 0

POST /meeting-create-poc-url HTTP/1.1
Content-Type: application/x-www-form-urlencoded
Content-Length: 1292

Title=A+new+meeting&
Category=TEKM&
Active=true&
Description=Description+related+to+the+new+meeting&
MeetingStart=2013-01-03T16%3a36%3a00&
MeetingEnd=2013-01-03T19%3a36%3a00&
VisibilityStart=2012-12-04T16%3a36%3a00&
VisibilityEnd=2013-02-02T16%3a36%3a00&
Participants%3a0.PartyNo=2000001001&
Participants%3a0.OrganizationNo=2000000001&
Stakeholders%3a0.Login=Sagsbehandler24&
Stakeholders%3a0.PickFirstMatch=false&
SecurityGroups%3a0.Name=SG4&
SecurityGroups%3a1.Name=SG5&
KeywordCode=00.07.00&
DeletionActionCode=B&
ProcessInstructionCode=G10
```

Modify meeting

A meeting may be modified by issuing an HTTP PATCH request targeted at the meeting resource URL.

The simple properties are straight forward to replace with similar typed values, but some of the complex meeting properties need special encoding when patching. These properties are:

Table 118. Properties needing special encoding when patching

Name	Description
Participants	Use an array of PartyReference objects to reference the required parties.
Stakeholders	Use an array of PartyReference objects to reference the required parties.
SecurityGroups	Use an array of PartyGroupReference objects to reference the required parties.

Expected link relations

self

<http://cbrain.com/casefile/rel/related-case>

Link to "shadow case" for the meeting.

Submission event log

Details about submissions from F2's self-service application can be stored as submission event log entries in the database. This is primarily for internal use and as such there is no way to read existing log entries via F2-REST.

Create submission event log entry

Follow the link relation <http://cbrain.com/casefile/rel/create-submission-event-log> from the service index to find a resource for creating submission event log entries. Then issue an HTTP POST request to it with the following parameters

Table 119. Input parameters for creating submission event log entry

Name	Description
IpAddress	string The IP-address of the device making the submission.
UserAgent	string The user's user agent provided by the user's browser.
Identity	string The identity of the user making the submission.
SubmissionKey	string The ID of the submission.
UrlReferrer	string The URL from which the user made the submission.

Configuration

Client configuration

Each client accessing F2-REST needs to be registered first. This is done by adding a <Client> entry to the file ClientValidationRules.config.

Registering a client means we assign a Client ID and Client Secret for the client, which allows it to access F2-REST, and then configure which features of F2-REST the client is allowed to use.

A typical <Client> entry for F2-REST may look something like this:

```
<?xml version="1.0" encoding="utf-8" ?>
<Clients xmlns="http://cbrain.com/oauth2/clientvalidation/schema">
  <Client>
    <!-- Client ID (choose one) -->
    <Id>ThirdPartyIntegrator</Id>
    <!-- Client secret (generate a secure random secret string) -->
    <Secret>968hjads01xzfwjrh74aafd3lsufpr</Secret>
    <!-- Client description (for internal book keeping) -->
    <Description>Client for blah blah ...</Description>
    <!-- OAuth2 specific: client ID in payload-body or header -->
    <AllowClientIdInBody>false</AllowClientIdInBody>
    <!-- Require F2 user password when authenticating (often false) -->
    <RequireUserPassword>false</RequireUserPassword>
    <!-- List of users this client is allowed to authenticate as -->
    <Users>
      <!-- User name -->
      <User>Lise Nielsen</User>
    </Users>
    <!-- Allow client to authenticate as all possible F2 users -->
    <AllowAllUsers>false</AllowAllUsers>
    <!-- Allow authentication using authorization code from Windows AD SSO -->
    <AllowAuthorizationCode>false</AllowAuthorizationCode>
    <!-- Allowed redirect URLs for Windows AD Single Sign On -->
    <RedirectUrls>
      <Url>...some URL to client callback resource ...</Url>
    </RedirectUrls>
    <!-- Enable login with F2 session identifier (used by desktop clients) -->
    <AllowF2SessionId>false</AllowF2SessionId>
  </Client>
</Clients>
```

See the separate configuration file format manual “Client validation rules – configuration files” for further information.

Authenticating with Windows AD

F2-REST can be configured to authenticate users against Windows AD (similar to the F2-Desktop Client), such that a third-party service may operate on behalf on any known domain user. The actual flow is described in the normal F2-REST manual.

To enable authentication using Windows AD, the following must be configured in F2-REST Web.config:

1. Activate use of F2-AuthenticateService by setting `OAuth2.UseAuthenticationServer` to true.
2. Make sure `OAuth2.AuthenticationServerUrl` refers to F2-AuthenticateService’s authorization URL (which is `<f2auth-root>/oauth2/authorize` where `<f2auth-root>` is the root directory of F2-AuthenticateService). If available, this should be the F2-AuthenticateService configured for single sign on.
3. Make sure `AuthenticationServerUrl` refers to the web interface of F2-AuthenticateService (which is `<f2auth-root>/F2AuthenticateService.asmx`). If available, this should be the F2-AuthenticateService configured for **anonymous** access.
4. Make sure `RequiredDomain` is set to the correct AD domain.

5. Make sure the configuration file `ClientValidationRules.config` is identical on both the F2-REST server and the F2-AuthenticateService.

OpenID Connect with AD Single Sign On

F2-REST also supports OpenID Connect (OIDC, see <http://openid.net/connect/>) which allows clients to verify the identity of the end-user based on the authentication performed by an external authorization server, as well as to obtain basic profile information about the end-user in an interoperable and REST-like manner.

Setting up OIDC requires one more OAuth2 setting in the `web.config` file:

1. Make sure `OAuth2.OpenIDTokenServiceUrl` refers to F2-AuthenticateService's token service URL (which is `<f2auth-root>/oauth2/tokenservice` where `<f2auth-root>` is the root directory of F2-AuthenticateService).

Please note that F2-REST does not function as an Identity Provider in itself, but delegates the operation to F2-AuthenticateService.

Browser login

F2-REST allows developers to browse through the REST API using a standard web-browser.

The default mechanism for logging in using the browser is "HTTP Basic Authentication" where the browser itself asks the end-user for his/her credentials and then transfer those credentials in an HTTP header for F2-REST to verify.

It is although also possible to use federated login with OpenID Connect, where F2-REST redirects the browser to some external identity provider where the end-user authenticates. This allows the end-user, typically a developer, to authenticate using for instance Windows AD (through cBrain's IdentityHub) or other OIDC compliant identity providers.

Federated login using OpenID Connect must be configured as described in the next section.

Federated login with OpenID Connect

In order to use OpenID Connect for federated login in to F2-REST, it is necessary to configure the identity provider in the file `IdentityProviders.config` found in the root of the F2-REST installation directory.

Once the identity provider configuration is done and the application setting `OAuth2.DefaultExternalIdentityProvider` is added to `web.config` and references the right identity provider, the service index page at `<F2-rest-root>/serviceindex` adds "Login"-link at the top, allowing the end-user to login through a browser.

The identity provider configuration is also used when a client tries to authenticate with a JWT identity token it has received from an external identity provider.

Rate Limiting

It is possible to enable Rate Limiting in F2-REST. A Rate Limit is the number of API calls an app or user can make within a given time period. If this limit is exceeded the app or user may be throttled. In F2-REST requests can be limited for individual Client IDs.

Rate Limiting is enabled by adding a Rate Limit rule section to the F2-REST web.config file:

```
<RateLimit>
  <rules>
    <rule entry="CientId1" limitPerMinute="5000" limitPerSecond="100" />
    <rule entry="CientId2" limitPerHour="50000" />
  </rules> +
</RateLimit>
```

Each rule should have an "entry" attribute containing the Client ID for which the rule applies. If the value is "*" the rule will be applied to all Client IDs individually, i.e. it corresponds to adding the same rule for each Client ID. If no "entry" attribute is specified the rule is ignored.

Each rule can have one or more limit attributes: "limitPerSecond", "limitPerMinute", "limitPerHour", "limitPerDay" and "limitPerWeek". The value of these indicate the number of requests allowed within the time period.

If a request is received that violates a rate limit rule, F2-REST will deny the request and return a response with status code 429 ("Too Many Requests").

Note that request history is not persisted, so restarting F2-REST will result in a reset of rate limiting.

Virus scanning of uploaded documents

F2-REST is able to scan every document received and replace it with a warning document, should the document contain a virus.

Please see the "F2 Security Server installation manual" for details about installing virus scanning.

The F2 Security Server itself is not required for virus scan of documents in F2-REST, so that part can be ignored - it is only the core virus scanning service that needs to be accessible.

When the virus scanning service is up and running, you should change the following app-settings in the F2-REST web.config file:

Setting	Description
AntiVirus.VirusFoundTemplate	The path to the warning template file, which should replace an uploaded file, should it contain a virus. The file must be a Word-file. A default template is already included as a part of the installation. This template can be found in the root of the F2-REST-folder. If you remove the '_' in the start of the filename, the default of this app-setting will match the file. The file can contain the following merge-fields: • \$definitionFileVersion\$ • \$threatName\$ • \$scannerIdentifier\$ • \$md5\$ • \$fileExtension\$
AntiVirus.UseAntiVirusInRest	A Boolean setting. If true, every document will be scanned. If true and the anti-virus service is not running, then F2-REST will return an HTTP error code 500.
AntiVirus.Engine	Name of a DLL containing the virus-scanning engine to be used.

Currently the only supported virus-scanning engine DLL is "CBrain.AntiVirus.ClamAV.ClamAVScanner,CBrain.AntiVirus.ClamAV".

Bibliography

- [Fielding] Fielding, Roy Thomas. Architectural Styles and the Design of Network-based Software Architectures. Irvine, California : University of California, Irvine, 2000.
- [OIDC] OpenID Connect. OpenID. [Online] <http://openid.net/connect/>.
- [OpenSearch] OpenSearch specification 1.1. OpenSearch.org. [Online] <http://www.opensearch.org/Specifications/OpenSearch/1.1>.
- [JSONPatch] RFC 6902 - JavaScript Object Notation (JSON) Patch. IETF. [Online] 2013. <https://tools.ietf.org/html/rfc6902>. RFC 6902.
- [ATOM] RFC 4287 - The Atom Syndication Format. IETF. [Online] 2005. <http://tools.ietf.org/html/rfc4287>. RFC 4287.
- [JSON] RFC 4627 - The application/json Media Type for JavaScript Object Notation (JSON). IETF. [Online] 2006. <http://tools.ietf.org/html/rfc4627>. RFC 4627.
- [MultipartFormdata] RFC 2388 - Returning Values from Forms: multipart/form-data. IETF. [Online] 1998. <http://www.ietf.org/rfc/rfc2388.txt>. RFC 2388.
- [OAuth] RFC 6749 - The OAuth 2.0 Authorization Framework. IETF. [Online] 2012. <https://tools.ietf.org/html/rfc6749>. RFC 6749.
- [XSD] W3C XML Schema Definition Language (XSD) 1.1 Part 2: Datatypes. W3C. [Online] 2012. <http://www.w3.org/TR/2012/REC-xmlschema11-2-20120405/>.
- [DATETIME] RFC 3339 - Date and Time on the Internet: Timestamps. IETF. [Online] 2002. <http://tools.ietf.org/html/rfc3339>. RFC 3339.
- [CSV] RFC 4180 - Common Format and MIME Type for Comma-Separated Values (CSV) Files. IETF. [Online] October 2005. <http://tools.ietf.org/html/rfc4180>. RFC 4180.
- [JWT] RFC 7519 - JSON Web Token (JWT). IETF. [Online] 2015. <https://tools.ietf.org/html/rfc7519>.
- [OAuthTokenInspection] RFC 7662 - OAuth 2.0 Token Introspection. IETF. [Online] 2015. <https://tools.ietf.org/html/rfc7662>
- [XMLTypes] W3C XML Schema Definition Language (XSD) 1.1 Part 2: Datatypes [Online 2012] <https://www.w3.org/TR/xmlschema11-2/>.
- [XMLSchema] W3C XML Schema Definition Language (XSD) 1.1 Part 1: Structures [Online] 2012. <https://www.w3.org/TR/xmlschema11-1/>.